

Corrigés des exercices

Méthodes numériques pour physiciens et ingénieurs

cours de P. Wofo (Université de Yaoundé I)

Corrigés rédigés à partir du cours et de
R. L. Burden, J. D. Faires, A. M. Burden, *Numerical Analysis*,
10^e édition, Cengage Learning, 2016.

Avant-propos

Ce document rassemble les corrigés de tous les exercices du cours « Méthodes numériques pour physiciens et ingénieurs » : les exercices placés dans les chapitres I à VI, puis les dix-neuf exercices de la fin du document.

Comment chaque corrigé est construit

Chaque corrigé suit le même plan, dans cet ordre :

1. **Pourquoi ?** On explique d'abord l'idée : quel est le problème, pourquoi la méthode choisie convient, quelle difficulté elle résout et à quelles conditions elle marche.
2. **Comment ? (étapes)** On déroule ensuite la solution étape par étape : mise en équations, calculs intermédiaires, puis algorithme et, quand l'énoncé le demande, programme.
3. **Vérification numérique.** Chaque résultat chiffré et chaque algorithme ont été réellement calculés ou exécutés. Les valeurs données sont celles que ces calculs ont produites.
4. **Références.** On indique où trouver la méthode dans le livre de référence ou dans une source publique.

Lorsqu'un énoncé est incomplet, ambigu ou semble contenir une erreur, on le dit dans un paragraphe « Remarque sur l'énoncé », et on précise l'interprétation retenue. Rien n'est supposé sans être signalé.

Références utilisées

La référence principale est le livre R. L. Burden, J. D. Faires et A. M. Burden, *Numerical Analysis*, 10^e édition, Cengage Learning, 2016, noté [BF]. Les renvois du type [BF, alg. 6.1, p. 368] donnent le numéro de l'algorithme ou du théorème et la page imprimée du livre. Les autres sources (articles, encyclopédies en ligne) sont listées dans la bibliographie à la fin du document.

Programmes et calculs

Les énoncés demandent parfois de traduire les algorithmes en Pascal, en Fortran ou en C. Tous les programmes sont donnés ici en **Python** (version 3, sans bibliothèque externe). Ils se trouvent dans le dossier `latex-corriges/programmes/` : chaque fichier contient les fonctions qui traduisent l'algorithme du corrigé puis, à la fin (après `if __name__ == "__main__":`), l'exemple utilisé dans la vérification. Le script `tester.sh` les exécute tous (résultats dans `calculs/programmes_resultats.txt`). Les calculs numériques et les tests d'algorithmes sont dans `latex-corriges/calculs/` (Python avec NumPy et SciPy) : `calculs.py`, `complements.py` et `tests_algorithmes.py`, avec leurs sorties dans les fichiers `*_resultats.txt` et `resultats.txt`.

Dans les algorithmes, la flèche $\leftarrow$ désigne l'affectation, et les notations sont celles du cours.

Table des matières

Avant-propos	1
Comment chaque corrigé est construit	1
Références utilisées	1
Programmes et calculs	1
Chapitre I : systèmes d'équations algébriques linéaires	4
Exercice I.1 : résolution du système triangulaire	4
Exercice I.2 : échange de lignes (pivot maximal)	5
Exercice I.3 : méthode de Gauss complète avec pivot maximal	6
Exercice I.4 : calcul des l_{ij} et des s_{ij}	7
Exercice I.5 : inversion d'une matrice et produit $L^{-1}b$	8
Exercice I.6 : coefficients m_{pp} et m_{pj} de Cholesky	9
Exercice I.7 : Jacobi et Gauss-Seidel avec condition d'arrêt	10
Chapitre II : équations algébriques non linéaires	12
Exercice II.1 : Newton-Raphson avec condition d'arrêt	12
Exercice II.2 : méthode de bisection	13
Exercice II.3 : schéma de Lin pour les racines réelles	14
Chapitre III : interpolation et moindres carrés	16
Exercice III.1 : programme d'interpolation de Lagrange	16
Complément : formules de a_0 et a_1 pour la droite des moindres carrés	17
Exercice III.2 : calcul des coefficients b_k et A_{kj}	17
Exercice III.3 : moindres carrés sur un tableau de mesures	18
Exercice III.4 : interpolation de Lagrange directe et inverse	19
Chapitre IV : intégration et dérivation numériques	20
Exercice IV.1 : dérivation à partir de l'interpolation quadratique	20
Exercice IV.2 : vitesses à partir d'un tableau de positions	20
Exercice IV.3 : trapèzes à la main pour $\int_1^2 dx/x$	21
Exercice IV.4 : trapèzes sans tableaux	22
Exercice IV.5 : algorithme de Simpson	22
Exercice IV.6 : formule d'intégration de degré 3 sur a, c, d, b	23
Exercice IV.7 : Simpson à la main pour $\int_0^1 4/(1+x^2) dx$	24
Chapitre V : équations différentielles et intégro-différentielles	25
Exercice V.1 : algorithme de Runge-Kutta d'ordre 2	25
Exercice V.2 : algorithme de Runge d'ordre 4	25
Exercice V.3 : algorithme d'Adams-Bashforth d'ordre 2	26

Exercice V.4 : prédicteur-correcteur AB2 – RK2 et programme	27
Exercice V.5 : algorithme de la méthode de tir	28
Exercice V.6 : discrétisation de l'équation intégral-différentielle	29
Chapitre VI : équations aux dérivées partielles	31
Exercice VI.1 : Gauss-Seidel et Jacobi pour l'équation de Poisson	31
Exercice VI.2 : algorithme du schéma explicite (chaleur)	32
Exercice VI.3 : stabilité de Crank-Nicolson et de Dufort-Frankel	33
Exercice VI.4 : algorithmes régressif, Crank-Nicolson, Dufort-Frankel	34
Exercice VI.5 : équation d'ondes, stabilité et algorithme	36
Exercices de fin de document (1 à 7)	38
Exercice 1 : maximum de la loi de Planck	38
Exercice 2 : volume d'un gaz de Van der Waals	39
Exercice 3 : facteur de compressibilité (De Santis)	40
Exercice 4 : $y' = -ay - by^4 + c$ par prédicteur-correcteur	41
Exercice 5 : $y'' + \frac{a}{x}y' + be^{cy} = 0$ par la méthode de tir	42
Exercice 6 : potentiel entre cathode et anode, $V'' = aV^{-1/2}$	44
Exercice 7 : équation du troisième ordre par la méthode de tir	47
Exercices de fin de document (8 à 13)	49
Exercice 8 : différences finies et méthode de Jacobi	49
Exercice 9 : oscillateur de Duffing forcé et amorti	51
Exercice 10 : méthode de Gauss	53
Exercice 11 : le pendule simple	55
Exercice 12 : capacité calorifique et chaleur absorbée	58
Exercice 13 : équation intégrale de Fredholm	60
Exercices de fin de document (14 à 19)	62
Exercice 14 : température dans un mur	62
Exercice 15 : conditions de stabilité	65
Exercice 16 : équation d'ondes amortie et non linéaire	66
Exercice 17 : pression dans la paroi d'un tube	70
Exercice 18 : équation de Poisson sur une plaque entaillée	72
Exercice 19 : équation de Laplace sur une plaque triangulaire	73
Bibliographie	75

Chapitre I : systèmes d'équations algébriques linéaires

Exercice I.1 : résolution du système triangulaire

Énoncé. Écrire l'algorithme de résolution du système algébrique triangulaire.

Pourquoi ? Après la triangularisation de Gauss, la dernière équation ne contient plus qu'une seule inconnue, x_n . L'avant-dernière contient x_{n-1} et x_n , et ainsi de suite : l'équation k ne fait intervenir que $x_k, x_{k+1}, \dots, x_n$. Si l'on résout les équations *de bas en haut*, chaque équation n'a donc plus qu'une seule inconnue nouvelle, x_k , car $x_{k+1}, \dots, x_n$ ont déjà été calculés. C'est la « remontée » (substitution arrière). Elle exige $a_{kk} \neq 0$ pour tout k , ce que la triangularisation avec pivot garantit lorsque la matrice est régulière.

Comment ? (étapes)

Étape 1. La dernière équation $a_{nn}x_n = b_n$ donne directement $x_n = b_n/a_{nn}$.

Étape 2. Pour $k = n - 1$, puis $n - 2, \dots$, jusqu'à 1 (ordre décroissant, c'est indispensable), l'équation k s'écrit

$$a_{kk}x_k + \sum_{l=k+1}^n a_{kl}x_l = b_k,$$

où toutes les x_l de la somme sont déjà connues. On calcule d'abord la somme $s = b_k - \sum_{l=k+1}^n a_{kl}x_l$, puis $x_k = s/a_{kk}$. C'est exactement la forme condensée du cours.

Étape 3. Coût : l'étape k demande $n - k$ multiplications et une division, soit au total $1 + \sum_{k=1}^{n-1} (n - k + 1) = (n^2 + n)/2$ multiplications ou divisions [BF, p. 371]. C'est négligeable devant le coût de la triangularisation (de l'ordre de $n^3/3$).

Algorithme : remontée

Données : n , la matrice triangulaire supérieure (a_{kl}) , le second membre (b_k)

$x_n \leftarrow b_n/a_{nn}$

Pour k allant de $n - 1$ à 1 (pas -1) **faire**

$s \leftarrow b_k$

Pour l allant de $k + 1$ à n **faire**

$s \leftarrow s - a_{kl} x_l$

Fin pour

$x_k \leftarrow s/a_{kk}$

Fin pour

Écrire $x_1, \dots, x_n$

Vérification numérique. Cet algorithme est la fonction `remontee` du programme `gauss.py` (exercice 10 de la fin du document). Sur un système 6×6 aléatoire dont on connaît la solution, l'erreur maximale obtenue est $2,4 \times 10^{-16}$.

Références. [BF, alg. 6.1, étapes 8 et 9, p. 368 ; décompte des opérations p. 370–371].

Exercice I.2 : échange de lignes (pivot maximal)

Énoncé. Écrire l'algorithme qui permet d'interchanger les lignes des systèmes algébriques dû à la règle de pivot maximal.

Pourquoi ? À l'étape k de la triangularisation, on divise par a_{kk} (multiplicateur a_{lk}/a_{kk}). Deux problèmes peuvent survenir :

- si $a_{kk} = 0$, la méthode s'arrête (division par zéro) alors que le système peut très bien avoir une solution ;
- si $|a_{kk}|$ est petit devant les autres coefficients de la colonne, les multiplicateurs sont grands et amplifient les erreurs d'arrondi. Burden et Faires montrent sur le système $0,003000x_1 + 59,14x_2 = 59,17$, $5,291x_1 - 6,130x_2 = 46,78$ (solution exacte $x_1 = 10$, $x_2 = 1$) qu'en arithmétique à quatre chiffres on obtient $x_1 \approx -10,00$ sans échange de lignes [BF, ex. 1, p. 376].

La règle du pivot maximal (pivot partiel) choisit comme équation pivotale, parmi les lignes k à n , celle dont le coefficient de x_k est le plus grand en valeur absolue. Les multiplicateurs vérifient alors $|a_{lk}/a_{kk}| \leq 1$: les erreurs ne sont plus amplifiées. Échanger deux équations ne change pas la solution du système.

Comment ? (étapes)

Étape 1. À l'étape k , chercher l'indice $p \geq k$ tel que $|a_{pk}| = \max_{k \leq i \leq n} |a_{ik}|$.

Étape 2. Si $a_{pk} = 0$, toute la colonne sous la diagonale est nulle : la matrice est singulière, on arrête.

Étape 3. Si $p \neq k$, échanger les lignes k et p de la matrice (il suffit de le faire pour les colonnes k à n , les colonnes précédentes étant déjà nulles) et les composantes b_k et b_p du second membre. L'échange utilise une variable auxiliaire t .

Algorithme : recherche du pivot et échange (étape k)

```
p ← k
Pour i allant de k + 1 à n faire
    Si |aik| > |apk| alors
        p ← i
Fin si
Fin pour
Si apk = 0 alors
    Écrire « matrice singulière »
    Arrêt
Fin si
Si p ≠ k alors
    Pour j allant de k à n faire
        t ← akj ; akj ← apj ; apj ← t
    Fin pour
    t ← bk ; bk ← bp ; bp ← t
Fin si
```

Références. [BF, § 6.2, pivot partiel, alg. 6.2, p. 378–379].

Exercice I.3 : méthode de Gauss complète avec pivot maximal

Énoncé. Écrire l'algorithme complet de la méthode de Gauss (triangularisation et résolution du système triangulaire) en prenant soin de faire jouer la règle du pivot maximal.

Pourquoi ? On assemble les deux morceaux précédents : pour chaque étape k , on choisit d'abord le pivot (exercice I.2), puis on élimine x_k des lignes $k+1$ à n avec l'algorithme de l'étape k du cours ; une fois le système triangulaire obtenu, on fait la remontée (exercice I.1). Le pivot doit être choisi à *chaque étape*, car les coefficients changent d'une étape à l'autre.

Comment ? (étapes)

Étape 1. Pour $k = 1, \dots, n-1$: choix du pivot et échange éventuel.

Étape 2. Toujours à l'étape k , pour chaque ligne $l > k$: calcul du multiplicateur $m = a_{lk}/a_{kk}$ (calculé une seule fois, ce qui évite de refaire la division pour chaque colonne), puis $b_l \leftarrow b_l - m b_k$ et $a_{lj} \leftarrow a_{lj} - m a_{kj}$ pour $j = k+1, \dots, n$.

Étape 3. Après la boucle, vérifier $a_{nn} \neq 0$, puis remontée.

Algorithme : Gauss avec pivot maximal

Données : n , $A = (a_{ij})$, $b = (b_i)$

Pour k allant de 1 à $n-1$ **faire**

recherche du pivot p et échange des lignes k et p

{exercice I.2}

Pour l allant de $k+1$ à n **faire**

$m \leftarrow a_{lk}/a_{kk}$

$b_l \leftarrow b_l - m b_k$

Pour j allant de $k+1$ à n **faire**

$a_{lj} \leftarrow a_{lj} - m a_{kj}$

Fin pour

$a_{lk} \leftarrow 0$

Fin pour

Fin pour

Si $a_{nn} = 0$ **alors**

Écrire « matrice singulière »

Arrêt

Fin si

remontée

{exercice I.1}

Le coût total est de $n^3/3 + n^2 - n/3$ multiplications ou divisions [BF, p. 371] ; la recherche du pivot n'ajoute que des comparaisons.

Vérification numérique. Exemple traité à la main (et par le programme `gauss.py`) : le système

$$2x_2 + x_3 = 5, \quad x_1 + x_2 + x_3 = 6, \quad 2x_1 + x_2 + 3x_3 = 13$$

ne peut pas être traité sans échange car $a_{11} = 0$.

— $k = 1$: le plus grand $|a_{i1}|$ est 2 (ligne 3). On échange les lignes 1 et 3 : $(2 \ 1 \ 3 \mid 13)$, $(1 \ 1 \ 1 \mid 6)$, $(0 \ 2 \ 1 \mid 5)$. Multiplicateurs $1/2$ et 0 : la ligne 2 devient $(0 \ \frac{1}{2} \ -\frac{1}{2} \mid -\frac{1}{2})$.

— $k = 2$: entre $\frac{1}{2}$ (ligne 2) et 2 (ligne 3), le pivot est 2 ; on échange les lignes 2 et 3. Multiplicateur $\frac{1/2}{2} = \frac{1}{4}$: la ligne 3 devient $(0 \ 0 \ -\frac{3}{4} \mid -\frac{7}{4})$.

— Remontée : $x_3 = 7/3$, $x_2 = (5 - 7/3)/2 = 4/3$, $x_1 = (13 - 4/3 - 3 \cdot 7/3)/2 = 7/3$.

Le programme affiche bien $x_1 = 2,3333333333$, $x_2 = 1,3333333333$, $x_3 = 2,3333333333$. Un test sur une matrice 5×5 aléatoire avec $a_{11} = 0$ donne une erreur de 10^{-15} (`tests_algorithms.py`).

Références. [BF, alg. 6.1, p. 368, et alg. 6.2, p. 378–379].

Exercice I.4 : calcul des l_{ij} et des s_{ij}

Énoncé. Écrire l'algorithme permettant de calculer les l_{ij} et s_{ij} .

Pourquoi ? Une fois $A = LS$ connue, résoudre $AX = b$ revient à résoudre deux systèmes triangulaires ($LY = b$, puis $SX = Y$), soit environ n^2 opérations, au lieu de refaire toute l'élimination (environ $n^3/3$). C'est très avantageux quand on doit résoudre plusieurs systèmes de même matrice A et de seconds membres différents. Il faut organiser le calcul dans un ordre où chaque élément ne dépend que d'éléments déjà calculés : c'est pourquoi le cours calcule alternativement une colonne de L puis une ligne de S .

Comment ? (étapes)

Étape 1. Écrire le produit LS terme à terme. Comme $l_{jk} = 0$ pour $k > j$, $s_{kp} = 0$ pour $k > p$ et $s_{pp} = 1$:

$$a_{jp} = \sum_{k=1}^{p-1} l_{jk}s_{kp} + l_{jp} \quad (j \geq p), \quad a_{pj} = \sum_{k=1}^{p-1} l_{pk}s_{kj} + l_{pp}s_{pj} \quad (j > p).$$

Étape 2. En isolant l'élément inconnu, on obtient les formules générales qui contiennent toutes celles du cours (pour $p = 1$, les sommes sont vides) :

$$l_{jp} = a_{jp} - \sum_{k=1}^{p-1} l_{jk}s_{kp} \quad (j = p, \dots, n), \quad s_{pj} = \frac{1}{l_{pp}} \left(a_{pj} - \sum_{k=1}^{p-1} l_{pk}s_{kj} \right) \quad (j = p+1, \dots, n).$$

Étape 3. Ordre du calcul : pour $p = 1, 2, \dots, n$, d'abord la colonne p de L (elle utilise les colonnes 1 à $p-1$ de L et les lignes 1 à $p-1$ de S , déjà connues), puis la ligne p de S (elle utilise en plus l_{pp} , que l'on vient de calculer). Il faut $l_{pp} \neq 0$; sinon la décomposition sans échange de lignes est impossible.

Algorithme : décomposition $A = LS$ (Crout)

Données : n , $A = (a_{ij})$

Pour p allant de 1 à n **faire**

Pour j allant de p à n **faire**

{colonne p de L }

$l_{jp} \leftarrow a_{jp}$

Pour k allant de 1 à $p-1$ **faire**

$l_{jp} \leftarrow l_{jp} - l_{jk}s_{kp}$

Fin pour

Fin pour

Si $l_{pp} = 0$ **alors**

Écrire « décomposition impossible »

Arrêt

Fin si

$s_{pp} \leftarrow 1$

Pour j allant de $p+1$ à n **faire**

{ligne p de S }

$s_{pj} \leftarrow a_{pj}$

Pour k allant de 1 à $p-1$ **faire**

$s_{pj} \leftarrow s_{pj} - l_{pk}s_{kj}$

Fin pour

$s_{pj} \leftarrow s_{pj}/l_{pp}$

Fin pour

Fin pour

Vérification numérique. Sur une matrice 5×5 aléatoire à diagonale dominante, on obtient $\max |LS - A| = 9 \times 10^{-16}$ (programme `tests_algorithms.py`).

Références. [BF, alg. 6.4, p. 410] avec le choix $u_{ii} = 1$: c'est la décomposition de Crout [BF, p. 410].

Exercice I.5 : inversion d'une matrice et produit $L^{-1}b$

Énoncé. Trouver dans la littérature l'algorithme d'inversion d'une matrice et de calcul des éléments du produit $L^{-1}b$.

Pourquoi ? Le système $SX = L^{-1}b$ du cours demande le vecteur $Y = L^{-1}b$. On ne calcule jamais la matrice L^{-1} elle-même : Y est la solution de $LY = b$, système triangulaire *inférieur* que l'on résout de haut en bas (« descente »), comme la remontée mais dans l'autre sens. C'est moins coûteux et plus précis que de former L^{-1} . De même, Burden et Faires rappellent qu'il n'est pas efficace de calculer A^{-1} pour résoudre $AX = b$ [BF, p. 392] ; l'inverse n'est utile que si on en a besoin pour elle-même.

Comment ? (étapes)

Étape 1. *Produit* $Y = L^{-1}b$. La première équation de $LY = b$ donne $y_1 = b_1/l_{11}$; la ligne i ne contient que $y_1, \dots, y_i$, d'où

$$y_i = \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij} y_j \right), \quad i = 2, \dots, n \quad [BF, p. 411].$$

Étape 2. *Inverse.* Si $B = A^{-1}$, alors $AB = I$: la colonne j de B , notée B_j , est la solution de $AB_j = e_j$, où e_j est la colonne j de la matrice identité [BF, p. 392–393]. On résout donc n systèmes de même matrice A . Avec la décomposition $A = LS$ faite une seule fois, chaque colonne coûte une descente ($LY = e_j$) et une remontée ($SB_j = Y$). Burden et Faires procèdent de façon équivalente par élimination de Gauss sur la matrice augmentée $[A \mid I]$ [BF, p. 393] ; la méthode de Jordan du cours transforme directement $[A \mid I]$ en $[I \mid A^{-1}]$.

Algorithme : descente $Y = L^{-1}b$

```

 $y_1 \leftarrow b_1/l_{11}$ 
Pour  $i$  allant de 2 à  $n$  faire
     $s \leftarrow b_i$ 
    Pour  $j$  allant de 1 à  $i-1$  faire
         $s \leftarrow s - l_{ij} y_j$ 
    Fin pour
     $y_i \leftarrow s/l_{ii}$ 
Fin pour
```

Algorithme : inverse de A

```

décomposer  $A = LS$  {exercice I.4}
Pour  $j$  allant de 1 à  $n$  faire
     $e \leftarrow$  colonne  $j$  de l'identité ( $e_i = 0$  pour  $i \neq j$ ,  $e_j = 1$ )
    résoudre  $LY = e$  par descente
    résoudre  $SX = Y$  par remontée {exercice I.1}
    Pour  $i$  allant de 1 à  $n$  faire
         $b_{ij} \leftarrow x_i$ 
    Fin pour
```

Fin pour

Écrire la matrice $B = A^{-1}$

Vérification numérique. Sur la même matrice 5×5 : erreur de résolution de $LSX = b$ égale à 6×10^{-17} , et $\max |AA^{-1} - I| = 2 \times 10^{-16}$.

Références. [BF, § 6.3, p. 391–393 (inverse) ; § 6.5, p. 411 (descente et remontée)].

Exercice I.6 : coefficients m_{pp} et m_{pj} de Cholesky

Énoncé. Écrire l'algorithme de calcul des coefficients m_{pp} et m_{pj} .

Pourquoi ? Pour une matrice symétrique, $A = M^t M$ avec une seule matrice triangulaire supérieure M à calculer : on stocke moitié moins de nombres et on fait environ deux fois moins d'opérations que pour LS . Le calcul des m_{pp} demande des racines carrées : il faut que les quantités sous la racine soient strictement positives. C'est le cas si et seulement si A est *définie positive* (et pas seulement symétrique) [BF, § 6.6, p. 423–424]. L'algorithme doit donc tester ce signe.

Comment ? (étapes)

Étape 1. M étant triangulaire supérieure ($m_{kp} = 0$ si $k > p$), le coefficient (p, j) de $M^t M$ pour $p \leq j$ vaut

$$a_{pj} = \sum_{k=1}^p m_{kp} m_{kj} = \sum_{k=1}^{p-1} m_{kp} m_{kj} + m_{pp} m_{pj}.$$

Étape 2. Pour $j = p$, on obtient $a_{pp} = \sum_{k < p} m_{kp}^2 + m_{pp}^2$, d'où $m_{pp} = \sqrt{a_{pp} - \sum_{k=1}^{p-1} m_{kp}^2}$. Pour $j > p$, on obtient $m_{pj} = (a_{pj} - \sum_{k=1}^{p-1} m_{kp} m_{kj}) / m_{pp}$. Ce sont les formules du cours (avec $m_{11} = \sqrt{a_{11}}$ et $m_{1j} = a_{1j} / m_{11}$ pour $p = 1$).

Étape 3. On calcule M ligne par ligne : la ligne p n'utilise que les lignes 1 à $p - 1$, déjà calculées.

Algorithme : Cholesky $A = M^t M$

Pour p allant de 1 à n **faire**

$s \leftarrow a_{pp}$

Pour k allant de 1 à $p - 1$ **faire**

$s \leftarrow s - m_{kp}^2$

Fin pour

Si $s \leq 0$ **alors**

Écrire « A n'est pas définie positive »

Arrêt

Fin si

$m_{pp} \leftarrow \sqrt{s}$

Pour j allant de $p + 1$ à n **faire**

$s \leftarrow a_{pj}$

Pour k allant de 1 à $p - 1$ **faire**

$s \leftarrow s - m_{kp} m_{kj}$

Fin pour

$m_{pj} \leftarrow s / m_{pp}$

Fin pour

Fin pour

Vérification numérique. Pour la matrice de [BF, ex. 4, p. 424],

$$A = \begin{pmatrix} 4 & -1 & 1 \\ -1 & 4,25 & 2,75 \\ 1 & 2,75 & 3,5 \end{pmatrix}, \quad \text{l'algorithme donne} \quad M = \begin{pmatrix} 2 & -0,5 & 0,5 \\ 0 & 2 & 1,5 \\ 0 & 0 & 1 \end{pmatrix},$$

qui est bien la transposée de la matrice L trouvée par Burden et Faires [BF, p. 424], et $M^t M = A$ exactement.

Références. [BF, alg. 6.6, p. 423–424] (forme $A = LL^t$, avec $L = M^t$).

Exercice I.7 : Jacobi et Gauss-Seidel avec condition d'arrêt

Énoncé. Écrire les algorithmes des méthodes de Jacobi et de Gauss-Seidel avec imposition de la condition d'arrêt.

Pourquoi ? Une méthode itérative produit une suite infinie $X^{(0)}, X^{(1)}, \dots$; il faut donc décider quand s'arrêter. Le cours recommande le test relatif $d = \sum_i |x_i^{(k+1)} - x_i^{(k)}| / \sum_i |x_i^{(k+1)}| < \varepsilon$, qui ne dépend pas de l'ordre de grandeur des inconnues. Comme la convergence n'est pas toujours assurée, on limite aussi le nombre d'itérations à $k_{\max}$ (une condition suffisante de convergence des deux méthodes est que A soit à diagonale strictement dominante [BF, th. 7.21, p. 464]). La différence entre les deux méthodes impose une différence de programmation :

- Jacobi n'utilise que les valeurs de l'itération précédente : il faut deux tableaux, l'ancien x et le nouveau y ;
- Gauss-Seidel utilise chaque nouvelle valeur dès qu'elle est calculée : un seul tableau suffit, mis à jour sur place ; il faut seulement garder l'ancienne valeur de x_i pour le test d'arrêt.

Comment ? (étapes)

Étape 1. Valeurs initiales $x_i = 0$ (ou 1), comme le propose le cours.

Étape 2. À chaque itération, calculer pour $i = 1, \dots, n$ la nouvelle valeur par la formule du cours, en accumulant le numérateur et le dénominateur de d .

Étape 3. Arrêter si $d < \varepsilon$ (convergence) ou si $k = k_{\max}$ (échec, à signaler).

Algorithme : Jacobi

Données : $n, A, b, \varepsilon, k_{\max}$

Pour i allant de 1 à n **faire**

$x_i \leftarrow 0$

Fin pour

$k \leftarrow 0$

Répéter

$k \leftarrow k + 1$; $num \leftarrow 0$; $den \leftarrow 0$

Pour i allant de 1 à n **faire**

$s \leftarrow b_i$

Pour j allant de 1 à n , $j \neq i$ **faire**

$s \leftarrow s - a_{ij} x_j$

Fin pour

$y_i \leftarrow s/a_{ii}$

$num \leftarrow num + |y_i - x_i|$; $den \leftarrow den + |y_i|$

Fin pour

Pour i allant de 1 à n **faire**

$x_i \leftarrow y_i$

Fin pour
Jusqu'à $num/den < \varepsilon$ **ou** $k \geq k_{\max}$
Si $num/den \geq \varepsilon$ **alors**
 Écrire « pas de convergence »
Fin si
Écrire $k, x_1, \dots, x_n$

Algorithme : Gauss-Seidel

Données : $n, A, b, \varepsilon, k_{\max}$

Pour i allant de 1 à n **faire**

$x_i \leftarrow 0$

Fin pour

$k \leftarrow 0$

Répéter

$k \leftarrow k + 1; num \leftarrow 0; den \leftarrow 0$

Pour i allant de 1 à n **faire**

$s \leftarrow b_i$

Pour j allant de 1 à $n, j \neq i$ **faire**

$s \leftarrow s - a_{ij} x_j$

$\{x_j \text{ est déjà la nouvelle valeur si } j < i\}$

Fin pour

$t \leftarrow s/a_{ii}$

$num \leftarrow num + |t - x_i|; den \leftarrow den + |t|$

$x_i \leftarrow t$

Fin pour

Jusqu'à $num/den < \varepsilon$ **ou** $k \geq k_{\max}$

Écrire $k, x_1, \dots, x_n$

Vérification numérique. Sur le système de [BF, ex. 1, p. 456] ($10x_1 - x_2 + 2x_3 = 6$, $-x_1 + 11x_2 - x_3 + 3x_4 = 25$, $2x_1 - x_2 + 10x_3 - x_4 = -11$, $3x_2 - x_3 + 8x_4 = 15$, solution $(1, 2, -1, 1)$), avec $\varepsilon = 10^{-3}$: Jacobi s'arrête après 10 itérations sur $(1,0001; 1,9998; -0,9998; 0,9998)$ et Gauss-Seidel après 5 itérations sur $(1,0001; 2,0000; -1,0000; 1,0000)$. Ce sont les mêmes nombres d'itérations que dans le livre (10 et 5), qui utilise un test d'arrêt voisin fondé sur la norme maximale [BF, p. 458–461].

Références. [BF, alg. 7.1, p. 459 (Jacobi); alg. 7.2, p. 462 (Gauss-Seidel); th. 7.21, p. 464].

Chapitre II : équations algébriques non linéaires

Exercice II.1 : Newton-Raphson avec condition d'arrêt

Énoncé. Écrire l'algorithme de la méthode de Newton-Raphson en y intégrant la condition d'arrêt.

Pourquoi ? La suite $x_{n+1} = x_n - f(x_n)/f'(x_n)$ ne s'arrête jamais d'elle-même : il faut un critère qui garantisse la précision E demandée. Le cours donne la majoration $|x_n - \bar{x}| \leq |f(x_n)|/m$ si $|f'| \geq m > 0$ sur un intervalle $[a, b]$ contenant x_n et la racine $\bar{x}$. Elle vient du théorème des accroissements finis : $f(x_n) = f(x_n) - f(\bar{x}) = f'(\xi)(x_n - \bar{x})$ avec ξ entre x_n et $\bar{x}$, donc $|x_n - \bar{x}| = |f(x_n)|/|f'(\xi)| \leq |f(x_n)|/m$. Le test $|f(x_n)|/m < E$ garantit donc $|x_n - \bar{x}| < E$. Deux sécurités sont nécessaires en plus : arrêter si $f'(x_n) = 0$ (la tangente est horizontale, la formule n'a plus de sens) et limiter le nombre d'itérations, car Newton peut diverger si x_0 est mal choisi (remarque du cours).

Comment ? (étapes)

Étape 1. Localiser la racine dans $[a, b]$ (changement de signe de f), choisir $x_0 \in [a, b]$ et calculer $m = \min(|f'(a)|, |f'(b)|)$ (valable lorsque $|f'|$ est monotone sur $[a, b]$, comme le suppose le cours ; sinon prendre le minimum de $|f'|$ sur $[a, b]$).

Étape 2. Itérer $x \leftarrow x - f(x)/f'(x)$.

Étape 3. Après chaque itération, tester $|f(x)|/m < E$; arrêter aussi si k atteint $k_{\max}$.

Algorithme : Newton-Raphson

Données : $f, f', a, b, x_0, E, k_{\max}$

$m \leftarrow \min(|f'(a)|, |f'(b)|)$

$x \leftarrow x_0 ; k \leftarrow 0$

Répéter

Si $f'(x) = 0$ **alors**

Écrire « dérivée nulle : changer x_0 »

Arrêt

Fin si

$x \leftarrow x - f(x)/f'(x)$

$k \leftarrow k + 1$

Jusqu'à $|f(x)|/m < E$ **ou** $k \geq k_{\max}$

Si $|f(x)|/m \geq E$ **alors**

Écrire « pas de convergence : changer x_0 »

Sinon

Écrire x, k

Fin si

Vérification numérique. Pour $f(x) = (x - 5)e^x + 5$ (exercice 1 de la fin du document), sur $[4, 9; 5]$: $f'(x) = (x - 4)e^x$ est croissante, $m = |f'(4, 9)| = 120,861$; depuis $x_0 = 5$ et avec $E = 10^{-12}$, l'algorithme s'arrête après 4 itérations sur $x = 4,965114231744$.

Références. [BF, alg. 2.3, p. 67].

Exercice II.2 : méthode de bisection

Énoncé. Écrire l'algorithme de la méthode de bisection.

Pourquoi ? Si f est continue et si $f(a)$ et $f(b)$ sont de signes opposés, le théorème des valeurs intermédiaires garantit une racine dans $[a, b]$. En gardant à chaque étape la moitié où le signe change, on conserve toujours une racine, et la longueur de l'intervalle est divisée par 2. La méthode converge donc *toujours* (contrairement à Newton), mais lentement. Son avantage : on connaît à l'avance le nombre d'étapes. Pour que l'intervalle final soit de longueur inférieure à ε , il faut $(b - a)/2^n < \varepsilon$, c'est-à-dire $n > \log_2((b - a)/\varepsilon)$.

Le test sur le signe se fait par le produit $f(a)f(c)$: il fonctionne que f soit croissante ou décroissante (le cours suppose f croissante, ce qui n'est pas nécessaire). On garde $f(a)$ en mémoire pour n'évaluer f qu'une fois par étape. Burden et Faires conseillent de tester le *signe* plutôt que le produit si l'on craint un dépassement de capacité [BF, p. 52].

Comment ? (étapes)

Étape 1. Vérifier $f(a)f(b) < 0$.

Étape 2. Tant que $b - a > \varepsilon$: calculer $c = (a + b)/2$ et $f(c)$; si $f(c) = 0$, la racine est c ; si $f(a)f(c) < 0$, la racine est dans $[a, c]$ et on pose $b \leftarrow c$; sinon elle est dans $[c, b]$ et on pose $a \leftarrow c$, $f(a) \leftarrow f(c)$.

Étape 3. Le résultat est le milieu du dernier intervalle.

Algorithme : bisection

Données : f, a, b, ε

$f_a \leftarrow f(a)$

Si $f_a f(b) > 0$ **alors**

 Écrire « pas de changement de signe sur $[a, b]$ »

 Arrêt

Fin si

Tant que $b - a > \varepsilon$ **faire**

$c \leftarrow (a + b)/2$; $f_c \leftarrow f(c)$

Si $f_c = 0$ **alors**

 Écrire c

 Arrêt

Fin si

Si $f_a f_c < 0$ **alors**

$b \leftarrow c$

Sinon

$a \leftarrow c$; $f_a \leftarrow f_c$

Fin si

Fin tant que

Écrire $(a + b)/2$

Vérification numérique. Avec la même fonction que ci-dessus, sur $[4,9;5]$ et $\varepsilon = 10^{-10}$: $\log_2(0,1/10^{-10}) = 29,9$, il faut donc 30 étapes, et l'algorithme donne effectivement $x = 4,9651142317$ en 30 étapes.

Références. [BF, alg. 2.1, p. 49].

Exercice II.3 : schéma de Lin pour les racines réelles

Énoncé. Établir l'algorithme de ce schéma de Lin pour les solutions réelles.

Pourquoi ? Si x est une racine, la dernière quantité de Horner est nulle : $b_n = a_n + x b_{n-1}(x) = 0$, d'où $x = -a_n/b_{n-1}(x)$. La racine est donc un point fixe de la fonction $g(x) = -a_n/b_{n-1}(x)$, et la méthode de Lin consiste à itérer $x_{k+1} = g(x_k)$: c'est une formule d'itération du premier ordre au sens du chapitre II. Une telle itération converge vers une racine r lorsque $|g'(r)| < 1$ et s'en éloigne lorsque $|g'(r)| > 1$; c'est ce qui explique la remarque du cours (« elle n'est pas toujours convergente »). Quand l'itération a convergé, $b_n \approx 0$ et $P_n(x) = (x - r)Q(x) + b_n$: les nombres $b_0, \dots, b_{n-1}$ sont les coefficients du quotient Q , de degré $n - 1$, sur lequel on recommence pour trouver la racine suivante (déflation).

Comment ? (étapes)

Étape 1. Partir d'une valeur x_0 .

Étape 2. Calculer par Horner $b_0 = a_0$, $b_j = a_j + x b_{j-1}$ pour $j = 1, \dots, n - 1$ (on s'arrête à b_{n-1}).

Étape 3. Nouvelle valeur $x' = -a_n/b_{n-1}$; arrêter si $|x' - x| < \varepsilon$, ou si $k_{\max}$ itérations ont été faites sans convergence (changer alors x_0).

Étape 4. Déflation : recalculer $b_0, \dots, b_{n-1}$ avec la racine trouvée, remplacer $(a_0, \dots, a_n)$ par $(b_0, \dots, b_{n-1})$, faire $n \leftarrow n - 1$ et recommencer à l'étape 1 tant que $n \geq 1$.

Algorithme : méthode de Lin avec déflation

Données : $n, a_0, \dots, a_n, x_0, \varepsilon, k_{\max}$

Tant que $n \geq 1$ **faire**

$x \leftarrow x_0$; $k \leftarrow 0$

Répéter

$k \leftarrow k + 1$; $b_0 \leftarrow a_0$

Pour j allant de 1 à $n - 1$ **faire**

$b_j \leftarrow a_j + x b_{j-1}$

Fin pour

Si $b_{n-1} = 0$ **alors**

Écrire « changer x_0 »

Arrêt

Fin si

$x' \leftarrow -a_n/b_{n-1}$; $d \leftarrow |x' - x|$; $x \leftarrow x'$

Jusqu'à $d < \varepsilon$ **ou** $k \geq k_{\max}$

Si $d \geq \varepsilon$ **alors**

Écrire « pas de convergence »

Arrêt

Fin si

Écrire racine x

$b_0 \leftarrow a_0$

Pour j allant de 1 à $n - 1$ **faire**

$b_j \leftarrow a_j + x b_{j-1}$

Fin pour

Pour j allant de 0 à $n - 1$ **faire**

$a_j \leftarrow b_j$

Fin pour

$n \leftarrow n - 1$

Fin tant que

(Pour $n = 1$, la boucle de Horner est vide et $x = -a_1/a_0$ est obtenu en une itération.)

Vérification numérique. Pour $P(x) = x^3 - 6x^2 + 11x - 6 = (x - 1)(x - 2)(x - 3)$, avec $\varepsilon = 10^{-10}$: depuis $x_0 = 0$ ou $1,5$, la méthode converge vers 1 (53 et 56 itérations) ; depuis $x_0 = 2,5$ ou $3,5$, elle converge vers 3 (7 itérations) ; elle ne converge jamais vers 2 . La théorie l'explique : ici $g(x) = 6/(x^2 - 6x + 11)$ et $g'(x) = -6(2x - 6)/(x^2 - 6x + 11)^2$, d'où $g'(1) = 2/3$ (convergence lente), $g'(2) = 4/3 > 1$ (racine répulsive) et $g'(3) = 0$ (convergence rapide). C'est un exemple concret de la non-convergence annoncée par le cours.

Références. Méthode du cours (Lin, 1941). Pour la théorie des itérations de point fixe et le critère $|g'| < 1$: [BF, § 2.2, th. 2.4, p. 61]. Horner et déflation : [BF, alg. 2.7, p. 94 ; déflation p. 95].

Chapitre III : interpolation et moindres carrés

Exercice III.1 : programme d'interpolation de Lagrange

Énoncé. Traduire cet algorithme dans un des langages de programmation de votre choix (Pascal, Fortran, C).

Pourquoi ? Le polynôme de Lagrange s'écrit $f(x_p) = \sum_k y_k f_k(x_p)$ avec $f_k(x_p) = \prod_{i \neq k} (x_p - x_i) / (x_k - x_i)$. L'algorithme du cours calcule chaque produit $f_k(x_p)$ dans la variable y_s (boucle intérieure), puis l'ajoute, multiplié par y_k , à la somme y_p (boucle extérieure). On n'a jamais besoin des coefficients du polynôme : on évalue directement sa valeur en x_p .

Comment ? (étapes)

Étape 1. Écrire une fonction qui reçoit la liste des x_k , la liste des y_k et x_p (en Python, `len(x)` donne le nombre de points $n + 1$).

Étape 2. Traduire les deux boucles imbriquées ; le test $i \neq k$ évite le facteur nul $(x_k - x_k)$ au dénominateur.

Étape 3. En Python, les listes commencent à l'indice 0 : `range(len(x))` parcourt les indices 0 à n au lieu de 1 à $n + 1$.

Programme Python (fichier `programmes/lagrange.py`) :

```
"""Chapitre III : interpolation de Lagrange (algorithme du cours).
Exercice III.4 : y_p pour x_p = 102, puis x_p pour y_p = 13,5."""

def lagrange(x, y, xp):
    """Valeur en xp du polynôme qui passe par les points (x[k], y[k])."""
    yp = 0.0
    for k in range(len(x)):
        ys = 1.0          # ys = f_k(xp)
        for i in range(len(x)):
            if i != k:
                ys = ys * (xp - x[i]) / (x[k] - x[i])
        yp = yp + y[k] * ys
    return yp

if __name__ == "__main__":
    x = [93.0, 96.2, 100.0, 104.2, 108.7]
    y = [11.38, 12.80, 14.70, 17.07, 19.91]
    print(f"x_p = 102    y_p = {lagrange(x, y, 102.0):.6f}")
    # interpolation inverse : on échange les rôles de x et de y
    print(f"y_p = 13,5   x_p = {lagrange(y, x, 13.5):.6f}")
```

Vérification numérique. Avec les données de l'exercice III.4 ci-dessous, le programme affiche $y_p = 15,793629$ pour $x_p = 102$.

Références. [BF, § 3.1, p. 106–108] pour la formule de Lagrange ; une alternative programmable est l'algorithme de Neville [BF, alg. 3.1, p. 120].

Complément : formules de a_0 et a_1 pour la droite des moindres carrés

Énoncé. Le cours demande d'établir en exercice les formules de a_0 et a_1 pour $f(x) = a_0 + a_1x$.

Pourquoi ? Le reste $R(a_0, a_1) = \sum_k [f_k - (a_0 + a_1x_k)]^2$ est une fonction du second degré des deux inconnues, positive : son minimum est atteint là où ses deux dérivées partielles s'annulent. On obtient deux équations linéaires (les « équations normales »).

Comment ? (étapes)

Étape 1. Dérivées partielles :

$$\frac{\partial R}{\partial a_0} = -2 \sum_k (f_k - a_0 - a_1x_k) = 0, \quad \frac{\partial R}{\partial a_1} = -2 \sum_k x_k (f_k - a_0 - a_1x_k) = 0.$$

Étape 2. En développant :

$$m a_0 + \left(\sum x_k\right) a_1 = \sum f_k, \quad \left(\sum x_k\right) a_0 + \left(\sum x_k^2\right) a_1 = \sum x_k f_k.$$

Étape 3. Par la règle de Cramer, avec $D = m \sum x_k^2 - (\sum x_k)^2$ (non nul dès que deux x_k sont distincts) :

$$a_0 = \frac{\sum x_k^2 \sum f_k - \sum x_k \sum x_k f_k}{D}, \quad a_1 = \frac{m \sum x_k f_k - \sum x_k \sum f_k}{D},$$

ce sont les formules du cours.

Références. [BF, § 8.1, p. 507–508].

Exercice III.2 : calcul des coefficients b_k et A_{kj}

Énoncé. Écrire les algorithmes de calculs des coefficients b_k et A_{kj} .

Pourquoi ? Les équations normales $\sum_j A_{kj} a_j = b_k$ viennent de $\partial R / \partial a_k = 0$ pour $R = \sum_i [f_i - \sum_j a_j g_j(x_i)]^2$. Chaque coefficient est une somme sur les m mesures : une boucle sur i suffit. Deux remarques rendent le calcul plus économique :

- $A_{kj} = A_{jk}$ (la matrice est symétrique) : on ne calcule que $j \geq k$ et on recopie ;
- les valeurs $g_k(x_i)$ servent plusieurs fois : on les calcule une seule fois et on les range dans un tableau $G_{ik} = g_k(x_i)$.

Une fois A et b calculés, le système se résout par la méthode de Gauss avec pivot (exercice I.3).

Comment ? (étapes)

Étape 1. Remplir $G_{ik} = g_k(x_i)$ pour $i = 1, \dots, m$ et $k = 0, \dots, n$.

Étape 2. Pour chaque k : $b_k = \sum_i G_{ik} f_i$, puis pour $j \geq k$: $A_{kj} = \sum_i G_{ik} G_{ij}$ et $A_{jk} = A_{kj}$.

Étape 3. Résoudre $A a = b$ par Gauss.

Algorithme : équations normales

Données : $m, (x_i, f_i), n$, les fonctions $g_0, \dots, g_n$

Pour i allant de 1 à m **faire**

Pour k allant de 0 à n **faire**

$G_{ik} \leftarrow g_k(x_i)$

Fin pour

Fin pour

Pour k allant de 0 à n **faire**

$b_k \leftarrow 0$

Pour i allant de 1 à m **faire**

$b_k \leftarrow b_k + G_{ik} f_i$

Fin pour

Pour j allant de k à n **faire**

$A_{kj} \leftarrow 0$

Pour i allant de 1 à m **faire**

$A_{kj} \leftarrow A_{kj} + G_{ik} G_{ij}$

Fin pour

$A_{jk} \leftarrow A_{kj}$

Fin pour

Fin pour

résoudre $Aa = b$ par la méthode de Gauss avec pivot

{exercice I.3}

Écrire $a_0, \dots, a_n$

Références. [BF, § 8.1, équations normales, p. 508–510].

Exercice III.3 : moindres carrés sur un tableau de mesures

Énoncé. $x = 1, 2, 3, 4, 5$; $y = 1,6$; 2 ; $2,3$; $2,4$; $2,5$. Déterminer par les moindres carrés a) $y = a_0 + a_1/x$; b) $y = a_0 + a_1/x + a_2e^x$.

Pourquoi ? On applique l'exercice précédent avec $g_0 = 1$, $g_1 = 1/x$ (et $g_2 = e^x$ en b). Les modèles sont non linéaires en x mais *linéaires en les coefficients* : les équations normales restent un système linéaire. Le choix $1/x$ est naturel car les mesures croissent de moins en moins vite.

Comment ? (étapes)

Étape 1. Cas a). Sommes nécessaires ($m = 5$) :

$$\sum \frac{1}{x_i} = \frac{137}{60} = 2,283333, \quad \sum \frac{1}{x_i^2} = 1,463611, \quad \sum y_i = 10,8, \quad \sum \frac{y_i}{x_i} = 4,466667.$$

Étape 2. Système normal :

$$\begin{pmatrix} 5 & 2,283333 \\ 2,283333 & 1,463611 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \end{pmatrix} = \begin{pmatrix} 10,8 \\ 4,466667 \end{pmatrix}.$$

Déterminant $D = 5 \times 1,463611 - 2,283333^2 = 2,104444$; Cramer :

$$a_0 = \frac{1,463611 \times 10,8 - 2,283333 \times 4,466667}{D} = \frac{5,608111}{2,104444} = 2,664889,$$

$$a_1 = \frac{5 \times 4,466667 - 2,283333 \times 10,8}{D} = \frac{-2,326667}{2,104444} = -1,105597.$$

Donc $y \approx 2,664889 - 1,105597/x$, avec un reste $R = 0,017529$ (écarts aux mesures : 0,0407 ; -0,1121 ; 0,0036 ; 0,0115 ; 0,0562).

Étape 3. Cas b). Sommes supplémentaires : $\sum e^{x_i} = 233,204184$, $\sum e^{x_i}/x_i = 56,440158$, $\sum e^{2x_i} = 25472,839781$, $\sum y_i e^{x_i} = 567,392556$. Le système 3×3

$$\begin{pmatrix} 5 & 2,283333 & 233,204184 \\ 2,283333 & 1,463611 & 56,440158 \\ 233,204184 & 56,440158 & 25472,839781 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \end{pmatrix} = \begin{pmatrix} 10,8 \\ 4,466667 \\ 567,392556 \end{pmatrix}$$

résolu par Gauss avec pivot donne

$$a_0 = 2,567598, \quad a_1 = -0,990980, \quad a_2 = 0,000964,$$

avec $R = 0,009502$.

Étape 4. Interprétation. Ajouter une fonction ne peut que diminuer R (le modèle a contient le modèle b avec $a_2 = 0$) : R passe de 0,0175 à 0,0095. Le coefficient a_2 est petit parce que e^x est très grand ($e^5 \approx 148$). Les nombres de la matrice sont d'ordres de grandeur très différents (de 1,46 à 25 473) : son conditionnement vaut environ $1,4 \times 10^5$, ce qui justifie le pivot et la double précision.

Vérification numérique. Valeurs obtenues par `calculs.py` et `complements.py`.

Références. [BF, § 8.1, p. 506–513].

Exercice III.4 : interpolation de Lagrange directe et inverse

Énoncé. Du tableau $x : 93,0 ; 96,2 ; 100,0 ; 104,2 ; 108,7$, $y : 11,38 ; 12,80 ; 14,70 ; 17,07 ; 19,91$, déterminer y_p pour $x_p = 102$ et x_p pour $y_p = 13,5$.

Pourquoi ? Avec 5 points, on utilise le polynôme de Lagrange de degré 4. Pour la question inverse (trouver x connaissant y), le cours indique d'échanger les rôles de x et de y : on interpole x comme fonction de y . C'est possible parce que y est strictement croissante sur le tableau (la fonction réciproque existe).

Comment ? (étapes)

Étape 1. Poids de Lagrange en $x_p = 102$: $f_k(102) = 0,043410 ; -0,218191 ; 0,791622 ; 0,413053 ; -0,029894$ (leur somme vaut 1, ce qui est un bon contrôle : le polynôme d'interpolation de la fonction constante 1 est 1).

Étape 2. $y_p = \sum_k y_k f_k(102) = 15,793629$, soit $y_p \approx 15,79$.

Étape 3. Interpolation inverse : les nœuds sont maintenant les y_k et les valeurs les x_k . Poids en 13,5 : $-0,084007 ; 0,710729 ; 0,435996 ; -0,069802 ; 0,007084$, d'où $x_p = 97,655750$.

Vérification numérique. Le programme `lagrange.py` donne $y_p = 15,793629$ et (en échangeant les deux listes) $x_p = 97,655750$. La réponse du cours (15,79 et 97,65) est retrouvée ; arrondi au centième, x_p vaut plutôt 97,66 (97,65 est la valeur tronquée).

Références. [BF, § 3.1, p. 106–108] ; interpolation inverse itérée : [BF, p. 122].

Chapitre IV : intégration et dérivation numériques

Exercice IV.1 : dérivation à partir de l'interpolation quadratique

Énoncé. Établir la formule de dérivation numérique à partir de l'interpolation quadratique.

Pourquoi ? On ne connaît f qu'en quelques points. On la remplace par le polynôme de degré 2 qui passe par trois points, et on dérive ce polynôme. Avec trois points au lieu de deux, l'erreur passe de l'ordre h (formule classique) à l'ordre h^2 .

Comment ? (étapes)

Étape 1. Points équidistants $x_1 = x - h$, $x_2 = x$, $x_3 = x + h$. On part de $f(t) \approx f(x_1)f_1(t) + f(x_2)f_2(t) + f(x_3)f_3(t)$ et on dérive les trois polynômes de base. Par exemple $f_1(t) = (t - x_2)(t - x_3)/((x_1 - x_2)(x_1 - x_3))$ donne $f'_1(t) = (2t - x_2 - x_3)/((x_1 - x_2)(x_1 - x_3))$.

Étape 2. Au point $t = x_2$:

$$\begin{aligned}f'_1(x_2) &= \frac{x_2 - x_3}{(x_1 - x_2)(x_1 - x_3)} = \frac{-h}{(-h)(-2h)} = -\frac{1}{2h}, \\f'_2(x_2) &= \frac{2x_2 - x_1 - x_3}{(x_2 - x_1)(x_2 - x_3)} = 0, \\f'_3(x_2) &= \frac{x_2 - x_1}{(x_3 - x_1)(x_3 - x_2)} = \frac{h}{(2h)(h)} = \frac{1}{2h}.\end{aligned}$$

D'où la formule centrée

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

Étape 3. Erreur. Les développements de Taylor à l'ordre 3, $f(x \pm h) = f(x) \pm hf'(x) + \frac{h^2}{2}f''(x) \pm \frac{h^3}{6}f'''(\xi_{\pm})$, donnent par différence

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{12}(f'''(\xi_+) + f'''(\xi_-)) = f'(x) + \frac{h^2}{6}f'''(\xi),$$

la dernière égalité venant du théorème des valeurs intermédiaires (f''' continue). On retrouve l'erreur du cours $E = -\frac{1}{6}h^2f'''(\xi)$: $f'(x) = \frac{f(x+h)-f(x-h)}{2h} - \frac{h^2}{6}f'''(\xi)$.

Étape 4. Au point $t = x_1$ (dérivée en bout d'intervalle, utile au bord d'un tableau), le même calcul donne $f'_1(x_1) = -3/(2h)$, $f'_2(x_1) = 2/h$, $f'_3(x_1) = -1/(2h)$, c'est-à-dire la seconde formule du cours $f'(x) \approx [-3f(x) + 4f(x+h) - f(x+2h)]/(2h)$, d'erreur $+\frac{1}{3}h^2f'''(\xi)$.

Références. [BF, § 4.1, formules à trois points (4.4) et (4.5), p. 174–175].

Exercice IV.2 : vitesses à partir d'un tableau de positions

Énoncé. $t = 1,0 ; 1,1 ; \dots ; 1,5$ et $x = 1,6487 ; 1,7333 ; 1,8221 ; 1,9155 ; 2,0138 ; 2,1170$. Calculer les vitesses à $t = 1,05 ; 1,15 ; 1,25 ; 1,35$ et 1,45.

Pourquoi ? Les instants demandés sont les *milieux* des intervalles du tableau. La formule centrée de l'exercice IV.1, appliquée avec le pas $h/2$ autour du milieu $t + h/2$, n'utilise justement que $x(t)$ et $x(t + h)$:

$$v\left(t + \frac{h}{2}\right) \approx \frac{x(t + h) - x(t)}{h}, \quad \text{erreur} = \frac{(h/2)^2}{6} x'''(\xi) = -\frac{h^2}{24} x'''(\xi).$$

C'est la deuxième formule classique du cours ($f(x + h/2) - f(x - h/2)$ divisé par h). On obtient donc une précision d'ordre h^2 avec les seules données.

Comment ? (étapes)

Étape 1. Différences successives des positions : 0,0846 ; 0,0888 ; 0,0934 ; 0,0983 ; 0,1032.

Étape 2. Division par $h = 0,1$:

$$v(1,05) \approx 0,846, \quad v(1,15) \approx 0,888, \quad v(1,25) \approx 0,934, \quad v(1,35) \approx 0,983, \quad v(1,45) \approx 1,032.$$

Étape 3. Précision. Les positions sont données à 5×10^{-5} près ; une différence divisée par $h = 0,1$ peut donc être fautive d'environ $2 \times 5 \times 10^{-5} / 0,1 = 10^{-3}$. Les vitesses ne sont fiables qu'à 10^{-3} près environ : c'est l'erreur d'arrondi, qui croît quand h diminue, alors que l'erreur de troncature décroît [BF, p. 179].

Vérification numérique. On constate que les données coïncident avec $e^{t/2}$ à $4,7 \times 10^{-5}$ près (calcul fait sur les six points). Si c'est bien la loi du mouvement, la vitesse exacte est $\frac{1}{2}e^{t/2}$, soit 0,8452 ; 0,8886 ; 0,9341 ; 0,9820 ; 1,0324 : les écarts avec nos valeurs restent inférieurs à 10^{-3} , comme prévu à l'étape 3.

Références. [BF, § 4.1, formule (4.5) p. 175 ; erreurs d'arrondi p. 178–179].

Exercice IV.3 : trapèzes à la main pour $\int_1^2 dx/x$

Énoncé. Calculer à la main $I = \int_1^2 dx/x$ pour $h = 0,2$, puis pour $h = 0,1$. Comparer à la valeur exacte.

Pourquoi ? La valeur exacte est connue, $I = \ln 2 = 0,693147$: l'exercice permet de vérifier la formule composite des trapèzes et la loi d'erreur en h^2 établie dans le cours, $E = -\frac{h^2}{12}(b-a)f''(\eta)$.

Comment ? (étapes)

Étape 1. $h = 0,2$ ($n = 5$). Valeurs de $f(x) = 1/x$: 1 ; 0,833333 ; 0,714286 ; 0,625 ; 0,555556 ; 0,5. Somme des valeurs intérieures : 2,728175.

$$T_{0,2} = \frac{0,2}{2} [1 + 2 \times 2,728175 + 0,5] = 0,1 \times 6,956349 = 0,695635.$$

Étape 2. $h = 0,1$ ($n = 10$). Valeurs intérieures : 0,909091 ; 0,833333 ; 0,769231 ; 0,714286 ; 0,666667 ; 0,625 ; 0,588235 ; 0,555556 ; 0,526316, de somme 6,187714.

$$T_{0,1} = 0,05 [1 + 2 \times 6,187714 + 0,5] = 0,693771.$$

Étape 3. Comparaison. Erreurs : $T_{0,2} - \ln 2 = 0,002488$ et $T_{0,1} - \ln 2 = 0,000624$. Leur rapport vaut $3,99 \approx 4$: diviser h par 2 divise l'erreur par 4, c'est la loi en h^2 . Les erreurs sont positives, ce qu'annonce la formule car $f''(x) = 2/x^3 > 0$. Majoration : sur $[1, 2]$, $|f''| \leq 2$, d'où $|E| \leq h^2/6$, soit 0,0067 et 0,0017 : elle est bien respectée.

Étape 4. Prolongement (Richardson–Romberg). Comme l’erreur est en h^2 , la combinaison $(4T_{0,1} - T_{0,2})/3 = 0,693150$ élimine le terme principal : l’erreur tombe à 3×10^{-6} .

Références. [BF, th. 4.5, p. 205 (trapèzes composites); intégration de Romberg, § 4.5, p. 211–216].

Exercice IV.4 : trapèzes sans tableaux

Énoncé. Écrire un autre algorithme qui n’utilise pas les tableaux.

Pourquoi ? Dans l’algorithme du cours, chaque valeur f_k n’est utilisée qu’une fois : il est inutile de la stocker. On l’ajoute à la somme dès qu’elle est calculée. La mémoire utilisée ne dépend plus de n . On calcule les nœuds par $x = a + kh$ plutôt que par additions répétées $x \leftarrow x + h$, pour ne pas accumuler les erreurs d’arrondi.

Comment ? (étapes)

Étape 1. Initialiser la somme avec les deux extrémités (poids 1/2).

Étape 2. Ajouter les $n - 1$ valeurs intérieures (poids 1).

Étape 3. Multiplier par h .

Algorithme : trapèzes composites sans tableaux

Données : la fonction f , a , b , n

$h \leftarrow (b - a)/n$

$S \leftarrow (f(a) + f(b))/2$

Pour k allant de 1 à $n - 1$ **faire**

$S \leftarrow S + f(a + kh)$

Fin pour

$S \leftarrow h S$

Écrire S

Vérification numérique. Cet algorithme est la fonction `trapezes` du programme `ex11_trapeze.py` (exercice 11 de la fin du document); appliqué à $1/x$ sur $[1, 2]$, il redonne les valeurs de l’exercice IV.3.

Références. [BF, p. 205] (même organisation dans l’algorithme 4.1).

Exercice IV.5 : algorithme de Simpson

Énoncé. Écrire un algorithme pour la méthode de Simpson.

Pourquoi ? La formule de Simpson s’applique sur des paires d’intervalles (un polynôme de degré 2 sur trois points) : le nombre n de sous-intervalles doit donc être *pair*. Les poids sont 1, 4, 2, 4, $\dots$, 2, 4, 1 : 4 aux nœuds d’indice impair (milieux des paires), 2 aux nœuds pairs intérieurs (extrémités communes à deux paires). L’erreur composite vaut $-\frac{b-a}{180}h^4 f^{(4)}(\mu)$ [BF, th. 4.4, p. 204] : la méthode est exacte pour les polynômes de degré 3.

Comment ? (étapes)

Étape 1. Vérifier que n est pair.

Étape 2. Accumuler séparément la somme S_1 des valeurs aux nœuds impairs et la somme S_2 des valeurs aux nœuds pairs intérieurs (c'est ce que fait l'algorithme 4.1 de Burden et Faires avec ses variables XI_1 et XI_2).

Étape 3. $I \approx \frac{h}{3}[f(a) + 4S_1 + 2S_2 + f(b)]$.

Algorithme : Simpson composite

Données : f, a, b, n (pair)

Si n impair **alors**

Écrire « n doit être pair »

Arrêt

Fin si

$h \leftarrow (b - a)/n$; $S_1 \leftarrow 0$; $S_2 \leftarrow 0$

Pour k allant de 1 à $n - 1$ **faire**

$x \leftarrow a + kh$

Si k impair **alors**

$S_1 \leftarrow S_1 + f(x)$

Sinon

$S_2 \leftarrow S_2 + f(x)$

Fin si

Fin pour

$I \leftarrow \frac{h}{3}(f(a) + 4S_1 + 2S_2 + f(b))$

Écrire I

Vérification numérique. Voir l'exercice IV.7 (calcul de π) et le programme `ex12_mc.py` (exercice 12 de la fin du document).

Références. [BF, alg. 4.1, p. 205 ; th. 4.4, p. 204].

Exercice IV.6 : formule d'intégration de degré 3 sur a, c, d, b

Énoncé. Établir une formule d'intégration numérique qui utilise une interpolation de degré 3 et qui interpole les points $a, b, c = (a + b)/4$ et $d = 3(a + b)/4$.

Remarque sur l'énoncé. Les points $(a + b)/4$ et $3(a + b)/4$ ne sont dans $[a, b]$, aux quarts de l'intervalle, que si $a = 0$. On interprète donc l'énoncé comme $c = a + (b - a)/4$ et $d = a + 3(b - a)/4$, ce qui coïncide avec lui lorsque $a = 0$.

Pourquoi ? Intégrer le polynôme d'interpolation de degré 3 revient à chercher des poids w_k tels que $\int_a^b f \approx \sum_k w_k f(x_k)$ soit exact pour $1, x, x^2, x^3$. Les quatre nœuds étant symétriques par rapport au milieu de $[a, b]$, les poids le sont aussi : il ne reste que deux inconnues.

Comment ? (étapes)

Étape 1. Changement de variable $x = a + (b - a)t$, $t \in [0, 1]$: $\int_a^b f(x) dx = (b - a) \int_0^1 f(a + (b - a)t) dt$. Les nœuds deviennent $t = 0, \frac{1}{4}, \frac{3}{4}, 1$.

Étape 2. Par symétrie, poids α en 0 et 1, et β en $\frac{1}{4}$ et $\frac{3}{4}$. Exactitude pour 1 : $2\alpha + 2\beta = 1$. Exactitude pour t^2 : $\alpha(0 + 1) + \beta(\frac{1}{16} + \frac{9}{16}) = \frac{1}{3}$, soit $\alpha + \frac{5}{8}\beta = \frac{1}{3}$.

Étape 3. De la première, $\alpha = \frac{1}{2} - \beta$; en reportant : $\frac{1}{2} - \frac{3}{8}\beta = \frac{1}{3}$, d'où $\beta = \frac{4}{9}$ et $\alpha = \frac{1}{18}$. L'exactitude pour t et t^3 est automatique (les fonctions $t - \frac{1}{2}$ et $(t - \frac{1}{2})^3$ sont impaires par rapport au milieu, et les poids sont symétriques).

Étape 4. Formule obtenue :

$$\int_a^b f(x) dx \approx \frac{b-a}{18} [f(a) + 8f(c) + 8f(d) + f(b)].$$

Étape 5. Erreur. Pour t^4 , la formule donne $\frac{4}{9}(\frac{1}{256} + \frac{81}{256}) + \frac{1}{18} = \frac{19}{96}$ au lieu de $\frac{1}{5}$: l'écart (exact moins formule) vaut $\frac{1}{480}$. Comme $(t^4)^{(4)} = 24$, l'erreur est, pour $f^{(4)}$ à peu près constante,

$$E \approx \frac{(b-a)^5}{11\,520} f^{(4)}.$$

Le noyau de Peano de cette formule change légèrement de signe (calcul numérique) : on ne peut pas écrire rigoureusement $E = K f^{(4)}(\xi)$, mais on a la majoration $|E| \leq 9,13 \times 10^{-5} (b-a)^5 \max |f^{(4)}|$.

Vérification numérique. Sur $\int_0^1 e^x dx = e - 1$: l'erreur vaut $1,44 \times 10^{-4}$, proche de la prévision $e^{0,5}/11\,520 = 1,43 \times 10^{-4}$. À titre de comparaison, la formule de Newton-Cotes à quatre points équidistants (« 3/8 de Simpson », erreur $-\frac{3}{80}h^5 f^{(4)}$ avec $h = (b-a)/3$, soit $(b-a)^5/6480$) donne $-2,58 \times 10^{-4}$: la formule trouvée est environ deux fois plus précise.

Références. [BF, § 4.3 : degré de précision, p. 196 ; formules de Newton-Cotes fermées et règle des 3/8, th. 4.2, p. 196–197].

Exercice IV.7 : Simpson à la main pour $\int_0^1 4/(1+x^2) dx$

Énoncé. Utiliser la méthode de Simpson pour calculer manuellement $I = \int_0^1 \frac{4}{1+x^2} dx$ avec $h = 0,1$.

Pourquoi ? Une primitive est $4 \arctan x$, donc $I = 4 \arctan 1 = \pi$. On peut ainsi mesurer l'erreur exacte de la méthode.

Comment ? (étapes)

Étape 1. $n = 10$ (pair). Valeurs $f_k = 4/(1+x_k^2)$:

k	0	1	2	3	4	5
f_k	4	3,960396	3,846154	3,669725	3,448276	3,2
k	6	7	8	9	10	
f_k	2,941176	2,684564	2,439024	2,209945	2	

Étape 2. Sommes : indices impairs $S_1 = 3,960396 + 3,669725 + 3,2 + 2,684564 + 2,209945 = 15,724629$; indices pairs intérieurs $S_2 = 3,846154 + 3,448276 + 2,941176 + 2,439024 = 12,674631$.

Étape 3.

$$I \approx \frac{0,1}{3} [4 + 4 \times 15,724629 + 2 \times 12,674631 + 2] = \frac{0,1}{3} \times 94,247778 = 3,1415926.$$

Étape 4. Comparaison : $\pi = 3,14159265$; l'erreur vaut -4×10^{-8} . La majoration $\frac{b-a}{180} h^4 \max |f^{(4)}|$ avec $\max_{[0,1]} |f^{(4)}| = 96$ (atteint en $x = 0$) vaut $5,3 \times 10^{-5}$: elle est respectée, mais très pessimiste ici.

Vérification numérique. `calculs.py` : $S = 3,1415926139$, erreur $-3,965 \times 10^{-8}$.

Références. [BF, th. 4.4, p. 204 ; alg. 4.1, p. 205].

Chapitre V : équations différentielles et intégral-différentielles

Dans tout ce chapitre, les algorithmes ont été testés sur $y' = -2xy$, $y(0) = 1$, de solution exacte $y = e^{-x^2}$, en calculant $y(1)$ avec n pas ($h = 1/n$).

Exercice V.1 : algorithme de Runge-Kutta d'ordre 2

Énoncé. Écrire l'algorithme de la méthode de Runge-Kutta d'ordre 2.

Pourquoi ? On a $y_{k+1} = y_k + \int_{x_k}^{x_{k+1}} f(x, y) dx$. La méthode des trapèzes donnerait $y_{k+1} = y_k + \frac{h}{2} [f(x_k, y_k) + f(x_{k+1}, y_{k+1})]$, mais y_{k+1} est inconnu dans le second membre. On le remplace par la prévision d'Euler $y_k + hf(x_k, y_k)$: la formule devient explicite et reste d'ordre 2 (erreur locale $O(h^3)$, erreur globale $O(h^2)$). Il faut deux évaluations de f par pas ; on stocke la première dans une variable pour ne pas la recalculer.

Comment ? (étapes)

Étape 1. $L_1 = hf(x_k, y_k)$ (pente au début du pas).

Étape 2. $L_2 = hf(x_k + h, y_k + L_1)$ (pente au point prévu par Euler).

Étape 3. $y_{k+1} = y_k + (L_1 + L_2)/2$, puis $x_{k+1} = x_k + h$.

Algorithme : Runge-Kutta d'ordre 2

Données : f , x_0 , y_0 , h , n

$x \leftarrow x_0$; $y \leftarrow y_0$

Pour k allant de 1 à n **faire**

$L_1 \leftarrow hf(x, y)$

$L_2 \leftarrow hf(x + h, y + L_1)$

$y \leftarrow y + (L_1 + L_2)/2$

$x \leftarrow x + h$

Écrire x , y

Fin pour

Vérification numérique. Erreurs sur $y(1)$: $1,17 \times 10^{-3}$ ($n = 10$), $3,01 \times 10^{-4}$ ($n = 20$), $7,60 \times 10^{-5}$ ($n = 40$). Chaque division de h par 2 divise l'erreur par 3,9 puis 4,0 : ordre 2 global.

Références. [BF, méthode d'Euler modifiée, p. 286].

Exercice V.2 : algorithme de Runge d'ordre 4

Énoncé. Établir l'algorithme de la méthode de Runge d'ordre 4.

Pourquoi ? La méthode applique la formule de Simpson à l'intégrale $L = \int_{x_k}^{x_k+h} f dx$; les valeurs inconnues de y au milieu et à la fin du pas sont remplacées par des prévisions. Le cours donne les quatre quantités $L_1, \dots, L_4$ à calculer dans un ordre précis, car chacune utilise les précédentes.

Comment ? (étapes)

Étape 1. $L_1 = hf(x_k, y_k)$.

Étape 2. $L_2 = hf(x_k + h, y_k + L_1)$ (prévision d'Euler en fin de pas).

Étape 3. $L_3 = hf(x_k + h, y_k + L_2)$ (prévision améliorée en fin de pas).

Étape 4. $L_4 = hf(x_k + h/2, y_k + L_1/2)$ (prévision au milieu du pas).

Étape 5. $y_{k+1} = y_k + (L_1 + 4L_4 + L_3)/6$: ce sont les poids 1, 4, 1 de Simpson.

Algorithme : Runge d'ordre 4 (formules du cours)

Données : f, x_0, y_0, h, n

$x \leftarrow x_0; y \leftarrow y_0$

Pour k allant de 1 à n **faire**

$L_1 \leftarrow hf(x, y)$

$L_2 \leftarrow hf(x + h, y + L_1)$

$L_3 \leftarrow hf(x + h, y + L_2)$

$L_4 \leftarrow hf(x + h/2, y + L_1/2)$

$y \leftarrow y + (L_1 + 4L_4 + L_3)/6$

$x \leftarrow x + h$

Écrire x, y

Fin pour

Vérification numérique. et **remarque sur l'ordre.** Le cours annonce une erreur $O(h^5)$. Les essais numériques montrent un ordre plus faible pour ces formules précises : en divisant h par 2, l'erreur globale est divisée par environ 8 (et non 16), soit une erreur globale $O(h^3)$ et une erreur locale $O(h^4)$.

	$n = 10$	$n = 20$	$n = 40$
$y' = y$: Runge (cours)	$-1,05 \times 10^{-4}$	$-1,36 \times 10^{-5}$	$-1,74 \times 10^{-6}$
$y' = y$: Runge-Kutta classique	$-2,08 \times 10^{-6}$	$-1,36 \times 10^{-7}$	$-8,67 \times 10^{-9}$
$y' = -2xy$: Runge (cours)	$-7,48 \times 10^{-5}$	$-9,06 \times 10^{-6}$	$-1,11 \times 10^{-6}$
$y' = -2xy$: Runge-Kutta classique	$1,63 \times 10^{-6}$	$1,03 \times 10^{-7}$	$6,41 \times 10^{-9}$

La méthode de Runge-Kutta classique (dite de Kutta-Simpson dans le cours, $y_{k+1} = y_k + (L_1 + 2L_2 + 2L_3 + L_4)/6$) a bien une erreur divisée par 16 : c'est elle qu'il faut utiliser quand on veut l'ordre 4 [BF, alg. 5.2, p. 288].

Références. [BF, alg. 5.2, p. 288]; calculs du tableau : `tests_algorithmes.py` (fonctions `runge_cours` et `rk4`).

Exercice V.3 : algorithme d'Adams-Bashforth d'ordre 2

Énoncé. Écrire l'algorithme de la méthode d'Adams-Bashforth du second ordre.

Pourquoi ? La formule $y_{k+1} = y_k + \frac{h}{2} [3f(x_k, y_k) - f(x_{k-1}, y_{k-1})]$ réutilise la pente du pas précédent : une seule nouvelle évaluation de f par pas (au lieu de deux pour RK2). C'est une méthode à *deux pas* : elle ne peut pas démarrer seule, il faut d'abord y_1 , calculé par une méthode à un pas du même ordre (RK2, comme le propose le cours).

Comment ? (étapes)

Étape 1. Démarrage : y_1 par RK2 ; garder $f_0 = f(x_0, y_0)$ dans la variable f_{prec} .

Étape 2. Pour $k = 1, \dots, n-1$: $f_k = f(x_k, y_k)$, puis $y_{k+1} = y_k + \frac{h}{2}(3f_k - f_{\text{prec}})$.

Étape 3. Décaler : $f_{\text{prec}} \leftarrow f_k$.

Algorithme : Adams-Bashforth d'ordre 2

Données : f, x_0, y_0, h, n

$x \leftarrow x_0 ; y \leftarrow y_0$

$f_{\text{prec}} \leftarrow f(x, y)$

$y \leftarrow y + \frac{h}{2}(f_{\text{prec}} + f(x+h, y+hf_{\text{prec}}))$

{démarrage RK2}

$x \leftarrow x + h$

Pour k allant de 1 à $n-1$ faire

$f_k \leftarrow f(x, y)$

$y \leftarrow y + \frac{h}{2}(3f_k - f_{\text{prec}})$

$f_{\text{prec}} \leftarrow f_k$

$x \leftarrow x + h$

Écrire x, y

Fin pour

Vérification numérique. Erreurs sur $y(1)$: $-6,15 \times 10^{-3}$, $-1,54 \times 10^{-3}$, $-3,84 \times 10^{-4}$ pour $n = 10, 20, 40$: rapport 4, ordre 2.

Références. [BF, méthode d'Adams-Bashforth à deux pas, p. 307].

Exercice V.4 : prédicteur-correcteur AB2 – RK2 et programme

Énoncé. Établir l'algorithme de la méthode de prédicteur-correcteur, utilisant comme prédicteur la formule d'Adams-Bashforth du second ordre et comme correcteur la formule de RK2, et le traduire en Pascal, Fortran ou C.

Pourquoi ? La formule des trapèzes $y_{k+1} = y_k + \frac{h}{2}[f(x_k, y_k) + f(x_{k+1}, y_{k+1})]$ est plus précise qu'Adams-Bashforth, mais implicite. On *prédit* donc y_{k+1} par Adams-Bashforth (explicite), puis on *corrige* en mettant cette prévision dans le second membre de la formule des trapèzes. On gagne en précision pour deux évaluations de f par pas.

Comment ? (étapes)

Étape 1. Démarrage : y_1 par RK2.

Étape 2. Prédiction : $p = y_k + \frac{h}{2}(3f_k - f_{k-1})$.

Étape 3. Correction : $y_{k+1} = y_k + \frac{h}{2}(f_k + f(x_{k+1}, p))$.

Étape 4. On peut répéter la correction (en remplaçant p par la valeur corrigée) pour se rapprocher de la solution de la formule implicite ; une seule correction suffit à garder l'ordre 2.

Algorithme : prédicteur AB2 – correcteur RK2

Données : f, x_0, y_0, h, n

$x \leftarrow x_0 ; y \leftarrow y_0 ; f_{\text{prec}} \leftarrow f(x, y)$

$y \leftarrow y + \frac{h}{2}(f_{\text{prec}} + f(x+h, y+hf_{\text{prec}})) ; x \leftarrow x + h$

Pour k allant de 1 à $n-1$ faire

$f_k \leftarrow f(x, y)$

$p \leftarrow y + \frac{h}{2}(3f_k - f_{\text{prec}})$

{prédicteur}

$$y \leftarrow y + \frac{h}{2}(f_k + f(x+h, p)) \quad \{\text{correcteur}\}$$

$$f_{\text{prec}} \leftarrow f_k; x \leftarrow x + h$$

Écrire x, y

Fin pour

Programme Python (fichier programmes/pc_ab2.py) :

```

"""Chapitre V : prédicteur Adams-Bashforth d'ordre 2,
correcteur RK2 (trapèzes).
Exemple : y' = -2 x y, y(0) = 1, solution exacte y = exp(-x^2)."""
import math

def predicteur_correcteur(f, x0, y0, xmax, n):
    """Retourne la liste des couples (x_k, y_k), k = 0, ..., n."""
    h = (xmax - x0) / n
    x, y = x0, y0
    points = [(x, y)]
    # démarrage : y_1 par Runge-Kutta d'ordre 2
    fprec = f(x, y)
    y = y + h / 2 * (fprec + f(x + h, y + h * fprec))
    x = x0 + h
    points.append((x, y))
    for k in range(1, n):
        fk = f(x, y)
        p = y + h / 2 * (3 * fk - fprec)           # prédicteur AB2
        y = y + h / 2 * (fk + f(x + h, p))        # correcteur RK2
        fprec = fk
        x = x0 + (k + 1) * h
        points.append((x, y))
    return points

if __name__ == "__main__":
    f = lambda x, y: -2.0 * x * y
    for x, y in predicteur_correcteur(f, 0.0, 1.0, 1.0, 10):
        print(f"{x:5.2f} {y:12.8f} {math.exp(-x * x):12.8f}")
    for n in (10, 20, 40):
        y1 = predicteur_correcteur(f, 0.0, 1.0, 1.0, n)[-1][1]
        print(f"n = {n:3d}   erreur en x = 1 : {y1 - math.exp(-1):.3e}")

```

Vérification numérique. Le programme donne $y(1) = 0,36961867$ pour $n = 10$, au lieu de $e^{-1} = 0,36787944$: erreur $1,74 \times 10^{-3}$, à comparer à $6,15 \times 10^{-3}$ pour Adams-Bashforth seul. Avec $n = 20$ et 40 : $3,66 \times 10^{-4}$ et $8,38 \times 10^{-5}$ (ordre 2).

Références. [BF, principe des méthodes prédicteur-correcteur, p. 310–311 ; alg. 5.4, p. 311] (version d'ordre 4).

Exercice V.5 : algorithme de la méthode de tir

Énoncé. Établir l'algorithme de la méthode de tir.

Pourquoi ? Les méthodes des sections précédentes avancent à partir de conditions *initiales* ; elles ne savent pas imposer la valeur $y(b) = \eta_2$. On remplace la condition manquante $y'(a)$ par un paramètre γ , on résout le problème à valeurs initiales, et on ajuste γ pour que $F(\gamma) = y(b, \gamma) - \eta_2$ s'annule. La dérivée $F'(\gamma)$ n'étant pas connue, le cours la remplace par un quotient de différences entre deux tirs : c'est la méthode de la sécante. Il faut donc deux valeurs de départ, γ_0 et γ_1 .

Comment ? (étapes)

Étape 1. Écrire l'équation du second ordre comme un système : $y' = z$, $z' = f(x, y, z)$, avec $y(a) = \eta_1$, $z(a) = \gamma$.

Étape 2. Tirer avec γ_0 et γ_1 (intégration par Runge-Kutta d'ordre 4 pour systèmes) et noter $y(b, \gamma_0)$, $y(b, \gamma_1)$.

Étape 3. Nouvelle valeur par la formule du cours : $\gamma_{n+1} = \gamma_n - (\gamma_n - \gamma_{n-1})(y(b, \gamma_n) - \eta_2)/(y(b, \gamma_n) - y(b, \gamma_{n-1}))$.

Étape 4. Arrêter quand $|y(b, \gamma_n) - \eta_2| < \varepsilon$; garder la dernière trajectoire calculée, qui est la solution approchée.

Algorithme : tir avec la sécante

Données : f , a , b , η_1 , η_2 , γ_0 , γ_1 , N (pas), ε , $k_{\max}$

$Y_0 \leftarrow y(b)$ obtenu par RK4 depuis $y(a) = \eta_1$, $z(a) = \gamma_0$

Pour k allant de 1 à $k_{\max}$ **faire**

$Y_1 \leftarrow y(b)$ obtenu par RK4 depuis $y(a) = \eta_1$, $z(a) = \gamma_1$

Si $|Y_1 - \eta_2| < \varepsilon$ **alors**

Écrire γ_1 et la trajectoire (x_i, y_i)

Arrêt

Fin si

$\gamma \leftarrow \gamma_1 - (\gamma_1 - \gamma_0)(Y_1 - \eta_2)/(Y_1 - Y_0)$

$\gamma_0 \leftarrow \gamma_1$; $Y_0 \leftarrow Y_1$; $\gamma_1 \leftarrow \gamma$

Fin pour

Écrire « pas de convergence »

La procédure « RK4 pour un système » est détaillée à l'exercice 5 de la fin du document.

Vérification numérique. Pour $y'' = -y$, $y(0) = 0$, $y(\pi/2) = 1$ (solution $\sin x$, donc $\gamma = 1$), avec $\gamma_0 = 0,5$, $\gamma_1 = 2$: $\gamma = 1,0000000003$ après 2 itérations. Pour un problème linéaire, $y(b, \gamma)$ est une fonction affine de γ et la sécante trouve la valeur exacte (aux erreurs de RK4 près) dès la première correction.

Références. [BF, § 11.2, sécante pour le tir p. 694 ; alg. 11.2, p. 696] (version avec Newton).

Exercice V.6 : discrétisation de l'équation intégral-différentielle

Énoncé. Discrétiser l'équation intégral-différentielle $y'(x) = 2y(x) + f(x) + \int_0^x k(x, t) y(t) dt$.

Remarque sur l'énoncé. Il faut une condition initiale $y(0) = y_0$, que l'énoncé ne donne pas ; on la suppose connue.

Pourquoi ? L'équation mélange une dérivée et une intégrale dont la borne supérieure est x (type Volterra). Sur une grille $x_i = ih$, l'intégrale au point x_i ne fait intervenir que $y_0, \dots, y_i$, c'est-à-dire des valeurs déjà calculées quand on avance de x_i à x_{i+1} . On peut donc combiner une formule d'intégration (trapèzes) et un schéma pour la dérivée (Euler, puis trapèzes pour monter en ordre).

Comment ? (étapes)

Étape 1. Intégrale au nœud x_i par les trapèzes composites :

$$I_i = h \left[\frac{1}{2} k(x_i, x_0) y_0 + \sum_{j=1}^{i-1} k(x_i, x_j) y_j + \frac{1}{2} k(x_i, x_i) y_i \right], \quad I_0 = 0.$$

Étape 2. Second membre au nœud x_i : $F_i = 2y_i + f(x_i) + I_i$.

Étape 3. Schéma d'Euler : $y_{i+1} = y_i + hF_i$, pour $i = 0, 1, \dots, N-1$. Il est explicite, d'ordre 1.

Étape 4. Variante d'ordre 2 (trapèzes en x) : $y_{i+1} = y_i + \frac{h}{2}(F_i + F_{i+1})$. Comme F_{i+1} dépend linéairement de y_{i+1} (par $2y_{i+1}$ et par le dernier terme $\frac{h}{2}k(x_{i+1}, x_{i+1})y_{i+1}$ de I_{i+1}), on isole y_{i+1} :

$$y_{i+1} \left[1 - \frac{h}{2} \left(2 + \frac{h}{2} k(x_{i+1}, x_{i+1}) \right) \right] = y_i + \frac{h}{2} \left[F_i + f(x_{i+1}) + h \left(\frac{1}{2} k(x_{i+1}, x_0) y_0 + \sum_{j=1}^i k(x_{i+1}, x_j) y_j \right) \right].$$

Algorithme : schéma d'Euler pour l'équation intégro-différentielle

Données : f, k, y_0, h, N

Pour i allant de 0 à $N-1$ faire

$I \leftarrow 0$

Si $i > 0$ alors

$I \leftarrow \frac{1}{2} k(x_i, x_0) y_0 + \frac{1}{2} k(x_i, x_i) y_i$

Pour j allant de 1 à $i-1$ faire

$I \leftarrow I + k(x_i, x_j) y_j$

Fin pour

$I \leftarrow h I$

Fin si

$y_{i+1} \leftarrow y_i + h(2y_i + f(x_i) + I)$

Fin pour

(Il faut garder toutes les valeurs y_j en mémoire, car l'intégrale les utilise toutes.)

Vérification numérique. Test construit pour avoir une solution connue : $k = 1, f(x) = 1 - 2e^x, y(0) = 1$; alors $y = e^x$ convient (car $2e^x + 1 - 2e^x + (e^x - 1) = e^x$). Erreurs sur $y(1)$:

N	10	20	40	80
Euler	$-2,08 \times 10^{-1}$	$-1,17 \times 10^{-1}$	$-6,22 \times 10^{-2}$	$-3,21 \times 10^{-2}$
Trapèzes	$5,85 \times 10^{-3}$	$1,45 \times 10^{-3}$	$3,61 \times 10^{-4}$	$9,02 \times 10^{-5}$

Les rapports successifs (environ 2 pour Euler, 4 pour les trapèzes) confirment les ordres 1 et 2.

Références. Formule des trapèzes : [BF, th. 4.5, p. 205]; principe de l'intégration pas à pas : chapitre V du cours.

Chapitre VI : équations aux dérivées partielles

Outil commun : l'analyse de stabilité de von Neumann. Plusieurs exercices demandent une condition de stabilité. On suit la démarche du cours. *Pourquoi* : une erreur commise à un instant (arrondi, donnée) se propage par le schéma lui-même ; si elle est amplifiée à chaque pas, elle finit par détruire la solution. Pour un schéma linéaire à coefficients constants, on décompose la perturbation en modes de Fourier $\varepsilon_{i,j} = \xi^j e^{Iph}$ ($I^2 = -1$) ; chaque mode est multiplié par le *facteur d'amplification* ξ à chaque pas de temps. *Comment* : on remplace $\varepsilon_{i,j}$ dans le schéma, on simplifie par $\xi^j e^{Iph}$, on obtient ξ en fonction de $\theta = ph$, et le schéma est stable si $|\xi| \leq 1$ pour tous les θ . Les identités utiles sont

$$e^{I\theta} + e^{-I\theta} - 2 = 2 \cos \theta - 2 = -4 \sin^2 \frac{\theta}{2}, \quad e^{I\theta} - e^{-I\theta} = 2I \sin \theta.$$

Le cours utilise $\sin(iph) e^{-rjk}$, ce qui revient au même ($e^{-rk} = \xi$). On note $s = \sin^2(\theta/2) \in [0, 1]$.

Exercice VI.1 : Gauss-Seidel et Jacobi pour l'équation de Poisson

Énoncé. Écrire le schéma d'itération de Gauss-Seidel et de Jacobi pour résoudre l'équation de Poisson.

Pourquoi ? Le système (P2)–(P3) a $(n-1)(m-1)$ inconnues mais chaque équation n'en contient que cinq. L'élimination de Gauss détruirait cette structure creuse (remplissage) et coûterait cher ; les méthodes itératives n'utilisent que les cinq voisins et ne stockent que la grille. C'est la méthode de Gauss-Seidel que Burden et Faires utilisent pour ce système dans leur algorithme 12.1 [BF, p. 738–739].

Comment ? (étapes)

Étape 1. On isole $U_{i,j}$ dans (P2). Avec $r = h/k$:

$$U_{i,j} = \frac{U_{i+1,j} + U_{i-1,j} + r^2(U_{i,j+1} + U_{i,j-1}) - h^2 f_{i,j}}{2(1 + r^2)}.$$

Étape 2. Jacobi : tout le second membre est pris à l'itération q ; il faut une deuxième grille V pour les nouvelles valeurs.

Étape 3. Gauss-Seidel : on parcourt la grille (i croissant, puis j croissant) et on écrase les valeurs ; $U_{i-1,j}$ et $U_{i,j-1}$ sont alors déjà à l'itération $q+1$:

$$U_{i,j}^{(q+1)} = \frac{U_{i+1,j}^{(q)} + U_{i-1,j}^{(q+1)} + r^2(U_{i,j+1}^{(q)} + U_{i,j-1}^{(q+1)}) - h^2 f_{i,j}}{2(1 + r^2)}.$$

Étape 4. Les valeurs au bord, fixées par (P3), ne sont jamais modifiées. Test d'arrêt du chapitre I sur la somme des variations.

Algorithme : Gauss-Seidel pour (P2)–(P3)

Données : $a, b, c, d, n, m, f, g, \varepsilon, q_{\max}$

$h \leftarrow (b-a)/n$; $k \leftarrow (d-c)/m$; $r^2 \leftarrow (h/k)^2$

$U_{i,j} \leftarrow g(x_i, y_j)$ sur le bord ; $U_{i,j} \leftarrow 0$ à l'intérieur

Pour q allant de 1 à $q_{\max}$ **faire**

$num \leftarrow 0$; $den \leftarrow 0$


```

Pour  $i$  allant de 1 à  $n - 1$  faire
  Pour  $j$  allant de 1 à  $m - 1$  faire
     $t \leftarrow [U_{i+1,j} + U_{i-1,j} + r^2(U_{i,j+1} + U_{i,j-1}) - h^2 f(x_i, y_j)] / (2(1 + r^2))$ 
     $num \leftarrow num + |t - U_{i,j}|$ ;  $den \leftarrow den + |t|$ 
     $U_{i,j} \leftarrow t$ 
  Fin pour
Fin pour
Si  $num/den < \varepsilon$  alors
  Écrire  $q$  et la grille  $U$ 
Arrêt
Fin si
Fin pour

```

Pour Jacobi, on écrit t dans une grille V ($V_{i,j} \leftarrow t$), et on recopie V dans U à la fin de chaque itération.

Vérification numérique. Le schéma à cinq points est exact quand les dérivées quatrièmes de la solution sont nulles [BF, p. 738]. Les algorithmes ont donc été testés sur des solutions polynomiales : $u = x^2 - y^2$ (Laplace, exercice 17) et $u = x^2 + y^2$ avec $f = 4$ (exercice 18) ; la solution discrète obtenue coïncide avec la solution exacte à 10^{-11} près.

Références. [BF, § 12.1, alg. 12.1, p. 738–739].

Exercice VI.2 : algorithme du schéma explicite (chaleur)

Énoncé. Établir l'algorithme pour le schéma explicite.

Pourquoi ? Le schéma (C2) donne directement chaque $U_{i,j+1}$ à partir de la ligne de temps j : on avance ligne par ligne. Il faut deux tableaux (ancien et nouveau temps), sinon $U_{i-1,j}$ serait écrasé par $U_{i-1,j+1}$ avant d'être utilisé. Le schéma n'est stable que si $\lambda = a^2 k / h^2 \leq 1/2$ [BF, p. 746–747] : l'algorithme doit le vérifier avant de calculer.

Comment ? (étapes)

Étape 1. Lire l, a, m, k , le nombre de pas N ; calculer $h = l/m$ et λ ; refuser si $\lambda > 1/2$.

Étape 2. Condition initiale (C3) : $U_i = f(x_i)$, $U_0 = U_m = 0$.

Étape 3. Pour chaque pas de temps : appliquer (C2) aux points intérieurs dans le tableau V , imposer les conditions aux limites, recopier V dans U .

Algorithme : schéma explicite

Données : l, a, f, m, k, N

$h \leftarrow l/m$; $\lambda \leftarrow a^2 k / h^2$

Si $\lambda > 1/2$ **alors**

Écrire « schéma instable : diminuer k »

Arrêt

Fin si

Pour i allant de 0 à m **faire**

$U_i \leftarrow f(ih)$

Fin pour

$U_0 \leftarrow 0$; $U_m \leftarrow 0$

Pour j allant de 1 à N **faire**

Pour i allant de 1 à $m - 1$ **faire**

```

     $V_i \leftarrow (1 - 2\lambda)U_i + \lambda(U_{i+1} + U_{i-1})$ 
Fin pour
 $V_0 \leftarrow 0; V_m \leftarrow 0$ 
Pour  $i$  allant de 0 à  $m$  faire
     $U_i \leftarrow V_i$ 
Fin pour
Écrire  $jk, U_0, \dots, U_m$ 
Fin pour

```

Vérification numérique. Programmé (avec d'autres conditions aux limites) dans `ex14_explicite.py` : exercice 14 de la fin du document.

Références. [BF, § 12.2, méthode des différences progressives, p. 743–747].

Exercice VI.3 : stabilité de Crank-Nicolson et de Dufort-Frankel

Énoncé. Établir les conditions de stabilité des schémas de Crank-Nicolson et de Dufort-Frankel.

Crank-Nicolson.

Pourquoi ? Le schéma moyenne les schémas progressif et régressif ; on s'attend à une stabilité aussi bonne que celle du schéma régressif.

Étape 1. Avec $\lambda = a^2 k / h^2$, le schéma du cours s'écrit $\varepsilon_{i,j+1} - \varepsilon_{i,j} = \frac{\lambda}{2} [\delta^2 \varepsilon_{i,j} + \delta^2 \varepsilon_{i,j+1}]$, où $\delta^2 \varepsilon_i = \varepsilon_{i+1} - 2\varepsilon_i + \varepsilon_{i-1}$.

Étape 2. Pour un mode de Fourier, δ^2 devient la multiplication par $-4s$. D'où $\xi - 1 = \frac{\lambda}{2}(-4s)(1 + \xi)$, soit

$$\xi = \frac{1 - 2\lambda s}{1 + 2\lambda s}.$$

Étape 3. Pour $\lambda > 0$ et $s \in [0, 1]$, le numérateur est toujours de valeur absolue inférieure ou égale au dénominateur : $|\xi| \leq 1$. Le schéma de Crank-Nicolson est **inconditionnellement stable**, sans condition sur λ .

Dufort-Frankel.

Pourquoi ? Le schéma de Richardson est instable ; Dufort et Frankel remplacent $U_{i,j}$ par la moyenne $(U_{i,j+1} + U_{i,j-1})/2$ dans le terme de diffusion pour le stabiliser. Il y a trois niveaux de temps : on obtient une équation du second degré en ξ .

Étape 1. Le schéma s'écrit $\varepsilon_{i,j+1} - \varepsilon_{i,j-1} = 2\lambda(\varepsilon_{i+1,j} + \varepsilon_{i-1,j} - \varepsilon_{i,j+1} - \varepsilon_{i,j-1})$.

Étape 2. Avec $\varepsilon_{i,j} = \xi^j e^{Ii\theta}$ et division par $\xi^{j-1} e^{Ii\theta}$: $\xi^2 - 1 = 2\lambda(2\xi \cos \theta - \xi^2 - 1)$, c'est-à-dire

$$(1 + 2\lambda)\xi^2 - 4\lambda \cos \theta \xi - (1 - 2\lambda) = 0.$$

Étape 3. Racines : $\xi = [2\lambda \cos \theta \pm \sqrt{1 - 4\lambda^2 \sin^2 \theta}] / (1 + 2\lambda)$.

- Si $4\lambda^2 \sin^2 \theta \leq 1$, les racines sont réelles et $|\xi| \leq (2\lambda |\cos \theta| + 1) / (1 + 2\lambda) \leq 1$.
- Sinon, elles sont complexes conjuguées et $|\xi|^2 = |\text{produit des racines}| = (2\lambda - 1) / (2\lambda + 1) < 1$.

Dans tous les cas $|\xi| \leq 1$: le schéma de Dufort-Frankel est **inconditionnellement stable**, comme l'affirme le cours.

Vérification numérique. Le calcul numérique de $\max_{\theta} |\xi|$ pour $\lambda = 0,1; 0,5; 1; 5; 50$ donne 1 dans tous les cas, pour les deux schémas (le maximum 1 est atteint pour $\theta = 0$).

Remarque importante (consistance de Dufort-Frankel). La stabilité ne suffit pas : l'erreur de troncature du schéma de Dufort-Frankel est $O(k^2 + h^2 + k^2/h^2)$. Le schéma ne converge vers la solution de l'équation de la chaleur que si $k/h \rightarrow 0$ [DF2]. Le test numérique le confirme sur $u_t = u_{xx}$, $u(x, 0) = \sin \pi x$ (solution $e^{-\pi^2 t} \sin \pi x$), erreur maximale à $t = 0,5$:

	$h = 0,1$	$h = 0,05$	$h = 0,025$
$k/h = 0,1$ fixé	$2,96 \times 10^{-3}$	$3,17 \times 10^{-3}$	$3,23 \times 10^{-3}$
$\lambda = k/h^2 = 1$ fixé	$2,96 \times 10^{-3}$	$7,87 \times 10^{-4}$	$2,00 \times 10^{-4}$

Avec k/h constant, l'erreur ne diminue pas ; avec $k = h^2$, elle est divisée par 4 à chaque fois.

Références. Crank-Nicolson : [BF, p. 751–752] (inconditionnellement stable, ordre $O(k^2 + h^2)$). Dufort-Frankel : [VN] et [DF2] (bibliographie).

Exercice VI.4 : algorithmes régressif, Crank-Nicolson, Dufort-Frankel

Énoncé. Établir les algorithmes pour les méthodes des différences régressives, de Crank-Nicolson et de Dufort-Frankel.

Pourquoi ? Les deux premiers schémas sont implicites : à chaque pas, il faut résoudre un système linéaire. Ce système est *tridiagonal* (chaque équation ne lie que U_{i-1} , U_i , U_{i+1}) et à diagonale strictement dominante ; on le résout par la factorisation de Crout pour matrices tridiagonales, en $O(m)$ opérations au lieu de $O(m^3)$. Comme la matrice ne change pas d'un pas à l'autre, on pourrait même la factoriser une seule fois. Le troisième schéma est explicite mais à trois niveaux : il lui faut $U^{(0)}$ et $U^{(1)}$ pour démarrer.

Résolution d'un système tridiagonal (Crout). Système $\ell_i U_{i-1} + d_i U_i + u_i U_{i+1} = r_i$, $i = 1, \dots, M$ ($\ell_1 = u_M = 0$). On factorise en une matrice bidiagonale inférieure (de diagonale L_i) fois une bidiagonale supérieure à diagonale unité (de sur-diagonale S_i), puis descente et remontée.

Algorithme : Crout tridiagonal

$L_1 \leftarrow d_1$; $S_1 \leftarrow u_1/L_1$; $z_1 \leftarrow r_1/L_1$

Pour i allant de 2 à M **faire**

$L_i \leftarrow d_i - \ell_i S_{i-1}$

Si $i < M$ **alors**

$S_i \leftarrow u_i/L_i$

Fin si

$z_i \leftarrow (r_i - \ell_i z_{i-1})/L_i$

Fin pour

$U_M \leftarrow z_M$

Pour i allant de $M-1$ à 1 **faire**

$U_i \leftarrow z_i - S_i U_{i+1}$

Fin pour

Différences régressives.

Étape 1. (C5) aux points $i = 1, \dots, m-1$: $\ell_i = u_i = -\lambda$, $d_i = 1 + 2\lambda$, $r_i = U_i^{(j-1)}$ (les termes de bord sont nuls ici car $U_0 = U_m = 0$).

Étape 2. À chaque pas : résoudre le système tridiagonal, la solution devient la nouvelle ligne.

Crank-Nicolson.

Étape 1. En multipliant le schéma du cours par k : $(1 + \lambda)U_i^{(j+1)} - \frac{\lambda}{2}(U_{i+1}^{(j+1)} + U_{i-1}^{(j+1)}) = (1 - \lambda)U_i^{(j)} + \frac{\lambda}{2}(U_{i+1}^{(j)} + U_{i-1}^{(j)})$.

Étape 2. Donc $\ell_i = u_i = -\lambda/2$, $d_i = 1 + \lambda$, et r_i égal au second membre calculé avec l'ancienne ligne.

Dufort-Frankel.

Étape 1. En multipliant le schéma du cours par $2k$ et en regroupant les termes en $U_i^{(j+1)}$: $U_i^{(j+1)} = [(1 - 2\lambda)U_i^{(j-1)} + 2\lambda(U_{i+1}^{(j)} + U_{i-1}^{(j)})]/(1 + 2\lambda)$.

Étape 2. Démarrage : $U^{(0)} = f$, puis $U^{(1)}$ par un pas de Crank-Nicolson (un schéma à un pas d'ordre 2).

Étape 3. Trois tableaux : ancien ($j-1$), courant (j) et nouveau ($j+1$).

Algorithme : différences régressives et Crank-Nicolson

Données : l, a, f, m, k, N , choix du schéma

$h \leftarrow l/m$; $\lambda \leftarrow a^2 k/h^2$

Pour i allant de 0 à m **faire**

$U_i \leftarrow f(ih)$

Fin pour

$U_0 \leftarrow 0$; $U_m \leftarrow 0$

Pour j allant de 1 à N **faire**

Pour i allant de 1 à $m-1$ **faire**

Si régressif **alors**

$\ell_i \leftarrow -\lambda$; $d_i \leftarrow 1 + 2\lambda$; $u_i \leftarrow -\lambda$; $r_i \leftarrow U_i$

Sinon

{Crank-Nicolson}

$\ell_i \leftarrow -\lambda/2$; $d_i \leftarrow 1 + \lambda$; $u_i \leftarrow -\lambda/2$

$r_i \leftarrow (1 - \lambda)U_i + \frac{\lambda}{2}(U_{i+1} + U_{i-1})$

Fin si

Fin pour

résoudre le système tridiagonal (Crout); ranger la solution dans $U_1, \dots, U_{m-1}$

Écrire jk, U

Fin pour

Algorithme : Dufort-Frankel

$U_i^{\text{anc}} \leftarrow f(ih)$; $U^{\text{cour}} \leftarrow$ un pas de Crank-Nicolson depuis U^{anc}

Pour j allant de 2 à N **faire**

Pour i allant de 1 à $m-1$ **faire**

$U_i^{\text{nouv}} \leftarrow [(1 - 2\lambda)U_i^{\text{anc}} + 2\lambda(U_{i+1}^{\text{cour}} + U_{i-1}^{\text{cour}})]/(1 + 2\lambda)$

Fin pour

$U_0^{\text{nouv}} \leftarrow 0$; $U_m^{\text{nouv}} \leftarrow 0$

$U^{\text{anc}} \leftarrow U^{\text{cour}}$; $U^{\text{cour}} \leftarrow U^{\text{nouv}}$

Fin pour

Vérification numérique. Sur $u_t = u_{xx}$, $u(x, 0) = \sin \pi x$, erreur maximale à $t = 0,5$ pour $(h, k) = (0,1; 0,01)$, $(0,05; 0,005)$, $(0,025; 0,0025)$ (donc $\lambda = 1, 2, 4$, valeurs pour lesquelles le schéma explicite serait instable) : régressif $2,19 \times 10^{-3}$; $9,78 \times 10^{-4}$; $4,63 \times 10^{-4}$ (ordre 1 en k) ; Crank-Nicolson $2,68 \times 10^{-4}$; $6,61 \times 10^{-5}$; $1,65 \times 10^{-5}$ (ordre 2). Aucun des deux ne diverge. Pour Dufort-Frankel, voir la remarque de l'exercice VI.3.

Références. [BF, alg. 6.7, p. 427 (Crout tridiagonal) ; alg. 12.2, p. 749 (régressif) ; alg. 12.3, p. 753 (Crank-Nicolson)].

Exercice VI.5 : équation d'ondes, stabilité et algorithme

Énoncé. a) Établir la condition de stabilité du schéma explicite (O2). b) Établir l'algorithme de ce schéma.

a) Stabilité.

Pourquoi ? Le schéma (O2) a trois niveaux de temps : comme pour Dufort-Frankel, le facteur d'amplification est racine d'une équation du second degré.

Étape 1. Avec $\varepsilon_{i,j} = \xi^j e^{Ii\theta}$ et $\lambda = ak/h$, (O2) donne $\xi = 2(1 - \lambda^2) + \lambda^2(2 \cos \theta) - \xi^{-1}$, soit

$$\xi^2 - 2(1 - 2\lambda^2 s)\xi + 1 = 0, \quad s = \sin^2 \frac{\theta}{2}.$$

Étape 2. Le produit des racines vaut 1. Si les racines sont réelles et distinctes, l'une est de module > 1 : instabilité. Il faut donc des racines complexes conjuguées (ou une racine double), c'est-à-dire un discriminant négatif ou nul : $(1 - 2\lambda^2 s)^2 \leq 1$, soit $0 \leq \lambda^2 s \leq 1$. Alors $|\xi| = 1$.

Étape 3. Cette condition doit être vraie pour tous les modes, donc pour $s = 1$: $\lambda^2 \leq 1$, c'est-à-dire

$$\lambda = \frac{ak}{h} \leq 1.$$

C'est la condition du cours (condition de Courant-Friedrichs-Lewy) ; Burden et Faires donnent $\lambda \leq 1$ [BF, p. 762]. Le cas limite $\lambda = 1$ est admis par le livre ; le cours écrit $\lambda < 1$.

Vérification numérique. $\max_{\theta} |\xi| = 1$ pour $\lambda = 0,9$ et $\lambda = 1$, mais 1,33 pour $\lambda = 1,01$.

b) Algorithme.

Pourquoi ? Pour calculer la ligne $j + 1$, il faut les lignes j et $j - 1$: on initialise avec (O4) et (O6), puis on applique (O2). La formule (O6) est utilisée plutôt que (O5) pour que le démarrage soit du même ordre (k^2) que le schéma.

Algorithme : équation d'ondes, schéma explicite

Données : l, a, f, g, m, k, N

$h \leftarrow l/m$; $\lambda \leftarrow ak/h$

Si $\lambda > 1$ **alors**

Écrire « schéma instable »

Arrêt

Fin si

Pour i allant de 0 à m **faire**

$U_i^{\text{anc}} \leftarrow f(ih)$

Fin pour

$U_0^{\text{anc}} \leftarrow 0; U_m^{\text{anc}} \leftarrow 0$ $\{(O3), (O4)\}$
Pour i allant de 1 à $m - 1$ **faire**
 $U_i^{\text{cour}} \leftarrow (1 - \lambda^2)U_i^{\text{anc}} + \frac{\lambda^2}{2}(U_{i+1}^{\text{anc}} + U_{i-1}^{\text{anc}}) + k g(ih)$ $\{(O6)\}$
Fin pour
 $U_0^{\text{cour}} \leftarrow 0; U_m^{\text{cour}} \leftarrow 0$
Pour j allant de 1 à $N - 1$ **faire**
Pour i allant de 1 à $m - 1$ **faire**
 $U_i^{\text{nouv}} \leftarrow 2(1 - \lambda^2)U_i^{\text{cour}} + \lambda^2(U_{i+1}^{\text{cour}} + U_{i-1}^{\text{cour}}) - U_i^{\text{anc}}$ $\{(O2)\}$
Fin pour
 $U_0^{\text{nouv}} \leftarrow 0; U_m^{\text{nouv}} \leftarrow 0$
 $U^{\text{anc}} \leftarrow U^{\text{cour}}; U^{\text{cour}} \leftarrow U^{\text{nouv}}$
Écrire $(j + 1)k, U^{\text{cour}}$
Fin pour

Vérification numérique. Ce schéma est le cas $a = d = 0$ du programme `ex16_onde.py` (exercice 16 de la fin du document). Pour $f = \sin \pi x$, $g = 0$, $l = 1$, vitesse 1 (solution exacte $\sin \pi x \cos \pi t$), $h = 0,02$, $k = 0,01$ ($\lambda = 0,5$) : en $x = 0,5$, $t = 1,5$, on obtient $-5,8 \times 10^{-4}$ pour une valeur exacte nulle.

Références. [BF, (12.19)–(12.25), p. 758–760 ; alg. 12.4, p. 761 ; stabilité p. 762].

Exercices de fin de document (1 à 7)

Exercice 1 : maximum de la loi de Planck

1- Équation vérifiée par x_m .

Pourquoi ? Avec $x = hc/(\lambda kT)$, on a $\lambda = hc/(xkT)$, et l'intensité devient une fonction de x seul (à un facteur constant près). Comme $x \mapsto \lambda$ est strictement monotone ($dx/d\lambda = -hc/(\lambda^2 kT) \neq 0$), chercher le maximum en λ revient à chercher le maximum en x .

Étape 1. $I = 8\pi hc \lambda^{-5}/(e^x - 1) = 8\pi hc (kT/hc)^5 x^5/(e^x - 1)$.

Étape 2. On dérive $\varphi(x) = x^5/(e^x - 1)$:

$$\varphi'(x) = \frac{5x^4(e^x - 1) - x^5 e^x}{(e^x - 1)^2} = 0 \iff 5(e^x - 1) = x e^x.$$

Étape 3. L'équation de x_m est donc

$$f(x) = (x - 5)e^x + 5 = 0, \quad \text{ou encore} \quad x = 5(1 - e^{-x}).$$

Attention : $x = 0$ est solution évidente ($f(0) = -5 + 5 = 0$) mais correspond à $\lambda \rightarrow \infty$, pas au maximum ; on cherche la racine strictement positive.

2- Algorithme de Newton-Raphson.

Pourquoi ? f est dérivable, de dérivée simple $f'(x) = (x - 4)e^x$: Newton converge vite si l'on part près de la racine. Comme e^{-x} est petit pour x grand, l'écriture $x = 5(1 - e^{-x})$ suggère $x_0 = 5$. Sur $[4,9; 5]$, $f(4,9) < 0 < f(5)$, $f' > 0$ et $f'' = (x - 3)e^x > 0$, donc $f(x_0)f''(x_0) > 0$ en $x_0 = 5$: d'après la remarque du cours, la suite est monotone et converge.

Comment ? (étapes)

Étape 1. $x_0 = 5$, $m = \min |f'| = f'(4,9) = 120,86$ sur $[4,9; 5]$.

Étape 2. $x_{n+1} = x_n - \frac{(x_n - 5)e^{x_n} + 5}{(x_n - 4)e^{x_n}}$.

Étape 3. Arrêt quand $|f(x_n)|/m < E$ (algorithme de l'exercice II.1).

Algorithme : Newton pour x_m

Données : $E, k_{\max}$

$x \leftarrow 5$; $m \leftarrow (4,9 - 4)e^{4,9}$; $k \leftarrow 0$

Répéter

$x \leftarrow x - ((x - 5)e^x + 5)/((x - 4)e^x)$; $k \leftarrow k + 1$

Jusqu'à $|(x - 5)e^x + 5|/m < E$ **ou** $k \geq k_{\max}$

Écrire $x_m = x$ et $\lambda_m T = hc/(k x_m)$

Vérification numérique. Itérés : $x_1 = 4,966310265$, $x_2 = 4,965115686$, $x_3 = 4,965114231746$, $x_4 = 4,965114231744$ (convergence quadratique : le nombre de chiffres exacts double environ à chaque pas). Cette valeur est celle de la suite A094090 de l'OEIS (4,96511423174427...) [OEIS]. Avec les valeurs exactes des constantes ($h = 6,62607015 \times 10^{-34}$ J s, $c = 299\,792\,458$ m/s, $k = 1,380649 \times 10^{-23}$ J/K), on obtient la loi de Wien $\lambda_m T = hc/(k x_m) = 2,8978 \times 10^{-3}$ m K.

Références. [BF, alg. 2.3, p. 67] ; [OEIS].

Exercice 2 : volume d'un gaz de Van der Waals

Pourquoi ? Pour T et P donnés, V est racine de $g(V) = (P + a/V^2)(V - b) - RMT$; on ne sait pas l'isoler, il faut une méthode numérique : Newton. En multipliant par $V^2 > 0$, on obtient une équation polynomiale de degré 3, sans division par V :

$$f(V) = PV^3 - (Pb + K)V^2 + aV - ab = 0, \quad K = RMT.$$

Elle se dérive facilement, $f'(V) = 3PV^2 - 2(Pb + K)V + a$, et s'évalue par Horner (chapitre II). Le point de départ naturel est le volume du gaz parfait, $V_0 = K/P$ (on retrouve Van der Waals quand $a = b = 0$). Seules les racines $V > b$ ont un sens physique. Sous la température critique, le polynôme peut avoir trois racines réelles (liquide et gaz) ; partir de V_0 oriente vers la racine « gaz ».

Comment ? (étapes)

Étape 1. Lire a, b, R, M puis, pour chaque couple (T, P) : $K = RMT$, $V = K/P$.

Étape 2. Itérer $V \leftarrow V - f(V)/f'(V)$ jusqu'à $|\Delta V| < \varepsilon |V|$ (test relatif, V pouvant être très petit ou très grand selon les unités).

Étape 3. Vérifier $V > b$ et afficher.

Algorithme : Van der Waals

Données : $a, b, R, M, \varepsilon, k_{\max}$

Tant que il reste des couples (T, P) à traiter **faire**

Lire T, P

$K \leftarrow RMT$; $V \leftarrow K/P$; $k \leftarrow 0$

Répéter

$f \leftarrow ((PV - (Pb + K))V + a)V - ab$

{Horner}

$f' \leftarrow (3PV - 2(Pb + K))V + a$

$\Delta \leftarrow f/f'$; $V \leftarrow V - \Delta$; $k \leftarrow k + 1$

Jusqu'à $|\Delta| < \varepsilon |V|$ **ou** $k \geq k_{\max}$

Si $V \leq b$ **ou** $k \geq k_{\max}$ **alors**

Écrire « échec »

Sinon

Écrire T, P, V

Fin si

Fin tant que

Vérification numérique. Pour le dioxyde de carbone, avec $a = 3,592 \text{ L}^2 \text{ atm/mol}^2$, $b = 0,04267 \text{ L/mol}$ [VdW] et $R = 0,08206 \text{ L atm/(mol K)}$, pour une mole ($K = RT$) à $T = 300 \text{ K}$: à $P = 10 \text{ atm}$, $V = 2,354688 \text{ L}$ en 5 itérations (gaz parfait : $2,4618 \text{ L}$) ; à $P = 50 \text{ atm}$, $V = 0,358484 \text{ L}$ en 7 itérations (gaz parfait : $0,4924 \text{ L}$). Dans les deux cas les deux autres racines du polynôme sont complexes, la racine trouvée est donc la seule racine réelle.

Références. [BF, alg. 2.3, p. 67 ; Horner, alg. 2.7, p. 94] ; constantes : [VdW].

Exercice 3 : facteur de compressibilité (De Santis)

Remarque sur l'énoncé. L'expression $z = (1 + y + y^2 - y^3)/(1 - y)^3$, avec $y = b/(4v)$, est celle de Carnahan et Starling pour un gaz de sphères dures, reprise dans l'équation d'état de Carnahan-Starling-De Santis (1976) [CSD]. Dans le document scanné, le dénominateur est imprimé $(1 - y^3)$; c'est $(1 - y)^3$ qui est correct (voir la transcription du cours). On connaît z et v , on cherche b .

1- Principe de la méthode de Newton.

Pourquoi ? On remplace la courbe de f par sa tangente au point x_n ; le point où la tangente coupe l'axe donne $x_{n+1} = x_n - f(x_n)/f'(x_n)$ (chapitre II). La convergence est quadratique près d'une racine simple.

2- Algorithme.

Pourquoi ? On cherche d'abord y , puis $b = 4vy$. On multiplie l'équation par $(1 - y)^3$ pour supprimer le pôle $y = 1$:

$$f(y) = z(1 - y)^3 - (1 + y + y^2 - y^3) = 0, \quad f'(y) = -3z(1 - y)^2 - (1 + 2y - 3y^2).$$

Pour le départ : quand y est petit, $(1 - y)^{-3} = 1 + 3y + 6y^2 + \dots$, d'où $z = 1 + 4y + 10y^2 + \dots$; on peut donc prendre $y_0 = (z - 1)/4$. Physiquement $0 < y < 1$.

Comment ? (étapes)

Étape 1. Lire $z, v, \varepsilon, k_{\max}$; $y \leftarrow (z - 1)/4$ (ou une valeur donnée).

Étape 2. Itérer $y \leftarrow y - f(y)/f'(y)$ jusqu'à $|\Delta y| < \varepsilon$.

Étape 3. $b = 4vy$.

Algorithme : De Santis

Lire $z, v, y_0, \varepsilon, k_{\max}$

$y \leftarrow y_0$

Pour k allant de 1 à $k_{\max}$ **faire**

$f \leftarrow z(1 - y)^3 - (1 + y + y^2 - y^3)$

$f' \leftarrow -3z(1 - y)^2 - (1 + 2y - 3y^2)$

$\Delta \leftarrow f/f'; y \leftarrow y - \Delta$

Si $|\Delta| < \varepsilon$ **alors**

Écrire $y, b = 4vy$

Arrêt

Fin si

Fin pour

Écrire « pas de convergence »

3- Programme.

Programme Python (fichier programmes/desantis.py) :

```
"""Exercice 3 : facteur de compressibilité
    z = (1 + y + y^2 - y^3) / (1 - y)^3,    y = b / (4 v).
On cherche y (puis b) par Newton-Raphson sur
    f(y) = z (1 - y)^3 - (1 + y + y^2 - y^3) = 0."""

def desantis(z, v, y0, eps=1e-12, kmax=50):
    """Retourne (y, b, nombre d'itérations)."""
    y = y0
    for k in range(1, kmax + 1):
        f = z * (1 - y) ** 3 - (1 + y + y ** 2 - y ** 3)
        fp = -3 * z * (1 - y) ** 2 - (1 + 2 * y - 3 * y ** 2)
        dy = f / fp
        y = y - dy
        if abs(dy) < eps:
            return y, 4 * v * y, k
    raise RuntimeError(f"pas de convergence en {kmax} itérations")

if __name__ == "__main__":
    z, v = 3.9737609, 1.0                # valeur de z pour y = 0,3
    for y0 in (0.5, (z - 1) / 4):
        y, b, k = desantis(z, v, y0)
        print(f"y0 = {y0:.4f} : y = {y:.10f}    b = 4 v y = {b:.10f}"
              f"    ({k} itérations)")
```

Vérification numérique. Pour $y = 0,3$, la formule donne $z = 1,363/0,343 = 3,9737609$. Avec cette valeur de z , $v = 1$ et $y_0 = 0,5$, le programme retrouve en 6 itérations $y = 0,2999999984$ et $b = 1,1999999937$ (la valeur de z n'ayant que 8 chiffres, on ne peut pas espérer mieux) ; avec le départ $y_0 = (z - 1)/4 = 0,7434$, la convergence vers 0,3 demande 8 itérations.

Références. [BF, alg. 2.3, p. 67] ; [CSD].

Exercice 4 : $y' = -ay - by^4 + c$ par prédicteur-correcteur

1)- Schéma itératif.

Pourquoi ? On applique l'exercice V.4 avec $f(x, y) = -ay - by^4 + c$ (qui ne dépend pas de x). Le prédicteur AB2 est explicite mais il lui faut deux valeurs ; le démarrage se fait par RK2.

Comment ? (étapes)

Étape 1. Démarrage : $y_1 = y_0 + \frac{h}{2}[f(y_0) + f(y_0 + hf(y_0))]$.

Étape 2. Pour $k \geq 1$, prédiction $p_{k+1} = y_k + \frac{h}{2}[3f(y_k) - f(y_{k-1})]$.

Étape 3. Correction $y_{k+1} = y_k + \frac{h}{2}[f(y_k) + f(p_{k+1})]$, avec $f(y) = -ay - by^4 + c$.

2)- Algorithme. C'est celui de l'exercice V.4, avec cette fonction f ; on garde $f(y_{k-1})$ dans une variable pour ne calculer que deux valeurs de f par pas.

3)- Programme.

Programme Python (fichier programmes/ex4_pc.py) :

```

"""Exercice 4 :  $y' = -a y - b y^4 + c$ ,  $y(0) = y_0$ .
Prédicteur : Adams-Bashforth d'ordre 2 ; correcteur : RK2 (trapèzes)."""

def resoudre(a, b, c, y0, xmax, n, affichage=10):
    f = lambda y: -a * y - b * y ** 4 + c          # f ne dépend pas de x
    h = xmax / n
    y = y0
    print(f"{0.0:8.4f} {y:16.10f}")
    # démarrage : y_1 par RK2
    fprec = f(y)
    y = y + h / 2 * (fprec + f(y + h * fprec))
    for k in range(1, n):
        fk = f(y)
        p = y + h / 2 * (3 * fk - fprec)           # prédicteur AB2
        y = y + h / 2 * (fk + f(p))                # correcteur RK2
        fprec = fk
        if (k + 1) % affichage == 0:
            print(f"{(k + 1) * h:8.4f} {y:16.10f}")
    return y

if __name__ == "__main__":
    y2 = resoudre(a=1.0, b=0.5, c=2.0, y0=0.0, xmax=2.0, n=40)
    print(f"y(2) = {y2:.10f}")

```

Vérification numérique. Pour $a = 1$, $b = 0,5$, $c = 2$, $y_0 = 0$, $x = 2$ et $n = 40$, le programme donne $y(2) = 1,1425354104$; une intégration de référence très précise (SciPy) donne $1,1425030979$: erreur 3×10^{-5} . La solution tend vers l'état d'équilibre y^* , racine de $-y - 0,5y^4 + 2 = 0$, soit $y^* = 1,1439$.

Références. [BF, p. 307 (AB2) ; p. 310–311 (prédicteur-correcteur)].

Exercice 5 : $y'' + \frac{a}{x}y' + be^{cy} = 0$ par la méthode de tir

Pourquoi ? La condition $y'(0) = 0$ est en $x = 0$ et la condition $y(1) = 1$ en $x = 1$: il manque la valeur $y(0)$ pour pouvoir partir de $x = 0$. On la remplace par le paramètre de tir $\gamma = y(0)$ et on cherche γ tel que $F(\gamma) = y(1, \gamma) - 1 = 0$. Newton demande $F'(\gamma) = \partial y(1, \gamma) / \partial \gamma$: on l'obtient en dérivant l'équation par rapport à γ , ce qui donne une deuxième équation différentielle (méthode de Burden et Faires, alg. 11.2).

1)- Schéma itératif de Newton pour γ .

Étape 1. Posons $z(x) = \partial y(x, \gamma) / \partial \gamma$. En dérivant l'équation et les conditions initiales $y(0) = \gamma$, $y'(0) = 0$ par rapport à γ :

$$z'' + \frac{a}{x} z' + bc e^{c\gamma} z = 0, \quad z(0) = 1, \quad z'(0) = 0.$$

Étape 2. Newton : $\gamma_{n+1} = \gamma_n - \frac{y(1, \gamma_n) - 1}{z(1, \gamma_n)}$.

Étape 3. À chaque itération, on intègre ensemble les équations en y et en z (même γ_n), puis on corrige γ .

Le point singulier $x = 0$. Le terme $\frac{a}{x} y'$ n'est pas défini en $x = 0$. Mais $y'(0) = 0$, donc $y'(x)/x \rightarrow y''(0)$ quand $x \rightarrow 0$. L'équation en $x = 0$ devient $y''(0) + a y''(0) + bc e^{c\gamma} = 0$, d'où

$$y''(0) = -\frac{bc e^{c\gamma}}{1+a}, \quad \text{et de même} \quad z''(0) = -\frac{bc e^{c\gamma}}{1+a}.$$

Ce sont les valeurs à utiliser au premier pas.

2)- Méthode de Runge-Kutta d'ordre 4 pour un système. On pose $Y = (Y_1, Y_2, Y_3, Y_4) = (y, y', z, z')$; le système s'écrit $Y' = F(x, Y)$ avec

$$F(x, Y) = \left(Y_2, -\frac{a}{x} Y_2 - bc e^{cY_1}, Y_4, -\frac{a}{x} Y_4 - bc e^{cY_1} Y_3 \right)$$

(et les limites ci-dessus pour $x = 0$). RK4 s'applique composante par composante, avec les mêmes formules que pour une équation scalaire :

$$\begin{aligned} K_1 &= hF(x_i, Y_i), & K_2 &= hF\left(x_i + \frac{h}{2}, Y_i + \frac{1}{2}K_1\right), \\ K_3 &= hF\left(x_i + \frac{h}{2}, Y_i + \frac{1}{2}K_2\right), & K_4 &= hF(x_i + h, Y_i + K_3), \\ Y_{i+1} &= Y_i + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4). \end{aligned}$$

Il est essentiel de calculer *toutes* les composantes de K_1 avant de passer à K_2 , etc.

3)- Algorithme complet.

Algorithme : tir avec Newton

Données : $a, b, c, N, \gamma_0, \varepsilon, k_{\max}$

$h \leftarrow 1/N$; $\gamma \leftarrow \gamma_0$

Pour k allant de 1 à $k_{\max}$ **faire**

$x \leftarrow 0$; $Y \leftarrow (\gamma, 0, 1, 0)$

{conditions initiales}

Pour i allant de 1 à N **faire**

$K_1 \leftarrow hF(x, Y)$; $K_2 \leftarrow hF(x + h/2, Y + K_1/2)$

$K_3 \leftarrow hF(x + h/2, Y + K_2/2)$; $K_4 \leftarrow hF(x + h, Y + K_3)$

$Y \leftarrow Y + (K_1 + 2K_2 + 2K_3 + K_4)/6$; $x \leftarrow x + h$

Fin pour

$\Delta \leftarrow (Y_1 - 1)/Y_3$

{ $Y_1 = y(1), Y_3 = z(1)$ }

$\gamma \leftarrow \gamma - \Delta$

Si $|\Delta| < \varepsilon$ **alors**

Écrire γ (et refaire un tir pour afficher $y(x_i)$)

Arrêt

Fin si

Fin pour

Écrire « pas de convergence »

où $F(0, Y)$ utilise les limites $y''(0)$ et $z''(0)$.

Vérification numérique.

- Cas test exact $c = 0$: l'équation devient linéaire, $y = \gamma - \frac{b}{2(1+a)}x^2$, d'où $\gamma = 1 + \frac{b}{2(1+a)}$.
Pour $a = b = 1$: $\gamma = 1,25$. L'algorithme ($N = 200$) trouve 1,2500000000 en 2 itérations.
- $a = 1, b = 1, c = 0,5$: $\gamma = 1,4961487192$ (5 itérations depuis $\gamma_0 = 1$), avec $y(1) = 1$ à 10^{-12} près. Ce problème a une seconde solution, $\gamma = 9,5942$, que Newton atteint si l'on part plus haut : le point de départ choisit la solution.
- $a = 2, b = 0,5, c = 1$: $\gamma = 1,2739793373$.
- Pour $a = b = c = 1$, il n'y a *pas* de solution : le calcul de $y(1, \gamma)$ pour $\gamma \in [-5, 5]$ montre que son maximum vaut 0,693 (vers $\gamma \approx 2,1$), toujours inférieur à 1. Une méthode de tir qui ne converge pas peut donc signaler un problème sans solution, et pas seulement un mauvais γ_0 .

Références. [BF, § 11.2, p. 693–698, alg. 11.2, p. 696 ; RK4 pour les systèmes, alg. 5.7, p. 333].

Exercice 6 : potentiel entre cathode et anode, $V'' = aV^{-1/2}$

1-a- Itérations RK2.

Pourquoi ? On intègre $y' = f(x, y)$ entre x_k et x_{k+1} par les trapèzes et on remplace l'inconnue y_{k+1} du second membre par la prévision d'Euler (exercice V.1) :

$$y_{k+1} = y_k + \frac{h}{2} \left[f(x_k, y_k) + f(x_k + h, y_k + hf(x_k, y_k)) \right].$$

1-b- Algorithme pour (1).

Pourquoi ? On pose $W = V'$ (c'est l'opposé du champ électrique) : (1) devient le système $V' = W, W' = aV^{-1/2}$, avec $V(0) = W(0) = 0$. RK2 s'applique au vecteur (V, W) . Mais en $x = 0, V = 0$ et le second membre $a/\sqrt{V}$ est infini : on ne peut pas démarrer en 0. On cherche alors une solution de la forme $V = Cx^p$: $V'' = Cp(p-1)x^{p-2}$ et $aV^{-1/2} = aC^{-1/2}x^{-p/2}$ imposent $p-2 = -p/2$, soit $p = 4/3$, puis $C \cdot \frac{4}{3} \cdot \frac{1}{3} = aC^{-1/2}$, soit $C = (9a/4)^{2/3}$. On retrouve la loi de Child-Langmuir $V = (9a/4)^{2/3}x^{4/3}$ [CL]. On démarre donc en $x = \varepsilon$ petit, avec $V(\varepsilon) = C\varepsilon^{4/3}$ et $W(\varepsilon) = \frac{4}{3}C\varepsilon^{1/3}$.

Algorithme : RK2 pour $V'' = aV^{-1/2}$

Données : a, l, ε, n

$C \leftarrow (9a/4)^{2/3}; x \leftarrow \varepsilon; V \leftarrow C\varepsilon^{4/3}; W \leftarrow \frac{4}{3}C\varepsilon^{1/3}; h \leftarrow (l - \varepsilon)/n$

Pour i allant de 1 à n faire

$k_{1V} \leftarrow hW; k_{1W} \leftarrow ha/\sqrt{V}$

$k_{2V} \leftarrow h(W + k_{1W}); k_{2W} \leftarrow ha/\sqrt{V + k_{1V}}$

$V \leftarrow V + (k_{1V} + k_{2V})/2; W \leftarrow W + (k_{1W} + k_{2W})/2; x \leftarrow x + h$

Écrire x, V, W

Fin pour

1-c- Programme. Le même fichier contient aussi la fonction `tir` de la question 2.

Programme Python (fichier `programmes/ex6_rk2.py`) :

```

"""Exercice 6 : V'' = a V^(-1/2), écrit comme le système
    V' = W, W' = a / sqrt(V),
    résolu par RK2 (trapèzes). Le second membre n'est pas défini en V = 0 :
    on part de x = eps.
    Question 1 : V(0) = V'(0) = 0 ; départ avec la solution exacte
    V = C x^(4/3), C = (9a/4)^(2/3).
    Question 2 : V(0) = 0, V(l) = V0 ; tir sur gamma = V'(0) (sécante)."""
import math

def rk2(a, x, v, w, xfin, n):
    """Intègre de x à xfin en n pas ; retourne V(xfin), W(xfin)."""
    h = (xfin - x) / n
    for i in range(n):
        k1v = h * w
        k1w = h * a / math.sqrt(v)
        k2v = h * (w + k1w)
        k2w = h * a / math.sqrt(v + k1v)
        v = v + (k1v + k2v) / 2
        w = w + (k1w + k2w) / 2
    return v, w

def question1(a, l, eps, n):
    c = (9 * a / 4) ** (2 / 3)
    v0 = c * eps ** (4 / 3) # V(eps) exact
    w0 = 4 / 3 * c * eps ** (1 / 3) # V'(eps) exact
    v, w = rk2(a, eps, v0, w0, l, n)
    return v, c * l ** (4 / 3)

def tir(a, l, V0, eps, n, g0, g1, tol=1e-10, kmax=50):
    """Cherche gamma = V'(0) > 0 tel que V(l) = V0."""
    def V_l(g):
        # départ approché : V ~ g x + (4a/3) x^(3/2) / sqrt(g)
        v = g * eps + 4 * a / 3 * eps ** 1.5 / math.sqrt(g)
        w = g + 2 * a * math.sqrt(eps / g)
        return rk2(a, eps, v, w, l, n)[0]
    r0 = V_l(g0) - V0
    for k in range(kmax):
        r1 = V_l(g1) - V0
        if abs(r1) < tol:
            return g1, k + 1
        g0, g1, r0 = g1, g1 - (g1 - g0) * r1 / (r1 - r0), r1
    raise RuntimeError("pas de convergence")

if __name__ == "__main__":
    for n in (100, 200, 400):
        v, exact = question1(a=1.0, l=1.0, eps=1e-2, n=n)
        print(f"n = {n:3d} : V(1) = {v:.8f} exact = {exact:.8f}"
              f" erreur = {v - exact:.2e}")
    v, exact = question1(a=1.0, l=1.0, eps=1e-3, n=400)
    print(f"eps = 1e-3, n = 400 : V(1) = {v:.8f}")
    # près de la singularité x = 0, eps et le pas doivent être petits

```

```

for eps, n in ((1e-2, 4000), (1e-3, 16000), (1e-4, 64000)):
    g, k = tir(a=1.0, l=1.0, V0=2.5, eps=eps, n=n, g0=0.1, g1=1.0)
    print(f"tir (eps = {eps:g}, n = {n}) : gamma = {g:.6f}"
          f" en {k} itérations")

```

Vérification numérique. $a = 1, l = 1, \varepsilon = 10^{-2}$; valeur exacte $V(1) = (9/4)^{2/3} = 1,71707136$. Résultats : $n = 100 : 1,72325821$; $n = 200 : 1,71871098$; $n = 400 : 1,71749474$. Les erreurs ($6,2 \times 10^{-3}$; $1,6 \times 10^{-3}$; $4,2 \times 10^{-4}$) sont divisées par 4 : ordre 2. Partir trop près de la singularité dégrade la précision : avec $\varepsilon = 10^{-3}$ et $n = 400$, on obtient 1,7256.

2-a- Principe de la méthode de tir.

Pourquoi ? Les conditions sont maintenant $V(0) = 0$ et $V(l) = V_0$: une condition à chaque bout. On choisit la pente inconnue $\gamma = V'(0)$ (c'est-à-dire le champ sur la cathode), on intègre le problème à valeurs initiales, et on ajuste γ pour que $V(l, \gamma) = V_0$ (exercice V.5).

2-b- Algorithme.

Étape 1. Démarrage hors de la singularité : pour $\gamma > 0$, près de 0, $V \approx \gamma x$, donc $V'' \approx a/\sqrt{\gamma x}$; en intégrant deux fois : $W(\varepsilon) \approx \gamma + 2a\sqrt{\varepsilon/\gamma}$ et $V(\varepsilon) \approx \gamma\varepsilon + \frac{4a}{3}\varepsilon^{3/2}/\sqrt{\gamma}$.

Étape 2. Deux tirs γ_0, γ_1 , puis la formule de la sécante du cours jusqu'à $|V(l, \gamma) - V_0| < \text{tolérance}$.

Étape 3. Remarque physique : pour $\gamma = 0$ on retrouve Child-Langmuir, $V(l) = Cl^{4/3}$. Le calcul montre que $V(l, \gamma)$ croît avec γ (1,7173; 1,7233; 1,8429; 2,1223; 2,8746 pour $\gamma = 0,01$; 0,1; 0,5; 1; 2, avec $a = l = 1$) : une solution avec $V \geq 0$ n'existe que si $V_0 \geq Cl^{4/3}$.

Algorithme : tir pour $V(0) = 0, V(l) = V_0$

Données : $a, l, V_0, \varepsilon, n, \gamma_0, \gamma_1$, tolérance, $k_{\max}$

$R_0 \leftarrow V(l, \gamma_0) - V_0$ {RK2 depuis $x = \varepsilon$, étape 1}

Pour k allant de 1 à $k_{\max}$ **faire**

$R_1 \leftarrow V(l, \gamma_1) - V_0$

Si $|R_1| < \text{tolérance}$ **alors**

Écrire γ_1

Arrêt

Fin si

$\gamma \leftarrow \gamma_1 - (\gamma_1 - \gamma_0)R_1/(R_1 - R_0)$

$\gamma_0 \leftarrow \gamma_1$; $R_0 \leftarrow R_1$; $\gamma_1 \leftarrow \gamma$

Fin pour

Vérification numérique. $a = l = 1, V_0 = 2,5, \gamma_0 = 0,1, \gamma_1 = 1$: la fonction `tir` de `ex6_rk2.py` donne $\gamma = 1,526741$ ($\varepsilon = 10^{-2}, n = 4000$), $1,529373$ ($\varepsilon = 10^{-3}, n = 16\,000$) et $1,529628$ ($\varepsilon = 10^{-4}, n = 64\,000$), en 6 ou 7 itérations; une intégration de référence très précise (SciPy) donne $\gamma = 1,52966$. Près de la singularité $x = 0$, il faut à la fois un départ ε petit (le démarrage n'est qu'approché) et un pas fin (la dérivée seconde y est très grande).

Références. Child-Langmuir : [CL]; [BF, RK pour les systèmes, alg. 5.7, p. 333; tir, § 11.2, p. 694].

Exercice 7 : équation du troisième ordre par la méthode de tir

Remarque sur l'énoncé. Une équation du troisième ordre demande trois conditions ; l'énoncé n'en donne que deux, $y(a) = y_1$ et $y(b) = y_2$. Le problème est donc incomplet. On suppose ici une troisième condition donnée en $x = a$, $y'(a) = y'_1$ (avec une condition en b , par exemple $y'(b)$, la démarche serait la même avec deux paramètres de tir).

1)- Problème aux valeurs initiales.

Pourquoi ? En $x = a$, on connaît y et y' ; il manque $y''(a)$, que l'on remplace par le paramètre γ . On écrit l'équation comme un système du premier ordre pour pouvoir utiliser les méthodes du chapitre V.

Étape 1. $Y_1 = y, Y_2 = y', Y_3 = y''$:

$$Y'_1 = Y_2, \quad Y'_2 = Y_3, \quad Y'_3 = f(x, Y_1, Y_2, Y_3), \quad Y_1(a) = y_1, \quad Y_2(a) = y'_1, \quad Y_3(a) = \gamma.$$

Étape 2. On cherche γ tel que $Y_1(b, \gamma) = y_2$ (sécante).

2-a)- Principe du prédicteur-correcteur (premier ordre). Voir l'exercice V.4 : on prédit y_{k+1} par une formule explicite (Adams-Bashforth d'ordre 2, $p = y_k + \frac{h}{2}(3f_k - f_{k-1})$), puis on le corrige par une formule implicite (trapèzes, $y_{k+1} = y_k + \frac{h}{2}(f_k + f(x_{k+1}, p))$), dans laquelle on utilise la valeur prédite à la place de l'inconnue.

2-b)- Algorithme pour le problème du troisième ordre.

Pourquoi ? Les formules sont les mêmes, appliquées au vecteur Y (toutes les composantes sont prédites, puis toutes sont corrigées). On les place dans la boucle de tir.

Algorithme : tir + prédicteur-correcteur pour $y''' = f(x, y, y', y'')$

Données : $f, a, b, y_1, y'_1, y_2, N, \gamma_0, \gamma_1, \varepsilon, k_{\max}$

$h \leftarrow (b - a)/N$

Procédure PC(γ) :

$x \leftarrow a; Y \leftarrow (y_1, y'_1, \gamma); F_p \leftarrow F(x, Y)$

$Y \leftarrow Y + \frac{h}{2}(F_p + F(x + h, Y + hF_p)); x \leftarrow x + h$

{démarrage RK2}

pour $k = 1$ à $N - 1$: $F_k \leftarrow F(x, Y); P \leftarrow Y + \frac{h}{2}(3F_k - F_p);$

$Y \leftarrow Y + \frac{h}{2}(F_k + F(x + h, P)); F_p \leftarrow F_k; x \leftarrow x + h$

retourner Y_1

{ $\approx y(b, \gamma)$ }

Programme principal :

$R_0 \leftarrow PC(\gamma_0) - y_2$

Pour k allant de 1 à $k_{\max}$ **faire**

$R_1 \leftarrow PC(\gamma_1) - y_2$

Si $|R_1| < \varepsilon$ **alors**

Écrire γ_1

Arrêt

Fin si

$\gamma \leftarrow \gamma_1 - (\gamma_1 - \gamma_0)R_1/(R_1 - R_0); \gamma_0 \leftarrow \gamma_1; R_0 \leftarrow R_1; \gamma_1 \leftarrow \gamma$

Fin pour

avec $F(x, Y) = (Y_2, Y_3, f(x, Y_1, Y_2, Y_3))$.

Vérification numérique. Test avec solution connue $y = \sin x + x^2$: elle vérifie $y''' = -\cos x = 2x - y'$, $y(0) = 0$, $y'(0) = 1$, $y(1) = \sin 1 + 1$, et $y''(0) = 2$. L'algorithme retrouve $\gamma = 2,00025923$ ($N = 20$), $2,00006339$ ($N = 40$), $2,00001560$ ($N = 80$) : erreur divisée par 4, ordre 2 du prédicteur-correcteur.

Références. [BF, p. 310–311 ; systèmes et équations d'ordre supérieur, § 5.9, p. 331–335].

Exercices de fin de document (8 à 13)

Exercice 8 : différences finies et méthode de Jacobi

1-a- Dérivée première centrée.

Pourquoi ? C'est le calcul de l'exercice IV.1 : on interpole y par le polynôme de degré 2 passant par $x - h$, x , $x + h$, et on le dérive en x .

Comment ? (étapes)

Étape 1. Les dérivées des polynômes de base en x valent $-1/(2h)$, 0 et $1/(2h)$ (calcul détaillé à l'exercice IV.1).

Étape 2. D'où $y'(x) \approx (y(x+h) - y(x-h))/(2h)$, avec l'erreur $-\frac{h^2}{6}y'''(\xi)$.

1-b- Dérivée seconde.

Pourquoi ? On applique la formule de a) deux fois, à la fonction y' , avec le pas $h/2$: $y''(x) \approx [y'(x+h/2) - y'(x-h/2)]/h$.

Comment ? (étapes)

Étape 1. Par a) avec le pas $h/2$: $y'(x+h/2) \approx (y(x+h) - y(x))/h$ et $y'(x-h/2) \approx (y(x) - y(x-h))/h$.

Étape 2. D'où

$$y''(x) \approx \frac{y(x+h) - 2y(x) + y(x-h)}{h^2}.$$

Étape 3. Erreur : en additionnant les développements de Taylor de $y(x \pm h)$ jusqu'à l'ordre 4, les termes impairs disparaissent et il reste $-\frac{h^2}{12}y^{(4)}(\xi)$. C'est pourquoi l'énoncé suppose y de classe C^4 .

2- Système algébrique.

Pourquoi ? On écrit l'équation aux nœuds $x_i = a + ih$, $i = 1, \dots, n-1$, $h = (b-a)/n$, en remplaçant y' et y'' par les formules centrées. Les inconnues sont $y_1, \dots, y_{n-1}$; $y_0 = c$ et $y_n = d$ sont connues.

Comment ? (étapes)

Étape 1. $\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} = p_i \frac{y_{i+1} - y_{i-1}}{2h} + q_i y_i + g_i$.

Étape 2. En multipliant par h^2 et en regroupant :

$$\left(1 + \frac{h}{2}p_i\right)y_{i-1} - (2 + h^2q_i)y_i + \left(1 - \frac{h}{2}p_i\right)y_{i+1} = h^2g_i, \quad i = 1, \dots, n-1.$$

Étape 3. Pour $i = 1$ et $i = n-1$, les termes connus $y_0 = c$ et $y_n = d$ passent au second membre. Le système est *tridiagonal*, d'erreur de troncature $O(h^2)$. Burden et Faires montrent qu'il a une solution unique si $q \geq 0$ et $h < 2/L$, avec $L = \max |p|$ [BF, th. 11.3, p. 702] ; la condition $h|p_i| < 2$ rend aussi $1 \pm \frac{h}{2}p_i$ positifs.

3-a- Principe de Jacobi. On isole y_i dans l'équation i et on calcule les nouvelles valeurs à partir des anciennes uniquement (exercice I.7) :

$$y_i^{(k+1)} = \frac{h^2 g_i - (1 + \frac{h}{2} p_i) y_{i-1}^{(k)} - (1 - \frac{h}{2} p_i) y_{i+1}^{(k)}}{-(2 + h^2 q_i)}.$$

Le coefficient diagonal $2 + h^2 q_i$ est au moins égal à la somme $|1 + \frac{h}{2} p_i| + |1 - \frac{h}{2} p_i| = 2$ des autres (si $q_i \geq 0$ et $h |p_i| \leq 2$) : la matrice est à diagonale dominante, ce qui favorise la convergence (lente, car la dominance n'est pas stricte quand $q_i = 0$).

3-b- Algorithme.

Algorithme : différences finies + Jacobi

Données : $a, b, c, d, n, p, q, g, \varepsilon, k_{\max}$

$h \leftarrow (b - a)/n$

Pour i allant de 1 à $n - 1$ **faire**

$x \leftarrow a + ih$; $B_i \leftarrow 1 + \frac{h}{2} p(x)$; $A_i \leftarrow -(2 + h^2 q(x))$

$C_i \leftarrow 1 - \frac{h}{2} p(x)$; $D_i \leftarrow h^2 g(x)$; $y_i \leftarrow 0$

Fin pour

$y_0 \leftarrow c$; $y_n \leftarrow d$; $k \leftarrow 0$

Répéter

$k \leftarrow k + 1$; $num \leftarrow 0$; $den \leftarrow 0$

Pour i allant de 1 à $n - 1$ **faire**

$z_i \leftarrow (D_i - B_i y_{i-1} - C_i y_{i+1}) / A_i$

$num \leftarrow num + |z_i - y_i|$; $den \leftarrow den + |z_i|$

Fin pour

Pour i allant de 1 à $n - 1$ **faire**

$y_i \leftarrow z_i$

Fin pour

Jusqu'à $num/den < \varepsilon$ **ou** $k \geq k_{\max}$

Écrire $k, (x_i, y_i)$

3-c- Programme.

Programme Python (fichier programmes/ex8_jacobi.py) :

```

"""Exercice 8 : y'' = p(x) y' + q(x) y + g(x), y(a) = c, y(b) = d.
Différences finies centrées (ordre 2), puis méthode itérative de Jacobi.
Exemple (Burden-Faires, exemple 1 de la section 11.3, p. 703) :
p = -2/x, q = 2/x^2, g = sin(ln x)/x^2, 1 <= x <= 2, y(1) = 1, y(2) = 2."""
import math

```

```

def differences_finies_jacobi(p, q, g, a, b, c, d, n, eps, kmax):
    h = (b - a) / n
    # équation i : B_i y_{i-1} + A_i y_i + C_i y_{i+1} = D_i
    A, B, C, D = [0.0] * n, [0.0] * n, [0.0] * n, [0.0] * n
    for i in range(1, n):
        x = a + i * h
        B[i] = 1 + h / 2 * p(x)
        A[i] = -(2 + h * h * q(x))
        C[i] = 1 - h / 2 * p(x)
        D[i] = h * h * g(x)
    y = [0.0] * (n + 1)          # valeurs initiales nulles à l'intérieur

```

```

y[0], y[n] = c, d
for k in range(1, kmax + 1):
    z = y[:] # Jacobi : nouvelles valeurs à part
    num = den = 0.0
    for i in range(1, n):
        z[i] = (D[i] - B[i] * y[i - 1] - C[i] * y[i + 1]) / A[i]
        num += abs(z[i] - y[i])
        den += abs(z[i])
    y = z
    if num / den < eps:
        return y, k
return y, kmax

if __name__ == "__main__":
    y, k = differences_finies_jacobi(
        lambda x: -2 / x, lambda x: 2 / x ** 2,
        lambda x: math.sin(math.log(x)) / x ** 2,
        a=1.0, b=2.0, c=1.0, d=2.0, n=10, eps=1e-12, kmax=10000)
    print(f"Itérations : {k}")
    for i, yi in enumerate(y):
        print(f"{1 + i / 10:6.2f} {yi:14.8f}")

```

Vérification numérique. Le programme a été testé sur l'exemple 1 de [BF, p. 703] : $y'' = -\frac{2}{x}y' + \frac{2}{x^2}y + \frac{\sin(\ln x)}{x^2}$, $1 \leq x \leq 2$, $y(1) = 1$, $y(2) = 2$, $n = 10$. Avec $\varepsilon = 10^{-12}$, Jacobi converge en 450 itérations et donne $y(1,1) = 1,09260052$, $y(1,5) = 1,48112026$, $y(1,9) = 1,89392110$, etc. : toutes les valeurs coïncident avec la table 11.3 de Burden et Faires (obtenue par résolution directe). Le nombre élevé d'itérations illustre la lenteur de Jacobi (Gauss-Seidel ou la résolution tridiagonale directe, exercice VI.4, seraient plus rapides).

Références. [BF, (4.9), p. 178 ; § 11.3, alg. 11.3 et th. 11.3, p. 700–703].

Exercice 9 : oscillateur de Duffing forcé et amorti

1)- Équation du mouvement et adimensionnement.

Pourquoi ? La force qui dérive du potentiel est $-du/dy = -ky - k_1y^3$; la viscosité linéaire donne $-\alpha\dot{y}$; la force extérieure est $F \cos \omega t$. Adimensionner réduit le nombre de paramètres et donne des nombres d'ordre 1, plus sûrs numériquement.

Comment ? (étapes)

Étape 1. Deuxième loi de Newton :

$$M\ddot{y} + \alpha\dot{y} + ky + k_1y^3 = F \cos \omega t.$$

Étape 2. Échelle de temps : la pulsation propre $\omega_0 = \sqrt{k/M}$, $\tau = \omega_0 t$. Échelle de longueur : le déplacement statique $y_s = F/k$, $Y = y/y_s$. Alors $\dot{y} = y_s\omega_0 Y'$ et $\ddot{y} = y_s\omega_0^2 Y'' = (F/M)Y''$ (le prime désigne d/d τ).

Étape 3. En reportant et en divisant par F :

$$Y'' + \mu Y' + Y + \beta Y^3 = \cos(\Omega\tau), \quad \mu = \frac{\alpha}{M\omega_0} = \frac{\alpha}{\sqrt{kM}}, \quad \beta = \frac{k_1 F^2}{k^3}, \quad \Omega = \frac{\omega}{\omega_0}.$$

Il ne reste que trois paramètres sans dimension. (D'autres choix d'échelle de longueur sont possibles, par exemple $\sqrt{k/k_1}$, qui donne $\beta = 1$ et une amplitude de forçage sans dimension.) Conditions initiales : $Y(0) = Y'(0) = 0$.

2)- Algorithme RK4.

Pourquoi ? RK4 est une méthode à un pas : il suffit d'écrire l'équation du second ordre comme un système du premier ordre et d'appliquer RK4 au vecteur (voir l'exercice 5).

Étape 1. $X_1 = Y, X_2 = Y' : X'_1 = X_2, X'_2 = \cos(\Omega\tau) - \mu X_2 - X_1 - \beta X_1^3$.

Étape 2. À chaque pas, calcul des quatre pentes vectorielles, puis combinaison 1, 2, 2, 1.

Algorithme : RK4 pour le Duffing adimensionné

Données : μ, β, Ω, h, N

$\tau \leftarrow 0; X_1 \leftarrow 0; X_2 \leftarrow 0$

Pour i allant de 1 à N **faire**

$k_1 \leftarrow hX_2; l_1 \leftarrow hG(\tau, X_1, X_2)$

$k_2 \leftarrow h(X_2 + l_1/2); l_2 \leftarrow hG(\tau + h/2, X_1 + k_1/2, X_2 + l_1/2)$

$k_3 \leftarrow h(X_2 + l_2/2); l_3 \leftarrow hG(\tau + h/2, X_1 + k_2/2, X_2 + l_2/2)$

$k_4 \leftarrow h(X_2 + l_3); l_4 \leftarrow hG(\tau + h, X_1 + k_3, X_2 + l_3)$

$X_1 \leftarrow X_1 + (k_1 + 2k_2 + 2k_3 + k_4)/6; X_2 \leftarrow X_2 + (l_1 + 2l_2 + 2l_3 + l_4)/6$

$\tau \leftarrow \tau + h$

Écrire τ, X_1, X_2

Fin pour

avec $G(\tau, X_1, X_2) = \cos(\Omega\tau) - \mu X_2 - X_1 - \beta X_1^3$.

3)- Moindres carrés pour les a_n et b_n .

Pourquoi ? On dispose de beaucoup plus de valeurs y_i que d'inconnues ($2N$) : on ne peut pas interpoler, on minimise

$$R = \sum_{i=1}^m \left[y_i - \sum_{n=1}^N (a_n \cos n\omega t_i + b_n \sin n\omega t_i) \right]^2.$$

Le modèle est linéaire en les a_n, b_n : c'est le cadre du chapitre III avec les $2N$ fonctions $g = \cos n\omega t$ et $\sin n\omega t$.

Comment ? (étapes)

Étape 1. Équations normales : $\partial R / \partial a_n = 0$ et $\partial R / \partial b_n = 0$ donnent, pour $n = 1, \dots, N$,

$$\sum_{p=1}^N \left[a_p \sum_i \cos p\omega t_i \cos n\omega t_i + b_p \sum_i \sin p\omega t_i \cos n\omega t_i \right] = \sum_i y_i \cos n\omega t_i,$$

et la même équation avec $\sin n\omega t_i$ à la place de $\cos n\omega t_i$ en facteur. C'est un système $2N \times 2N$, symétrique ; on calcule ses coefficients avec l'algorithme de l'exercice III.2 et on le résout par Gauss.

Étape 2. Cas particulier très utile : si les t_i sont équidistants et couvrent exactement une période $T = 2\pi/\omega$ ($t_i = t_0 + iT/m, i = 0, \dots, m-1$, avec $m > 2N$), les sommes croisées sont nulles (orthogonalité discrète) et $\sum_i \cos^2 = \sum_i \sin^2 = m/2$. Le système devient diagonal :

$$a_n = \frac{2}{m} \sum_i y_i \cos n\omega t_i, \quad b_n = \frac{2}{m} \sum_i y_i \sin n\omega t_i.$$

Étape 3. Précautions. Le modèle suppose un régime périodique de période $2\pi/\omega$ et sans terme constant : il faut écarter le régime transitoire (qui dépend des conditions initiales) avant de prendre les y_i . En variables sans dimension, la pulsation à utiliser est Ω et le temps τ .

Algorithme : coefficients de Fourier par moindres carrés

Données : m valeurs (t_i, y_i) en régime permanent, ω , N

Pour i allant de 1 à m **faire**

Pour n allant de 1 à N **faire**

$G_{i,n} \leftarrow \cos n\omega t_i$; $G_{i,N+n} \leftarrow \sin n\omega t_i$

Fin pour

Fin pour

former $A_{pq} = \sum_i G_{ip} G_{iq}$ et $B_p = \sum_i G_{ip} y_i$ ($p, q = 1, \dots, 2N$)

{exercice III.2}

résoudre $Ac = B$ par Gauss

$\{c = (a_1, \dots, a_N, b_1, \dots, b_N)\}$

(si échantillonnage uniforme sur une période : $a_n \leftarrow \frac{2}{m} \sum_i y_i \cos n\omega t_i$, $b_n \leftarrow \frac{2}{m} \sum_i y_i \sin n\omega t_i$)

Écrire a_n, b_n

Vérification numérique. Avec $\mu = 0,2$, $\beta = 0,1$, $\Omega = 1,2$, RK4 avec 64 pas par période, 100 périodes écartées puis $m = 64$ points sur une période, $N = 5$: $a_1 = 2,07476$, $b_1 = 2,12381$, $a_3 = -0,04201$, $b_3 = 0,04485$, $a_5 = -0,00091$, $b_5 = -0,00083$, et $a_2 = b_2 = a_4 = b_4 = 0$ (le terme Y^3 étant impair, seules les harmoniques impaires apparaissent). Les équations normales et les formules d'orthogonalité donnent les mêmes valeurs (écart 3×10^{-12}), et le reste vaut $R = 2 \times 10^{-8}$.

Références. [BF, RK4 pour les systèmes, alg. 5.7, p. 333 ; moindres carrés, § 8.1 ; approximation trigonométrique discrète, § 8.5, p. 545–550].

Exercice 10 : méthode de Gauss

1- Principe. On élimine les inconnues une à une : à l'étape k , on choisit l'équation pivotale (plus grand $|a_{ik}|$, $i \geq k$), on soustrait de chaque ligne $l > k$ le multiple a_{lk}/a_{kk} de la ligne k pour annuler le coefficient de x_k . Après $n - 1$ étapes, le système est triangulaire ; on le résout par remontée. *Pourquoi le pivot* : éviter la division par zéro et l'amplification des erreurs d'arrondi (exercice I.2).

2- Algorithme de triangularisation. C'est l'algorithme de l'exercice I.3 (boucle sur k avec recherche du pivot, échange, puis élimination).

3- Résolution du système triangulaire. C'est l'algorithme de remontée de l'exercice I.1.

4- Programme. Le programme contient les deux fonctions `triangularise` (questions 1 et 2) et `remontee` (question 3), réunies dans `gauss`.

Programme Python (fichier `programmes/gauss.py`) :

```

"""Exercices 10 et 13 (et chapitre I) : méthode de Gauss avec pivot
partiel (triangularisation), puis résolution du système triangulaire
(remontée). La matrice a est une liste de lignes, modifiée sur place."""
import random

def triangularise(a, b):
    n = len(b)

```

```

for k in range(n - 1):
    # pivot maximal : ligne ip telle que |a[ip][k]| soit maximal
    ip = k
    for i in range(k + 1, n):
        if abs(a[i][k]) > abs(a[ip][k]):
            ip = i
    if a[ip][k] == 0:
        raise ValueError("matrice singulière")
    if ip != k:
        # échange des lignes k et ip
        a[k], a[ip] = a[ip], a[k]
        b[k], b[ip] = b[ip], b[k]
    for i in range(k + 1, n):
        # élimination de x_k
        m = a[i][k] / a[k][k]
        for j in range(k + 1, n):
            a[i][j] = a[i][j] - m * a[k][j]
        a[i][k] = 0.0
        b[i] = b[i] - m * b[k]

def remontee(a, b):
    n = len(b)
    x = [0.0] * n
    x[n - 1] = b[n - 1] / a[n - 1][n - 1]
    for k in range(n - 2, -1, -1):
        s = b[k]
        for l in range(k + 1, n):
            s = s - a[k][l] * x[l]
        x[k] = s / a[k][k]
    return x

def gauss(a, b):
    a = [ligne[:] for ligne in a]
    b = b[:]
    triangularise(a, b)
    return remontee(a, b)

if __name__ == "__main__":
    # système de l'exercice I.3 (a11 = 0 : échange de lignes obligatoire)
    x = gauss([[0, 2, 1], [1, 1, 1], [2, 1, 3]], [5, 6, 13])
    for i, xi in enumerate(x, start=1):
        print(f"x{i} = {xi:16.10f}")
    # système 6 x 6 aléatoire de solution connue, avec a11 = 0
    random.seed(1)
    n = 6
    a = [[random.uniform(-1, 1) for j in range(n)] for i in range(n)]
    a[0][0] = 0.0
    xe = [random.uniform(-1, 1) for i in range(n)]
    b = [sum(a[i][j] * xe[j] for j in range(n)) for i in range(n)]
    x = gauss(a, b)
    erreur = max(abs(x[i] - xe[i]) for i in range(n))
    print(f"test 6 x 6 : erreur maximale = {erreur:.2e}")

```

Vérification numérique. Sur le système $2x_2 + x_3 = 5$, $x_1 + x_2 + x_3 = 6$, $2x_1 + x_2 + 3x_3 = 13$ (où $a_{11} = 0$), le programme donne $x = (7/3, 4/3, 7/3)$ (détail du calcul à l'exercice I.3); sur un système 6×6 aléatoire avec $a_{11} = 0$, l'erreur est de $2,4 \times 10^{-16}$.

Références. [BF, alg. 6.1, p. 368; alg. 6.2, p. 378].

Exercice 11 : le pendule simple

1-a- Algorithme RK2.

Pourquoi ? On écrit $Y'' + \omega_0^2 \sin Y = 0$ comme un système : $Y' = Z$, $Z' = -\omega_0^2 \sin Y$, avec $Y(0) = Y_0$, $Z(0) = Z_0$ (pour un pendule lâché sans vitesse, $Z_0 = 0$). RK2 (exercice V.1) s'applique au vecteur (Y, Z) : on calcule les deux pentes de Euler pour les deux composantes, puis on fait la moyenne.

Algorithme : RK2 pour le pendule

Données : ω_0 , Y_0 , Z_0 , $t_{\max}$, n

$h \leftarrow t_{\max}/n$; $t \leftarrow 0$; $Y \leftarrow Y_0$; $Z \leftarrow Z_0$

Pour i allant de 1 à n **faire**

$k_{1Y} \leftarrow hZ$; $k_{1Z} \leftarrow -h\omega_0^2 \sin Y$

$k_{2Y} \leftarrow h(Z + k_{1Z})$; $k_{2Z} \leftarrow -h\omega_0^2 \sin(Y + k_{1Y})$

$Y \leftarrow Y + (k_{1Y} + k_{2Y})/2$; $Z \leftarrow Z + (k_{1Z} + k_{2Z})/2$; $t \leftarrow t + h$

Écrire t , Y , Z

Fin pour

1-b- Programme.

Programme Python (fichier programmes/ex11_pendule.py) :

```
"""Exercice 11, question 1 :  $Y'' + \omega_0^2 \sin Y = 0$ ,  $Y(0) = Y_0$ ,  $Y'(0) = Z_0$ .
Système  $Y' = Z$ ,  $Z' = -\omega_0^2 \sin Y$ , résolu par RK2 (trapèzes)."""
import math
```

```
def pendule_rk2(w0, y0, z0, tmax, n, npas):
    """Affiche (t, Y, Z) tous les npas pas."""
    h = tmax / n
    y, z = y0, z0
    print(f"{0.0:16.8f}{y:16.8f}{z:16.8f}")
    for i in range(1, n + 1):
        k1y = h * z
        k1z = -h * w0 ** 2 * math.sin(y)
        k2y = h * (z + k1z)
        k2z = -h * w0 ** 2 * math.sin(y + k1y)
        y = y + (k1y + k2y) / 2
        z = z + (k1z + k2z) / 2
        if i % npas == 0:
            print(f"{i * h:16.8f}{y:16.8f}{z:16.8f}")
    return y, z
```

```
if __name__ == "__main__":
    pendule_rk2(w0=1.0, y0=2 * math.pi / 9, z0=0.0,
                tmax=4.0, n=4000, npas=1000)
```

Vérification numérique. $\omega_0 = 1$, $Y_0 = 2\pi/9$, $Z_0 = 0$, $h = 10^{-3}$, comparaison avec une intégration de référence (SciPy, tolérance 10^{-12}) :

t	1	2	3	4
RK2	0,39768602	-0,25372859	-0,67967973	-0,51945770
référence	0,39768609	-0,25372840	-0,67967966	-0,51945795

Les écarts sont inférieurs à 3×10^{-7} .

2-a- Formule composite des trapèzes et algorithme pour (2).

Pourquoi ? On découpe $[0, \pi/2]$ en n intervalles de longueur $h = \pi/(2n)$; sur chacun, l'aire sous la courbe est remplacée par celle du trapèze : $\frac{h}{2}(f(x_{k-1}) + f(x_k))$. En sommant, chaque nœud intérieur est compté deux fois :

$$\int_0^{\pi/2} f \approx h \left[\frac{1}{2}f(0) + \sum_{k=1}^{n-1} f(kh) + \frac{1}{2}f(\pi/2) \right], \quad f(x) = (1 - A \sin^2 x)^{-1/2},$$

d'erreur $-\frac{h^2}{12} \cdot \frac{\pi}{2} f''(\eta)$ (chapitre IV). Puis $T/T_0 = \frac{2}{\pi} \times$ intégrale. L'intégrande est défini et régulier car $A = \sin^2(Y_0/2) < 1$ pour $|Y_0| < \pi$.

Algorithme : période par les trapèzes

Données : Y_0, n

$A \leftarrow \sin^2(Y_0/2)$; $h \leftarrow \pi/(2n)$

$S \leftarrow (f(0) + f(\pi/2))/2$

Pour k allant de 1 à $n-1$ **faire**

$S \leftarrow S + f(kh)$

Fin pour

Écrire $T/T_0 = \frac{2}{\pi} h S$

2-b- Programme. Le même fichier traite aussi la question 3, avec la fonction `droite_moindres_carres`.

Programme Python (fichier `programmes/ex11_trapeze.py`) :

```

"""Exercice 11, questions 2 et 3.
T(Y0)/T0 = (2/pi) * intégrale de 0 à pi/2 de dx / sqrt(1 - A sin^2 x),
avec A = sin^2(Y0/2), par la formule composite des trapèzes (sans
tableaux), puis moindres carrés T/T0 = a + b Y0^2."""
import math

def trapezes(f, a, b, n):
    h = (b - a) / n
    s = (f(a) + f(b)) / 2
    for k in range(1, n):
        s = s + f(a + k * h)
    return h * s

def periode_relative(y0, n):
    A = math.sin(y0 / 2) ** 2
    f = lambda x: 1 / math.sqrt(1 - A * math.sin(x) ** 2)
    return 2 / math.pi * trapezes(f, 0.0, math.pi / 2, n)

```

```

def droite_moindres_carres(X, Y):
    """(a, b) tels que Y = a + b X : équations normales, Cramer."""
    m = len(X)
    s1, s2 = sum(X), sum(x * x for x in X)
    sy, sxy = sum(Y), sum(x * y for x, y in zip(X, Y))
    det = m * s2 - s1 * s1
    return (s2 * sy - s1 * sxy) / det, (m * sxy - s1 * sy) / det

if __name__ == "__main__":
    Y0 = [math.pi / 18, math.pi / 9, math.pi / 6, 2 * math.pi / 9]
    for y0 in Y0:
        print(f"Y0 = {y0:.4f} : T/T0 = {periode_relative(y0, 4):.8f}"
              f" (n = 4) {periode_relative(y0, 8):.8f} (n = 8)")
    X = [y * y for y in Y0]
    a, b = droite_moindres_carres(X, [1.002, 1.008, 1.020, 1.030])
    print(f"tableau de l'énoncé : a = {a:.6f}    b = {b:.6f}")
    exactes = [periode_relative(y, 8) for y in Y0]
    a, b = droite_moindres_carres(X, exactes)
    print(f"valeurs exactes      : a = {a:.6f}    b = {b:.6f}")

```

Vérification numérique. Avec seulement $n = 4$: $T/T_0 = 1,00190719$; $1,00766903$; $1,01740880$; $1,03134052$ pour $Y_0 = \pi/18, \pi/9, \pi/6, 2\pi/9$, et $n = 8$ donne les mêmes huit chiffres. Cette précision, bien meilleure que ce qu'annonce l'erreur en h^2 , s'explique : l'intégrande est régulier, pair et de période π , et les trapèzes sur une période d'une fonction périodique régulière convergent exponentiellement vite [TW]. Les valeurs coïncident avec la formule exacte $T = \frac{2T_0}{\pi} K(\sin(Y_0/2))$ (K : intégrale elliptique complète de première espèce) [LA].

3- Moindres carrés $T/T_0 = a + bY_0^2$.

Pourquoi ? Le modèle est linéaire en a et b avec la variable $X = Y_0^2$: c'est la droite des moindres carrés (chapitre III) en X .

Comment ? (étapes)

Étape 1. $X_i = Y_{0,i}^2 = 0,030462$; $0,121847$; $0,274156$; $0,487388$. Sommes ($m = 4$) : $\sum X = 0,913852$, $\sum X^2 = 0,328483$, $\sum T/T_0 = 4,06$, $\sum X T/T_0 = 0,934993$.

Étape 2. Système : $4a + 0,913852 b = 4,06$; $0,913852 a + 0,328483 b = 0,934993$. Déterminant $0,478806$.

Étape 3. $a = (0,328483 \times 4,06 - 0,913852 \times 0,934993)/0,478806 = 1,000814$; $b = (4 \times 0,934993 - 0,913852 \times 4,06)/0,478806 = 0,062093$.

Remarque sur l'énoncé. Les valeurs exactes de T/T_0 (question 2) sont $1,0019$; $1,0077$; $1,0174$; $1,0313$. Trois valeurs du tableau sont correctes à l'arrondi près, mais la troisième ($1,020$ pour $\pi/6$) ne l'est pas (la valeur exacte est $1,0174$) ; c'est aussi le point du plus grand écart à la droite ($0,0022$). Avec les valeurs exactes, les moindres carrés donnent $a = 0,999856$ et $b = 0,064452$. Pour comparaison, le développement théorique pour les petites amplitudes est $T/T_0 = 1 + Y_0^2/16 + \dots$, soit $a = 1$ et $b = 0,0625$ [LA].

Références. [BF, alg. 5.7, p. 333 ; th. 4.5, p. 205 ; § 8.1] ; [TW], [LA].

Exercice 12 : capacité calorifique et chaleur absorbée

1- Chaleur Q par Simpson.

Pourquoi ? À volume constant, $dQ = C_v dT$, donc $Q = \int_{0,1}^{0,5} C_v(T) dT$. On a trois points équidistants ($h = 0,2$) : c'est exactement le cas de la formule de Simpson simple.

Comment ? (étapes)

Étape 1. $Q \approx \frac{h}{3} [C_v(0,1) + 4C_v(0,3) + C_v(0,5)]$.

Étape 2. $Q \approx \frac{0,2}{3} [0,211 + 4 \times 0,694 + 1,361] \times 10^{-4} = \frac{0,2}{3} \times 4,348 \times 10^{-4}$.

Étape 3. $Q \approx 0,289867 \times 10^{-4} \text{ cal/mol}$.

2-a- Système vérifié par a et b . Ce sont les équations normales de la droite (complément du chapitre III) :

$$m a + \left(\sum T_i \right) b = \sum C_i, \quad \left(\sum T_i \right) a + \left(\sum T_i^2 \right) b = \sum T_i C_i.$$

2-b- Algorithmes.

Pourquoi ? Les quatre sommes se calculent en une seule boucle sur les mesures ; le système 2×2 se résout par la règle de Cramer (pas besoin de Gauss pour deux inconnues), à condition que le déterminant $m \sum T_i^2 - (\sum T_i)^2$ soit non nul.

Algorithme : droite des moindres carrés

$s_1 \leftarrow 0$; $s_2 \leftarrow 0$; $s_y \leftarrow 0$; $s_{ty} \leftarrow 0$

Pour i allant de 1 à m **faire**

$s_1 \leftarrow s_1 + T_i$; $s_2 \leftarrow s_2 + T_i^2$; $s_y \leftarrow s_y + C_i$; $s_{ty} \leftarrow s_{ty} + T_i C_i$

Fin pour

$D \leftarrow m s_2 - s_1^2$

$a \leftarrow (s_2 s_y - s_1 s_{ty}) / D$; $b \leftarrow (m s_{ty} - s_1 s_y) / D$

Écrire a, b

2-c- Programme. Il contient aussi la question 2-d.

Programme Python (fichier `programmes/ex12_mc.py`) :

```
"""Exercice 12 : moindres carrés  $C_v = a + b T$  (équations normales),  
puis chaleur  $Q$  par Simpson composite sur les points expérimentaux  
(abscisses équidistantes, nombre de points impair)."""
```

```
def moindres_carres(T, C):  
    m = len(T)  
    s1 = s2 = sy = sty = 0.0  
    for t, c in zip(T, C):          # 1) coefficients du système normal  
        s1 += t  
        s2 += t * t  
        sy += c  
        sty += t * c  
    print(f"Système : {m:8.4f} {s1:10.4f} | {sy:10.4f}")  
    print(f"          {s1:8.4f} {s2:10.4f} | {sty:10.4f}")
```

```

det = m * s2 - s1 * s1          # 2) résolution 2 x 2 (Cramer)
return (s2 * sy - s1 * sty) / det, (m * sty - s1 * sy) / det

def simpson_tableau(T, C):
    m = len(C)
    if m % 2 == 0:
        raise ValueError("il faut un nombre impair de points")
    h = T[1] - T[0]
    q = C[0] + C[m - 1]
    for i in range(1, m - 1):      # indices impairs : poids 4
        q += 4 * C[i] if i % 2 == 1 else 2 * C[i]
    return h / 3 * q

if __name__ == "__main__":
    T = [0.1, 0.3, 0.5]
    C = [0.211, 0.694, 1.361]      # Cv en 1e-4 cal/(K.mol)
    a, b = moindres_carres(T, C)
    print(f"a = {a:.6f}    b = {b:.6f}")
    print(f"Q (Simpson) = {simpson_tableau(T, C):.6f}    (x 1e-4 cal/mol)")
    q = a * (T[-1] - T[0]) + b * (T[-1] ** 2 - T[0] ** 2) / 2
    print(f"Q (droite) = {q:.6f}")

```

2-d- Q par Simpson composite sur les mesures.

Pourquoi ? Simpson composite exige des abscisses équidistantes et un nombre *pair* d'intervalles, donc un nombre *impair* de mesures. Les poids sont 1, 4, 2, 4, $\dots$, 4, 1 (exercice IV.5). Si l'on préfère utiliser la droite ajustée, on intègre exactement $a + bT$.

Algorithme : Simpson composite sur un tableau

```

Si  $m$  pair alors
    Écrire « nombre de points impair requis »
    Arrêt
Fin si
 $h \leftarrow T_2 - T_1$ ;  $Q \leftarrow C_1 + C_m$ 
Pour  $i$  allant de 2 à  $m - 1$  faire
    Si  $i$  pair alors
         $Q \leftarrow Q + 4C_i$ 
    Sinon
         $Q \leftarrow Q + 2C_i$ 
    Fin si
Fin pour
 $Q \leftarrow hQ/3$ 
Écrire  $Q$ 

```

(Avec la numérotation $i = 1, \dots, m$, les points d'indice pair sont les milieux des paires d'intervalles.)

Vérification numérique. Sur les trois mesures données, le programme affiche le système $3a + 0,9b = 2,266$; $0,9a + 0,35b = 0,9098$, puis $a = -0,107167$ et $b = 2,875$ (en 10^{-4} cal/(K mol)), et $Q = 0,289867$ ($\times 10^{-4}$ cal/mol) par Simpson. L'intégrale exacte de la droite, $a(0,5 - 0,1) + b(0,5^2 - 0,1^2)/2 = 0,302133$, diffère de 4 % : avec seulement trois points, une droite décrit mal des données qui ne sont pas alignées.

Références. [BF, th. 4.4, p. 204; alg. 4.1, p. 205; § 8.1, p. 507–508].

Exercice 13 : équation intégrale de Fredholm

Remarque sur l'énoncé. L'énoncé écrit $t_i = ih$ pour n points. Pour que la formule des trapèzes utilise les bornes a et b , on prend, comme le cours (chapitre V), $t_i = a + (i-1)h$, $i = 1, \dots, n$, avec $h = (b-a)/(n-1)$; ainsi $t_1 = a$ et $t_n = b$.

1)- Coefficients c_i .

Pourquoi ? La formule composite des trapèzes donne le poids $h/2$ aux deux extrémités et h aux points intérieurs. On l'écrit sous la forme $(b-a) \sum c_i (\dots)$ du cours en divisant les poids par $b-a$.

Comment ? (étapes)

Étape 1. $\int_a^b k(x, t) \phi(t) dt \approx h [\frac{1}{2} k(x, t_1) \phi_1 + k(x, t_2) \phi_2 + \dots + k(x, t_{n-1}) \phi_{n-1} + \frac{1}{2} k(x, t_n) \phi_n]$.

Étape 2. Comme $h = (b-a)/(n-1)$:

$$c_1 = c_n = \frac{1}{2(n-1)}, \quad c_i = \frac{1}{n-1} \quad (i = 2, \dots, n-1),$$

et l'on vérifie que $\sum c_i = 1$ (la formule est exacte pour une constante).

2)- Le système algébrique.

Pourquoi ? L'équation (2) est vraie pour tout x ; en particulier aux nœuds. En prenant $x = t_j$, le membre de gauche devient $\phi(t_j) = \phi_j$, et le membre de droite ne contient que les inconnues $\phi_1, \dots, \phi_n$: on obtient n équations linéaires à n inconnues, c'est-à-dire (3)

$$\phi_j = f_j + (b-a) \sum_{i=1}^n c_i k(t_j, t_i) \phi_i, \quad j = 1, \dots, n.$$

(C'est la méthode de Nyström.)

3)- Les matrices A , X , b . On passe les inconnues à gauche : $\phi_j - (b-a) \sum_i c_i k(t_j, t_i) \phi_i = f_j$.
Donc

$$A_{ji} = \delta_{ji} - (b-a) c_i k(t_j, t_i), \quad X = (\phi_1, \dots, \phi_n)^t, \quad b = (f_1, \dots, f_n)^t,$$

où $\delta_{ji} = 1$ si $j = i$ et 0 sinon : $A = I - (b-a) K C$, avec $K_{ji} = k(t_j, t_i)$ et $C = \text{diag}(c_1, \dots, c_n)$.

4)- Résolution par Gauss. a) Triangularisation : algorithme de l'exercice I.3 (avec pivot maximal). b) Système triangulaire : remontée, exercice I.1. c) Programme : fonctions `triangularise` et `remontee` de `gauss.py`, données à l'exercice 10 ci-dessus.

Vérification numérique. Sur l'exemple du cours, $\phi(x) = \frac{5x}{6} - \frac{1}{9} + \frac{1}{3} \int_0^1 (t+x) \phi(t) dt$ (solution exacte $\phi = x$, donc $k(x, t) = (x+t)/3$), l'erreur maximale de la méthode des trapèzes vaut $2,48 \times 10^{-2}$; $6,17 \times 10^{-3}$; $1,54 \times 10^{-3}$; $3,85 \times 10^{-4}$ pour $n = 3, 5, 9, 17$: divisée par 4 quand h est divisé par 2 (ordre 2 des trapèzes). Le cours, avec Simpson et 3 points, obtient la solution exacte, car Simpson intègre exactement le produit $(t+x)t$, polynôme de degré 2 en t .

5)- Zéro de ϕ par bisection.

Pourquoi ? La résolution donne ϕ aux nœuds seulement; mais la formule (2) permet ensuite de calculer $\phi(x)$ en *n'importe quel* x (interpolation de Nyström), ce dont la bisection a besoin. Si $\phi(a)\phi(b) < 0$, la bisection (exercice II.2) converge vers un zéro.

Algorithme : zéro de ϕ

résoudre (4) : $\phi_1, \dots, \phi_n$
définir $\phi(x) = f(x) + (b - a) \sum_{i=1}^n c_i k(x, t_i) \phi_i$
 $\alpha \leftarrow a$; $\beta \leftarrow b$
Si $\phi(\alpha)\phi(\beta) > 0$ **alors**
 Écrire « pas de changement de signe »
 Arrêt
Fin si
Tant que $\beta - \alpha > \varepsilon$ **faire**
 $\gamma \leftarrow (\alpha + \beta)/2$
 Si $\phi(\alpha)\phi(\gamma) \leq 0$ **alors**
 $\beta \leftarrow \gamma$
 Sinon
 $\alpha \leftarrow \gamma$
 Fin si
Fin tant que
Écrire $(\alpha + \beta)/2$

Vérification numérique. Avec le noyau $(x + t)/3$ sur $[0, 1]$ et $f(x) = x - \frac{1}{2} - \frac{1}{36}$, la solution exacte est $\phi = x - \frac{1}{2}$ (zéro en 0,5). La bisection sur la solution de Nyström donne 0,494718 ($n = 5$), 0,498680 ($n = 9$), 0,499670 ($n = 17$) : l'erreur suit celle de la discrétisation (ordre 2).

Références. [BF, alg. 6.2, p. 378 ; alg. 2.1, p. 49 ; th. 4.5, p. 205].

Exercices de fin de document (14 à 19)

Exercice 14 : température dans un mur

Remarque sur l'énoncé. À $t = 0$, la face $x = L$ est à T_0 (condition initiale) mais doit être à T_L (condition aux limites) : les deux conditions ne se raccordent pas si $T_L \neq T_0$. On impose T_L dès le premier nœud de temps ($T_{m,0} = T_L$) ; ce choix n'influe que sur les tout premiers pas. En $x = 0$, $T_0 + T_1 \sin 0 = T_0$: pas de conflit. Le temps t est mesuré dans l'unité implicite de la loi $\sin(\pi t/2)$ (période 4).

1-a- Forme discrète par différences progressives.

Pourquoi ? C'est le schéma explicite du cours (C2) appliqué à $\partial T / \partial t = \alpha \partial^2 T / \partial x^2$; seules les conditions aux limites changent.

Comment ? (étapes)

Étape 1. Grille $x_i = i\Delta x$, $\Delta x = L/m$; $t_j = j\Delta t$; $T_{i,j} \approx T(x_i, t_j)$; $\lambda = \alpha\Delta t / \Delta x^2$.

Étape 2. Dérivée en temps progressive, dérivée en espace centrée :

$$\frac{T_{i,j+1} - T_{i,j}}{\Delta t} = \alpha \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} \implies T_{i,j+1} = \lambda T_{i+1,j} + (1 - 2\lambda)T_{i,j} + \lambda T_{i-1,j},$$

pour $i = 1, \dots, m-1$ et $j \geq 0$.

Étape 3. Conditions : $T_{i,0} = T_0$ ($0 \leq i < m$), $T_{m,j} = T_L$, $T_{0,j} = T_0 + T_1 \sin(\pi j\Delta t/2)$.

Étape 4. Erreur de discrétisation : $O(\Delta t)$ pour la dérivée en temps et $O(\Delta x^2)$ pour la dérivée en espace, soit $O(\Delta t + \Delta x^2)$ [BF, p. 747]. Le schéma est stable si $\lambda \leq \frac{1}{2}$ (exercice 15).

1-b- Algorithme.

Algorithme : mur, schéma explicite

Données : $L, \alpha, T_0, T_1, T_L, m, \Delta t, t_{\max}$

$\Delta x \leftarrow L/m$; $\lambda \leftarrow \alpha\Delta t / \Delta x^2$

Si $\lambda > 1/2$ **alors**

Écrire « instable : réduire Δt »

Fin si

Pour i allant de 0 à $m-1$ **faire**

$U_i \leftarrow T_0$

Fin pour

$U_m \leftarrow T_L$

Pour j allant de 1 à $t_{\max}/\Delta t$ **faire**

Pour i allant de 1 à $m-1$ **faire**

$V_i \leftarrow (1 - 2\lambda)U_i + \lambda(U_{i+1} + U_{i-1})$

Fin pour

$V_0 \leftarrow T_0 + T_1 \sin(\pi j\Delta t/2)$; $V_m \leftarrow T_L$

Pour i allant de 0 à m **faire**

$U_i \leftarrow V_i$

Fin pour

Fin pour

Écrire $(i\Delta x, U_i)$

1-c- Programme.

Programme Python (fichier programmes/ex14_explicite.py) :

```
"""Exercice 14, question 1 : mur d'épaisseur L,  $dT/dt = \alpha d^2T/dx^2$ ,
 $T(x,0) = T_0$ ,  $T(0,t) = T_0 + T_1 \sin(\pi t / 2)$ ,  $T(L,t) = T_L$ .
Schéma explicite (différences progressives en temps)."""
import math

def mur_explicite(L, alpha, T0, T1, TL, m, dt, tmax):
    dx = L / m
    lam = alpha * dt / dx ** 2
    print(f"lambda = {lam:.4f}")
    if lam > 0.5:
        print("Attention : lambda > 1/2, schéma instable")
    U = [T0] * m + [TL] # condition initiale (TL dès t > 0)
    nt = round(tmax / dt)
    for j in range(1, nt + 1):
        V = U[:]
        for i in range(1, m):
            V[i] = (1 - 2 * lam) * U[i] + lam * (U[i + 1] + U[i - 1])
        V[0] = T0 + T1 * math.sin(math.pi * j * dt / 2)
        V[m] = TL
        U = V
    return [(i * dx, U[i]) for i in range(m + 1)]

if __name__ == "__main__":
    profil = mur_explicite(L=1, alpha=1, T0=20, T1=0, TL=100,
                           m=10, dt=0.004, tmax=2)
    for x, T in profil:
        print(f"{x:6.2f} {T:12.6f}")
    profil = mur_explicite(L=1, alpha=0.1, T0=20, T1=10, TL=30,
                           m=10, dt=0.04, tmax=3)
    for x, T in profil:
        print(f"{x:6.2f} {T:12.6f}")
    profil = mur_explicite(L=1, alpha=0.1, T0=20, T1=10, TL=30,
                           m=10, dt=0.06, tmax=3)
    print("dt = 0,06 :", [f"{T:.3e}" for x, T in profil[1:3]])
```

Vérification numérique.

- Sans oscillation ($T_1 = 0$, $T_0 = 20$, $T_L = 100$, $\alpha = 1$, $L = 1$, $m = 10$, $\Delta t = 0,004$, $\lambda = 0,4$), à $t = 2$ le programme donne le profil linéaire 20, 28, 36, ..., 100 : c'est le régime permanent exact $T = T_0 + (T_L - T_0)x/L$ (le schéma reproduit exactement les fonctions affines en x).
- $\alpha = 0,1$, $T_0 = 20$, $T_1 = 10$, $T_L = 30$, $\Delta t = 0,04$ ($\lambda = 0,4$) : à $t = 3$, $T(0) = 10$ (car $\sin(3\pi/2) = -1$), $T(0,5) = 24,36$, $T(L) = 30$.
- Avec $\Delta t = 0,06$ ($\lambda = 0,6 > \frac{1}{2}$), le calcul explose (valeurs de l'ordre de 10^5 à $t = 3$) : l'instabilité prévue.

2-a- Schéma de Richardson.

Pourquoi ? On remplace la dérivée en temps par la différence centrée $(T_{i,j+1} - T_{i,j-1})/(2\Delta t)$, d'erreur $O(\Delta t^2)$, pour gagner un ordre en temps.

Comment ? (étapes)

Étape 1. $T_{i,j+1} = T_{i,j-1} + 2\lambda(T_{i+1,j} - 2T_{i,j} + T_{i-1,j})$, erreur de discrétisation $O(\Delta t^2 + \Delta x^2)$ [BF, (12.14), p. 751].

Étape 2. Stabilité (méthode de l'introduction du chapitre VI) : $\xi - \xi^{-1} = 2\lambda(-4s)$, soit $\xi^2 + 8\lambda s \xi - 1 = 0$. Le produit des racines vaut -1 et le discriminant $64\lambda^2 s^2 + 4$ est positif : les racines sont réelles, distinctes, de produit de module 1, donc l'une a un module > 1 dès que $s > 0$. Le schéma est **instable quel que soit** λ (le cours et [BF, p. 751] le signalent).

2-b- Algorithme.

Pourquoi ? Le schéma a trois niveaux : il faut $T_{i,0}$ (condition initiale) et $T_{i,1}$, donné par un pas du schéma explicite, comme le suggère l'énoncé.

Algorithme : Richardson

$U^{\text{anc}} \leftarrow$ condition initiale ; $U^{\text{cour}} \leftarrow$ un pas explicite (question 1)

Pour j allant de 1 à $N - 1$ **faire**

Pour i allant de 1 à $m - 1$ **faire**

$U_i^{\text{nouv}} \leftarrow U_i^{\text{anc}} + 2\lambda(U_{i+1}^{\text{cour}} - 2U_i^{\text{cour}} + U_{i-1}^{\text{cour}})$

Fin pour

$U_0^{\text{nouv}} \leftarrow T_0 + T_1 \sin(\pi(j+1)\Delta t/2)$; $U_m^{\text{nouv}} \leftarrow T_L$

$U^{\text{anc}} \leftarrow U^{\text{cour}} ; U^{\text{cour}} \leftarrow U^{\text{nouv}}$

Fin pour

2-c- Programme.

Programme Python (fichier programmes/ex14_richardson.py) :

```
"""Exercice 14, question 2 : schéma de Richardson (saute-mouton)
   T(i, j+1) = T(i, j-1) + 2 lambda (T(i+1, j) - 2 T(i, j) + T(i-1, j)),
   démarré avec T(i, 1) donné par le schéma explicite de la question 1."""
import math

def mur_richardson(L, alpha, T0, T1, TL, m, dt, tmax):
    dx = L / m
    lam = alpha * dt / dx ** 2
    print(f"lambda = {lam:.4f}")
    bord0 = lambda t: T0 + T1 * math.sin(math.pi * t / 2)
    ancien = [T0] * m + [TL] # niveau j = 0
    courant = ancien[:] # niveau j = 1 : schéma explicite
    for i in range(1, m):
        courant[i] = ((1 - 2 * lam) * ancien[i]
                      + lam * (ancien[i + 1] + ancien[i - 1]))
    courant[0], courant[m] = bord0(dt), TL
    nt = round(tmax / dt)
    for j in range(1, nt): # niveaux j + 1 = 2, 3, ...
        nouveau = courant[:]
        for i in range(1, m):
            nouveau[i] = ancien[i] + 2 * lam * (
                courant[i + 1] - 2 * courant[i] + courant[i - 1])
```

```

nouveau[0], nouveau[m] = bord0((j + 1) * dt), TL
ancien, courant = courant, nouveau
if (j + 1) % (nt // 10) == 0:
    print(f"t = {(j + 1) * dt:6.3f}"
          f"    T(L/2) = {courant[m // 2]:.4e}")

if __name__ == "__main__":
    mur_richardson(L=1, alpha=0.1, T0=20, T1=10, TL=30,
                  m=10, dt=0.01, tmax=3)

```

Vérification numérique. Même avec $\lambda = 0,1$ (très en dessous de la limite du schéma explicite), le programme donne $T(L/2) = -3,3 \times 10^2$ à $t = 0,3$, $-3,7 \times 10^{17}$ à $t = 1,2$ et $-2,2 \times 10^{47}$ à $t = 3$: la divergence confirme l'analyse. En pratique, pour l'ordre 2 en temps, on utilise Crank-Nicolson (exercice VI.4).

Références. [BF, p. 743–747 (schéma explicite); (12.14), p. 751 (Richardson)].

Exercice 15 : conditions de stabilité

1) Schéma explicite à une dimension.

Pourquoi ? On applique l'analyse de von Neumann (introduction du chapitre VI) : une perturbation vérifie le même schéma que la solution.

Comment ? (étapes)

Étape 1. $\varepsilon_{i,j+1} = (1 - 2\lambda)\varepsilon_{i,j} + \lambda(\varepsilon_{i+1,j} + \varepsilon_{i-1,j})$, $\lambda = a^2k/h^2$.

Étape 2. Avec $\varepsilon_{i,j} = \xi^j e^{Ii\theta}$: $\xi = 1 - 2\lambda + 2\lambda \cos \theta = 1 - 4\lambda s$, $s = \sin^2(\theta/2)$.

Étape 3. $|\xi| \leq 1 \iff -1 \leq 1 - 4\lambda s \leq 1$. L'inégalité de droite est toujours vraie; celle de gauche s'écrit $\lambda s \leq \frac{1}{2}$ pour tout $s \in [0, 1]$, donc (cas le plus défavorable $s = 1$, mode le plus oscillant)

$$\frac{a^2k}{h^2} \leq \frac{1}{2}.$$

2) Problème plan.

Étape 1. Schéma : $w_{i,l}^{j+1} = w_{i,l}^j + \lambda_1 \delta_x^2 w^j + \lambda_2 \delta_y^2 w^j$, avec $\lambda_1 = a^2k/h_1^2$, $\lambda_2 = a^2k/h_2^2$.

Étape 2. Mode $\xi^j e^{I(i\theta_1 + l\theta_2)}$: $\xi = 1 - 4\lambda_1 s_1 - 4\lambda_2 s_2$.

Étape 3. Le cas le plus défavorable est $s_1 = s_2 = 1$: $\xi = 1 - 4(\lambda_1 + \lambda_2) \geq -1$, d'où

$$a^2k \left(\frac{1}{h_1^2} + \frac{1}{h_2^2} \right) \leq \frac{1}{2}, \quad \text{soit} \quad \frac{a^2k}{h^2} \leq \frac{1}{4} \quad \text{si } h_1 = h_2 = h.$$

Le pas de temps admissible est deux fois plus petit qu'en dimension 1.

Vérification numérique. $\max |\xi| = 1$ pour $(\lambda_1, \lambda_2) = (0,25; 0,25)$ et $(0,3; 0,2)$, mais 1,2 pour $(0,3; 0,25)$.

3) Advection-diffusion dans le plan.

Comment ? (étapes)

Étape 1. Schéma (temps progressif, espace centré, h dans les deux directions) avec $d = \nu k/h^2$, $c_1 = u_1 k/h$, $c_2 = u_2 k/h$:

$$\Omega_{i,l}^{j+1} = \Omega_{i,l}^j - \frac{c_1}{2}(\Omega_{i+1,l}^j - \Omega_{i-1,l}^j) - \frac{c_2}{2}(\Omega_{i,l+1}^j - \Omega_{i,l-1}^j) + d(\delta_x^2 + \delta_y^2)\Omega_{i,l}^j.$$

Étape 2. Facteur d'amplification :

$$\xi = 1 - 4d(s_1 + s_2) - I(c_1 \sin \theta_1 + c_2 \sin \theta_2).$$

Étape 3. Condition établie par l'énoncé. Pour $\theta_1 = \theta_2 = \pi$ ($s_1 = s_2 = 1$, sinus nuls), $\xi = 1 - 8d$, et $|\xi| \leq 1$ impose

$$\frac{\nu k}{h^2} \leq \frac{1}{4}.$$

C'est une condition *nécessaire* ; elle est aussi suffisante si $u_1 = u_2 = 0$ (on retrouve la question 2 avec $a^2 = \nu$).

Étape 4. Condition supplémentaire due à l'advection. Pour θ petit, $|\xi|^2 \approx 1 - 2d|\theta|^2 + (c_1\theta_1 + c_2\theta_2)^2$; en choisissant θ parallèle à (c_1, c_2) , il faut $c_1^2 + c_2^2 \leq 2d$. Le calcul numérique de $\max |\xi|$ sur tous les (θ_1, θ_2) confirme que le schéma est stable si et seulement si $d \leq \frac{1}{4}$ et $c_1^2 + c_2^2 \leq 2d$ (par exemple, avec $d = 0,1$: stable pour $c_1 = c_2 = 0,316$, instable pour $0,33$). Ce résultat est en accord avec l'étude de Hindmarsh, Gresho et Griffiths [HGG].

Cas unidimensionnel. Avec $s = \sin^2(\theta/2)$ et $\sin^2 \theta = 4s(1-s)$:

$$|\xi|^2 = (1 - 4ds)^2 + 4c^2s(1-s) = 1 + s[(4c^2 - 8d) + s(16d^2 - 4c^2)].$$

Le crochet est une fonction affine de s ; il est négatif ou nul sur $[0, 1]$ si et seulement s'il l'est en $s = 0$ et en $s = 1$, c'est-à-dire

$$c^2 \leq 2d \quad \text{et} \quad d = \frac{\nu k}{h^2} \leq \frac{1}{2}.$$

La condition de diffusion devient $\nu k/h^2 \leq \frac{1}{2}$ (au lieu de $\frac{1}{4}$), et l'advection ajoute $(uk/h)^2 \leq 2\nu k/h^2$.

Vérification numérique. $d = 0,5$: stable pour $c = 1$, instable pour $c = 1,01$; $d = 0,2$: stable pour $c = 0,6$ ($c^2 = 0,36 \leq 0,4$), instable pour $c = 0,65$.

Références. [BF, p. 746–747] ; [VN] ; [HGG].

Exercice 16 : équation d'ondes amortie et non linéaire

1)- Signification des termes. Par analogie avec l'équation de la corde vibrante du cours, $\rho U_{tt} + kU_t = TU_{xx} + f$:

- $\partial^2 u / \partial t^2$: accélération (terme d'inertie) ;
- $a \partial u / \partial t$: frottement visqueux, amortissement ($a > 0$) ;
- $c^2 \partial^2 u / \partial x^2$: force de rappel élastique ; c est la célérité des ondes ($c^2 = T/\rho$ pour une corde) ;
- du^3 : force non linéaire cubique (selon le signe de d , elle s'oppose au déplacement, $d < 0$, ou l'amplifie, $d > 0$).

Conditions : extrémités fixes, déplacement initial f , vitesse initiale g .

2)- Forme discrète (différences centrées).

Pourquoi ? Les différences centrées sont d'ordre 2 en temps et en espace, comme pour (O2). Le terme d'amortissement centré fait apparaître $u_{i,j+1}$ des deux côtés ; on le regroupe : le schéma reste explicite.

Comment ? (étapes)

Étape 1. Avec $x_i = ih$ ($h = l/m$), $t_j = jk$:

$$\frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{k^2} + a \frac{u_{i,j+1} - u_{i,j-1}}{2k} = c^2 \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + d u_{i,j}^3.$$

Étape 2. On multiplie par k^2 , on pose $\lambda = ck/h$ et $\beta = ak/2$:

$$(1 + \beta) u_{i,j+1} = 2u_{i,j} - (1 - \beta) u_{i,j-1} + \lambda^2 (u_{i+1,j} - 2u_{i,j} + u_{i-1,j}) + k^2 d u_{i,j}^3,$$

pour $i = 1, \dots, m-1$, $j \geq 1$.

Étape 3. Conditions : $u_{0,j} = u_{m,j} = 0$; $u_{i,0} = f_i = f(ih)$.

Étape 4. Stabilité de la partie linéaire ($d = 0$) : ξ vérifie $(1 + \beta)\xi^2 - 2(1 - 2\lambda^2 s)\xi + (1 - \beta) = 0$. Si $\lambda \leq 1$ (et $0 \leq \beta < 1$), on vérifie que les racines sont de module ≤ 1 (le produit des racines vaut $(1 - \beta)/(1 + \beta) < 1$ et, si elles sont réelles, le polynôme est positif en $\xi = \pm 1$). La condition est donc encore $ck/h \leq 1$; numériquement, $\max |\xi| = 1$ pour $\lambda = 1$, $\beta = 0,1$, mais 1,71 pour $\lambda = 1,05$. Le terme du^3 n'est pas couvert par cette analyse linéaire.

3)- Expression de $u_{i,2}$.

Pourquoi ? Le schéma demande deux niveaux pour démarrer. Le niveau $t = k$ s'obtient par Taylor, comme (O6) du cours, mais en utilisant l'équation elle-même pour $u_{tt}(x, 0)$; le niveau $t = 2k$ s'obtient ensuite par le schéma avec $j = 1$.

Comment ? (étapes)

Étape 1. $u(x, k) = u(x, 0) + k u_t(x, 0) + \frac{k^2}{2} u_{tt}(x, 0) + O(k^3)$, et d'après l'équation, $u_{tt}(x, 0) = c^2 f''(x) + d f(x)^3 - a g(x)$. D'où

$$u_{l,1} = f_l + k g_l + \frac{k^2}{2} \left[c^2 \frac{f_{l+1} - 2f_l + f_{l-1}}{h^2} + d f_l^3 - a g_l \right],$$

qui utilise f_{l-1} , f_l , f_{l+1} et g_l .

Étape 2. Le schéma avec $j = 1$:

$$u_{i,2} = \frac{2u_{i,1} - (1 - \beta)f_i + \lambda^2(u_{i+1,1} - 2u_{i,1} + u_{i-1,1}) + k^2 d u_{i,1}^3}{1 + \beta}.$$

Il fait intervenir $u_{i-1,1}$, $u_{i,1}$, $u_{i+1,1}$, donc (étape 1) $f_{i-2}, \dots, f_{i+2}$ et g_{i-1}, g_i, g_{i+1} : c'est l'expression demandée.

Étape 3. Forme entièrement développée dans le cas $d = 0$ (vérifiée numériquement) :

$$(1 + \beta) u_{i,2} = (1 - 4\lambda^2 + 3\lambda^4 + \beta) f_i + 2\lambda^2(1 - \lambda^2)(f_{i+1} + f_{i-1}) + \frac{\lambda^4}{2}(f_{i+2} + f_{i-2}) \\ + k(1 - \beta) \left[2(1 - \lambda^2)g_i + \lambda^2(g_{i+1} + g_{i-1}) \right].$$

Contrôle : si $f \equiv F$ constante et $g = 0$, la somme des coefficients de f vaut $1 + \beta$, d'où $u_{i,2} = F$, ce qui est correct loin des bords.

4)- Algorithme.

Algorithme : onde amortie non linéaire

Données : $a, c, d, l, m, k, t_{\max}, f, g$

$h \leftarrow l/m$; $\lambda \leftarrow ck/h$; $\beta \leftarrow ak/2$

Si $\lambda > 1$ **alors**

Écrire « instable »

Fin si

Pour i allant de 0 à m **faire**

$U_i^{\text{anc}} \leftarrow f(ih)$

Fin pour

$U_0^{\text{anc}} \leftarrow 0$; $U_m^{\text{anc}} \leftarrow 0$

Pour i allant de 1 à $m - 1$ **faire**

$U_i^{\text{cour}} \leftarrow U_i^{\text{anc}} + kg_i + \frac{k^2}{2} [c^2(U_{i+1}^{\text{anc}} - 2U_i^{\text{anc}} + U_{i-1}^{\text{anc}})/h^2 + d(U_i^{\text{anc}})^3 - ag_i]$

Fin pour

$U_0^{\text{cour}} \leftarrow 0$; $U_m^{\text{cour}} \leftarrow 0$

Pour j allant de 1 à $t_{\max}/k - 1$ **faire**

Pour i allant de 1 à $m - 1$ **faire**

$U_i^{\text{nouv}} \leftarrow [2U_i^{\text{cour}} - (1 - \beta)U_i^{\text{anc}} + \lambda^2(U_{i+1}^{\text{cour}} - 2U_i^{\text{cour}} + U_{i-1}^{\text{cour}}) + k^2d(U_i^{\text{cour}})^3]/(1 + \beta)$

Fin pour

$U_0^{\text{nouv}} \leftarrow 0$; $U_m^{\text{nouv}} \leftarrow 0$

$U^{\text{anc}} \leftarrow U^{\text{cour}}$; $U^{\text{cour}} \leftarrow U^{\text{nouv}}$

Fin pour

Écrire (ih, U_i^{cour})

5)- Programme.

Programme Python (fichier `programmes/ex16_onde.py`) :

```
"""Exercice 16 : u_tt + a u_t = c^2 u_xx + d u^3 sur ]0, l[,
u(0,t) = u(l,t) = 0, u(x,0) = f(x), u_t(x,0) = g(x).
Différences centrées en temps et en espace (schéma à trois niveaux)."""
import math
```

```
def onde(a, c, d, l, m, k, tmax, f, g):
    h = l / m
    lam2 = (c * k / h) ** 2
    beta = a * k / 2
    if c * k / h > 1:
        print("Attention : lambda = c k / h > 1, schéma instable")
    # niveau 0 : u(x,0) = f(x)
    ancien = [f(i * h) for i in range(m + 1)]
```

```

ancien[0] = ancien[m] = 0.0
# niveau 1 : Taylor à l'ordre 2,  $u_{tt}(x,0) = c^2 f'' + d f^3 - a g$ 
courant = [0.0] * (m + 1)
for i in range(1, m):
    gi = g(i * h)
    d2f = (ancien[i + 1] - 2 * ancien[i] + ancien[i - 1]) / h ** 2
    courant[i] = (ancien[i] + k * gi
                  + k * k / 2 * (c * c * d2f + d * ancien[i] ** 3
                                - a * gi))

# niveaux 2, 3, ...
nt = round(tmax / k)
for j in range(1, nt):
    nouveau = [0.0] * (m + 1)
    for i in range(1, m):
        d2u = courant[i + 1] - 2 * courant[i] + courant[i - 1]
        nouveau[i] = (2 * courant[i] - (1 - beta) * ancien[i]
                      + lam2 * d2u
                      + k * k * d * courant[i] ** 3) / (1 + beta)
    ancien, courant = courant, nouveau
return courant

if __name__ == "__main__":
    f = lambda x: math.sin(math.pi * x)    # déplacement initial ( $l = 1$ )
    g = lambda x: 0.0                      # vitesse initiale
    u = onde(a=0, c=1, d=0, l=1, m=50, k=0.01, tmax=1.5, f=f, g=g)
    print(f"a = d = 0 : u(0,5 ; 1,5) = {u[25]:.8f} (exact 0)")
    u = onde(a=0.4, c=1, d=0, l=1, m=50, k=0.01, tmax=1.5, f=f, g=g)
    w = math.sqrt(math.pi ** 2 - 0.04)
    exact = math.exp(-0.3) * (math.cos(1.5 * w)
                              + 0.2 / w * math.sin(1.5 * w))
    print(f"a = 0,4, d = 0 : u(0,5 ; 1,5) = {u[25]:.8f}"
          f" (exact {exact:.8f})")
    for m in (50, 100, 200):
        u = onde(a=0.4, c=1, d=-2, l=1, m=m, k=0.5 / m, tmax=1.5,
                  f=f, g=g)
        print(f"a = 0,4, d = -2, m = {m:3d} : u(0,5 ; 1,5) ="
              f" {u[m // 2]:.8f}")

```

Vérification numérique. $l = 1, c = 1, f = \sin \pi x, g = 0, h = 0,02, k = 0,01$, valeur en $x = 0,5$ et $t = 1,5$:

- $a = d = 0$: $-5,8 \times 10^{-4}$ (exact : $\sin(\pi/2) \cos(1,5\pi) = 0$) ;
- $a = 0,4, d = 0$: $-0,054756$; la solution exacte $e^{-at/2} \sin \pi x [\cos \omega t + \frac{a}{2\omega} \sin \omega t]$, $\omega = \sqrt{\pi^2 - a^2/4}$, vaut $-0,054337$;
- $a = 0,4, d = -2$ (pas de solution exacte connue), avec $k = h/2$: $0,093020$ ($m = 50$), $0,093271$ ($m = 100$), $0,093334$ ($m = 200$) ; les écarts successifs ($2,5 \times 10^{-4}$ puis $6,3 \times 10^{-5}$) sont divisés par 4 : le schéma converge à l'ordre 2.

Références. [BF, (12.19)–(12.25), p. 758–760 ; alg. 12.4, p. 761 ; p. 762].

Exercice 17 : pression dans la paroi d'un tube

Remarque sur l'énoncé. La position du rectangle intérieur n'est pas cotée ; on le suppose centré, comme sur la figure (marges de 250 mm en x et de 200 mm en y).

1)- Adimensionnement.

Pourquoi ? On veut des grandeurs sans dimension d'ordre 1 et une équation sans paramètre. En prenant la *même* échelle L pour x et y , l'équation de Laplace garde sa forme ; avec des échelles différentes, un coefficient $(L/l)^2$ apparaîtrait devant $\partial^2/\partial Y^2$.

Comment ? (étapes)

Étape 1. $X = x/L$, $Y = y/L$, $u = p/p_s$. Comme $\partial^2 p / \partial x^2 = (p_s/L^2) \partial^2 u / \partial X^2$:

$$\frac{\partial^2 u}{\partial X^2} + \frac{\partial^2 u}{\partial Y^2} = 0.$$

Étape 2. Domaine : $0 \leq X \leq 1$, $0 \leq Y \leq 0,8$, privé du rectangle $0,25 < X < 0,75$, $0,2 < Y < 0,6$.

Étape 3. Conditions : $u = 0$ sur le bord extérieur, $u = 1$ sur le bord intérieur.

2-a)- Équation discrète.

Pourquoi ? On remplace chaque dérivée seconde par la différence centrée (exercice 8) ; c'est le cas $f = 0$ de (P2).

Étape 1. Avec $X_i = ih$, $Y_j = jk$: $\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{k^2} = 0$.

Étape 2. Pour $h = k$: $u_{i,j} = \frac{1}{4}(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1})$ (la valeur au centre est la moyenne des quatre voisins).

2-b)- Conditions aux limites discrètes.

Pourquoi ? Pour que les deux bords tombent sur des nœuds de la grille, il faut que h divise 1, 0,8, 0,25 et 0,2 : on prend $h = k = 0,05$ (50 mm), soit $i = 0, \dots, 20$ et $j = 0, \dots, 16$.

Étape 1. Bord extérieur : $u_{0,j} = u_{20,j} = 0$, $u_{i,0} = u_{i,16} = 0$.

Étape 2. Bord intérieur : $u_{i,j} = 1$ pour ($i = 5$ ou 15 , $4 \leq j \leq 12$) et pour ($j = 4$ ou 12 , $5 \leq i \leq 15$).

Étape 3. Inconnues : les nœuds avec $0 < i < 20$, $0 < j < 16$, hors du rectangle $5 \leq i \leq 15$, $4 \leq j \leq 12$.

3-a)- Méthode de Gauss-Seidel. On parcourt les nœuds inconnus et on remplace chaque $u_{i,j}$ par la moyenne de ses voisins, en utilisant tout de suite les valeurs déjà mises à jour ; on recommence jusqu'à ce que la variation relative soit inférieure à ε (exercices I.7 et VI.1).

3-b)- Algorithme.

Algorithme : Gauss-Seidel sur le cadre

fixer u sur les deux bords ; $u \leftarrow 0$ aux nœuds inconnus

Répéter

$num \leftarrow 0$; $den \leftarrow 0$

Pour i allant de 1 à 19 **faire**

Pour j allant de 1 à 15 **faire**

Si (i, j) n'est pas dans $[5, 15] \times [4, 12]$ **alors**

$t \leftarrow (u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1})/4$

$num \leftarrow num + |t - u_{i,j}|$; $den \leftarrow den + |t|$; $u_{i,j} \leftarrow t$

Fin si

Fin pour

Fin pour

Jusqu'à $num/den < \varepsilon$

Écrire $p_{i,j} = p_s u_{i,j}$

Remarque : le domaine et les conditions sont symétriques par rapport aux droites $X = 0,5$ et $Y = 0,4$; on pourrait ne calculer qu'un quart du domaine.

Vérification numérique. Avec $u = X^2 - Y^2$ imposé sur les bords (fonction harmonique, pour laquelle le schéma est exact), l'algorithme retrouve u à 4×10^{-12} près (124 itérations). Pour le problème posé ($\varepsilon = 10^{-10}$), il converge en 102 itérations et toutes les valeurs restent entre 0 et 1, comme le veut le principe du maximum discret du cours.

4-a)- Tube cylindrique.

Pourquoi ? Avec des bords circulaires concentriques, les coordonnées polaires (r, φ) épousent la géométrie, et le problème ne dépend pas de φ (symétrie de révolution).

Étape 1. Laplacien en coordonnées polaires : $\frac{\partial^2 p}{\partial r^2} + \frac{1}{r} \frac{\partial p}{\partial r} + \frac{1}{r^2} \frac{\partial^2 p}{\partial \varphi^2} = 0$.

Étape 2. Sans dépendance en φ : $\frac{d^2 u}{dr^2} + \frac{1}{r} \frac{du}{dr} = 0$, $u(R_1) = 1$, $u(R_2) = 0$ (avec $\rho = r/R_2$, l'équation garde la même forme sur $[\frac{1}{2}, 1]$).

Étape 3. Solution exacte (utile pour vérifier) : $\frac{d}{dr}(r \frac{du}{dr}) = 0$ donne $u = A \ln r + B$, d'où $u = \frac{\ln(r/R_2)}{\ln(R_1/R_2)}$.

4-b)- Discrétisation.

Étape 1. $r_i = R_1 + i\Delta r$, $\Delta r = (R_2 - R_1)/N$; différences centrées :

$$\left(1 - \frac{\Delta r}{2r_i}\right)u_{i-1} - 2u_i + \left(1 + \frac{\Delta r}{2r_i}\right)u_{i+1} = 0, \quad u_0 = 1, \quad u_N = 0.$$

C'est l'exercice 8 avec $p = -1/r$, $q = g = 0$: système tridiagonal, résolu par Crout ou par Jacobi/Gauss-Seidel.

Étape 2. Si le problème dépendait de φ , on utiliserait une grille (r_i, φ_j) avec $\varphi_j = j\Delta\varphi$ et la condition de périodicité $u_{i,j+M} = u_{i,j}$: $\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta r^2} + \frac{u_{i+1,j} - u_{i-1,j}}{2r_i \Delta r} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{r_i^2 \Delta \varphi^2} = 0$.

Vérification numérique. Avec $N = 40$, l'erreur maximale par rapport à la solution exacte est $4,8 \times 10^{-6}$.

Références. [BF, § 12.1, alg. 12.1, p. 738–739 ; § 11.3].

Exercice 18 : équation de Poisson sur une plaque entaillée

1)- Équation discrète.

Pourquoi ? Le domaine n'est pas un rectangle, mais il est formé de morceaux de rectangles : si la grille est choisie pour que tous les bords (y compris ceux de l'entaille) passent par des nœuds, on peut utiliser le schéma à cinq points (P2) partout à l'intérieur, sans formule spéciale près des bords.

Comment ? (étapes)

Étape 1. Grille : $h = L/(3p)$ et $k = l/(2q)$ avec p, q entiers ; alors les abscisses $L/3$ et $2L/3$ correspondent à $i = p$ et $i = 2p$, et la hauteur $l/2$ de l'entaille à $j = q$.

Étape 2. En tout nœud intérieur, avec $r = h/k$:

$$2(1 + r^2)u_{i,j} - (u_{i+1,j} + u_{i-1,j}) - r^2(u_{i,j+1} + u_{i,j-1}) = -h^2 f_{i,j}.$$

Étape 3. Nœuds intérieurs : $0 < i < 3p$, $0 < j < 2q$, sauf ceux de l'entaille et de ses bords ($p \leq i \leq 2p$ et $j \leq q$). Tous les autres nœuds sont sur la frontière et reçoivent $u_{i,j} = g(x_i, y_j)$.

2-a)- Méthode de Jacobi. On calcule de nouvelles valeurs en tous les nœuds intérieurs à partir des seules anciennes valeurs (deux grilles), puis on remplace l'ancienne grille ; on recommence jusqu'au test d'arrêt.

2-b)- Algorithme.

Algorithme : Jacobi sur la plaque entaillée

Données : $L, l, p, q, f, g, \varepsilon, q_{\max}$

$h \leftarrow L/(3p)$; $k \leftarrow l/(2q)$; $r^2 \leftarrow (h/k)^2$

intérieur(i, j) : vrai si $0 < i < 3p$, $0 < j < 2q$ et non ($p \leq i \leq 2p$ et $j \leq q$)

$u_{i,j} \leftarrow g(ih, jk)$ si non **intérieur**(i, j) ; $u_{i,j} \leftarrow 0$ sinon

Pour q' allant de 1 à $q_{\max}$ **faire**

$num \leftarrow 0$; $den \leftarrow 0$

Pour tous les (i, j) intérieurs **faire**

$v_{i,j} \leftarrow [u_{i+1,j} + u_{i-1,j} + r^2(u_{i,j+1} + u_{i,j-1}) - h^2 f(ih, jk)] / (2(1 + r^2))$

$num \leftarrow num + |v_{i,j} - u_{i,j}|$; $den \leftarrow den + |v_{i,j}|$

Fin pour

$u_{i,j} \leftarrow v_{i,j}$ pour tous les (i, j) intérieurs

Si $num/den < \varepsilon$ **alors**

 Écrire u

Arrêt

Fin si

Fin pour

Vérification numérique. $L = 1$, $l = 0,8$, $p = 4$, $q = 3$, solution test $u = x^2 + y^2$ (donc $f = 4$ et $g = x^2 + y^2$) : Jacobi converge en 154 itérations et l'erreur maximale est 6×10^{-12} (le schéma est exact pour ce polynôme).

Références. [BF, § 12.1, p. 734–739].

Exercice 19 : équation de Laplace sur une plaque triangulaire

Remarque sur l'énoncé. On prend pour n la normale *extérieure* au domaine (convention usuelle). Si n est la normale intérieure, il suffit de changer a en $-a$.

1)- Discrétisation.

Pourquoi ? Deux difficultés : l'hypoténuse est oblique, et les deux autres côtés portent une condition de Neumann. Pour la première, on choisit des pas tels que l'hypoténuse passe par des nœuds. Pour la seconde, on écrit l'équation aussi aux nœuds des côtés de Neumann, en introduisant des points « fictifs » hors du domaine, éliminés grâce à la condition sur la dérivée.

Comment ? (étapes)

Étape 1. Sommets $(0, 0)$, $(l, 0)$, $(0, h)$; hypoténuse $x/l + y/h = 1$. Pas $\Delta x = l/N$, $\Delta y = h/N$: le nœud (i, j) est sur l'hypoténuse si $i + j = N$. Inconnues : $i, j \geq 0$, $i + j < N$.

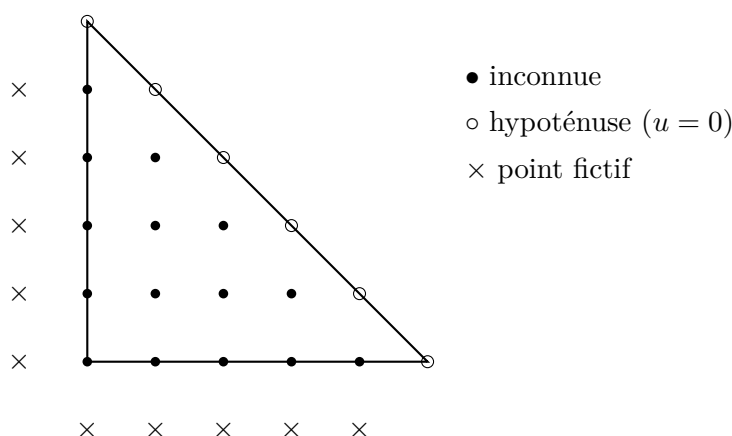
Étape 2. Hypoténuse : $u_{i,j} = 0$ si $i + j = N$.

Étape 3. Nœuds intérieurs et nœuds des côtés, schéma à cinq points ($\rho = \Delta x / \Delta y$) :

$$u_{i,j} = \frac{u_{i+1,j} + u_{i-1,j} + \rho^2(u_{i,j+1} + u_{i,j-1})}{2(1 + \rho^2)}.$$

Étape 4. Côté $y = 0$: la normale extérieure est $-\vec{e}_y$, donc $-\partial u / \partial y = a$. Par différence centrée, $(u_{i,1} - u_{i,-1}) / (2\Delta y) = -a$, d'où le point fictif $u_{i,-1} = u_{i,1} + 2a\Delta y$. De même sur $x = 0$: $u_{-1,j} = u_{1,j} + 2a\Delta x$. Au coin $(0, 0)$, on utilise les deux.

Étape 5. Lien avec le cours. En reportant $u_{i,-1}$ dans l'équation à cinq points écrite en $(i, 0)$, on obtient $u_{i,1} - (1 + \frac{\Delta y^2}{\Delta x^2})u_{i,0} + \frac{\Delta y^2}{2\Delta x^2}(u_{i+1,0} + u_{i-1,0}) = \Delta y \frac{\partial u}{\partial y}(x_i, 0)$, c'est-à-dire exactement la formule (P4) du cours avec $\partial u / \partial y = -a$: les deux approches coïncident.



2)- Algorithme.

Algorithme : Gauss-Seidel sur le triangle

Données : l, h, a, N, ε

$\Delta x \leftarrow l/N; \Delta y \leftarrow h/N; \rho^2 \leftarrow (\Delta x/\Delta y)^2$

$u_{i,j} \leftarrow 0$ pour $i + j \leq N$

Répéter

$num \leftarrow 0; den \leftarrow 0$

Pour i allant de 0 à $N - 1$ **faire**

Pour j allant de 0 à $N - 1 - i$ **faire**

$u_O \leftarrow u_{i-1,j}$ si $i > 0$, sinon $u_{1,j} + 2a\Delta x$

$u_S \leftarrow u_{i,j-1}$ si $j > 0$, sinon $u_{i,1} + 2a\Delta y$

$t \leftarrow [u_{i+1,j} + u_O + \rho^2(u_{i,j+1} + u_S)] / (2(1 + \rho^2))$

$num \leftarrow num + |t - u_{i,j}|; den \leftarrow den + |t|; u_{i,j} \leftarrow t$

Fin pour

Fin pour

Jusqu'à $num/den < \varepsilon$

Écrire $u_{i,j}$

(Les valeurs $u_{i+1,j}$ et $u_{i,j+1}$ utilisées au voisinage de l'hypoténuse sont les zéros imposés.)

Vérification numérique. Pour $l = h$, la fonction $u = a(l - x - y)$ est harmonique, nulle sur l'hypoténuse, et vérifie $-\partial u/\partial y = -\partial u/\partial x = a$: c'est la solution exacte. Avec $a = 2, l = h = 1, N = 10$, l'algorithme la retrouve à 4×10^{-12} près (543 itérations), ce qui valide le traitement des points fictifs. (Pour $l \neq h$, il n'existe pas de solution affine, mais l'algorithme est le même.)

Références. Cours, chapitre VI (conditions de Neumann, formule (P4)); [BF, § 12.1, alg. 12.1, p. 738–739].

Bibliographie

- [BF] R. L. Burden, J. D. Faires, A. M. Burden, *Numerical Analysis*, 10^e édition, Cengage Learning, 2016. Les renvois donnent les numéros d’algorithmes, de théorèmes ou d’équations et les pages imprimées du livre.
- [Cours] P. Woaf, *Méthodes numériques pour physiciens et ingénieurs*, Université de Yaoundé I (transcription LaTeX dans le dossier latex/ de ce dépôt).
- [OEIS] The On-Line Encyclopedia of Integer Sequences, suite A094090 (solution positive de $5(1 - e^u) + ue^u = 0$, constante de la loi de Wien), <https://oeis.org/A094090>.
- [VdW] Purdue University, « Deviations from the Ideal Gas Law » (table des constantes de Van der Waals), <https://chemed.chem.purdue.edu/genchem/topicreview/bp/ch4/deviation.php>.
- [CSD] Équation de Carnahan-Starling pour les sphères dures : « Playing with Marbles : Structural and Thermodynamic Properties of Hard-Sphere Systems », <https://arxiv.org/abs/1310.5578>; équation de Carnahan-Starling-De Santis : <https://iifiir.org/en/fridoc/a-modified-carnahan-starling-de-santis-equation-of-state-to-compute-106555>.
- [CL] Loi de Child-Langmuir : « Child’s Law », SIMION, https://simion.com/info/childs_law.html; « A new approach to the Child-Langmuir law », <https://arxiv.org/abs/1506.07417>.
- [LA] F. M. S. Lima, P. Arun, « An accurate formula for the period of a simple pendulum oscillating beyond the small-angle regime », <https://arxiv.org/abs/physics/0510206> (formule $T = \frac{2T_0}{\pi} K(\sin \frac{\theta_0}{2})$).
- [TW] L. N. Trefethen, J. A. C. Weideman, « The exponentially convergent trapezoidal rule », *SIAM Review* 56(3), 2014, p. 385–458.
- [VN] « Von Neumann stability analysis », Wikipédia (anglais), https://en.wikipedia.org/wiki/Von_Neumann_stability_analysis (stabilité inconditionnelle du schéma de Dufort-Frankel).
- [DF2] « New analysis of the Du Fort-Frankel methods », *Journal of Scientific Computing*, 2012, doi :10.1007/s10915-012-9627-2, <https://cris.iucc.ac.il/en/publications/new-analysis-of-the-du-fort-frankel-methods/> (erreur de troncature $O(k^2 + h^2 + k^2/h^2)$).
- [HGG] A. C. Hindmarsh, P. M. Gresho, D. F. Griffiths, « The stability of explicit Euler time-integration for certain finite difference approximations of the multi-dimensional advection-diffusion equation », *International Journal for Numerical Methods in Fluids* 4(9), 1984, doi :10.1002/fld.1650040905.

Où trouver d’autres exercices corrigés

Les exercices du cours reprennent des exercices classiques. Le livre [BF] donne les réponses d’une sélection d’exercices à la fin de l’ouvrage (« Answers for Selected Exercises », p. 787 et suivantes). En français, l’ouvrage d’A. Quarteroni, F. Saleri et P. Gervasio, *Calcul scientifique : cours, exercices corrigés et illustrations en MATLAB et Octave* (Springer), traite les mêmes méthodes avec des exercices corrigés.